

# Bank Account System

Account opening, fund transfer and account statement for Horizon Bank, built on PostgreSQL, Redis and Apache Kafka. You choose the application layer: Spring Boot, Node.js or Python. The focus is the data layer: schema, transactions, locking, caching and events. Test it with Postman or curl.

POSTGRESQL 16

REDIS 7

APACHE KAFKA 4.3

DOCKER COMPOSE

REST API

SPRING BOOT

NODE.JS

PYTHON

[00 About This Document](#)[01 Project Overview](#)[02 Business Requirements](#)[03 Solution Architecture](#)[04 Environment Setup](#)[05 Data Layer — PostgreSQL](#)[06 Data Layer — Redis Cache](#)[07 Data Layer — Kafka Messaging](#)[08 API Specification](#)[09 Implementation Roadmap](#)[10 Testing & Verification](#)[11 Evaluation & Submission](#)[12 Trainer Guide](#)[A Reference Code Sketches](#)[B Glossary & References](#)

| DATE       | PARTICULARS           | DEBIT    | CREDIT    | BALANCE   |
|------------|-----------------------|----------|-----------|-----------|
| 01-09-2026 | Initial deposit       |          | 50,000.00 | 50,000.00 |
| 10-09-2026 | TRF to 501000000002   | 5,000.00 |           | 45,000.00 |
| 15-09-2026 | TRF from 501000000003 |          | 12,000.00 | 57,000.00 |
| 28-09-2026 | TRF to 501000000004   | 2,000.00 |           | 55,000.00 |

# 00 About This Document

## Document control

| Item                     | Detail                                                                                |
|--------------------------|---------------------------------------------------------------------------------------|
| <b>Document</b>          | Bank Account System: Capstone Project Case Study                                      |
| <b>Document ID</b>       | HB-CAP-BAS-001                                                                        |
| <b>Version</b>           | 1.0, released for training                                                            |
| <b>Date</b>              | 01 October 2026                                                                       |
| <b>Level</b>             | Beginner (0–2 years of development experience)                                        |
| <b>Duration</b>          | 5 days full-time (or 10 half-day sessions)                                            |
| <b>Application layer</b> | Participant's choice: Java Spring Boot, Node.js or Python                             |
| <b>Data layer</b>        | PostgreSQL 16 (data) · Redis 7 (cache) · Apache Kafka 4.3 (messaging)                 |
| <b>Client</b>            | Postman or curl. A UI is optional and earns no marks.                                 |
| <b>Organisation</b>      | Horizon Bank is a <b>fictional</b> bank. All names, PANs and numbers are sample data. |

## Revision history

| Version | Date        | Change                                                                                               |
|---------|-------------|------------------------------------------------------------------------------------------------------|
| 0.9     | 28-Sep-2026 | Draft for trainer review                                                                             |
| 1.0     | 01-Oct-2026 | Released. Starter kit verified on Docker 29; every captured output in this book comes from that run. |

## What this document is

This is a **training case study and a project specification in one**. It reads like the documents a real project team gets: a business context, requirements with acceptance criteria, an architecture, a data design, an API contract, a test plan and a rubric. It also teaches. Each data-layer chapter has hands-on labs that you run against the real PostgreSQL, Redis and Kafka containers **before** you write any application code.

The case study has three features:

1. **Account opening**: create a customer (if new) and an account with an initial deposit.
2. **Money transfer**: move money from one Horizon Bank account to another, safely.
3. **Account statement enquiry**: list the transactions of an account for a date range.

The focus is the **data layer**: table design, constraints, transactions, row locking, caching and event messaging. The application layer is the thinnest layer that exposes these as REST APIs.

## Who should read what

| Reader                  | Read                                 | Skim                                  |
|-------------------------|--------------------------------------|---------------------------------------|
| <b>Participant</b>      | 01–11, Appendix A when you are stuck | 12 (Trainer Guide) is for the trainer |
| <b>Trainer / mentor</b> | Everything, especially 11 and 12     | —                                     |

| Reader                      | Read               | Skim                                |
|-----------------------------|--------------------|-------------------------------------|
| <b>Reviewer / evaluator</b> | 01, 02, 08, 10, 11 | 05–07 for the data-design reasoning |

## Prerequisites

**Knowledge:** one programming language at a basic level (Java, JavaScript or Python), basic SQL (`SELECT`, `INSERT`, `UPDATE`, `JOIN`), what a REST API and JSON are, and how to run commands in a terminal. You do **not** need prior Redis or Kafka experience. The labs teach what the project needs.

### Software on your laptop:

| Tool                                                              | Version                           | Used for                                               |
|-------------------------------------------------------------------|-----------------------------------|--------------------------------------------------------|
| Docker Desktop (WSL 2 on Windows)                                 | 4.x, Engine 27+                   | PostgreSQL, Redis, Kafka containers                    |
| Git + Git Bash (Windows)                                          | any recent                        | source control, running the <code>bash</code> examples |
| <b>One</b> of: JDK 21 + Maven 3.9 · Node.js 22 LTS · Python 3.12+ | —                                 | the application layer                                  |
| IDE                                                               | IntelliJ IDEA / VS Code / PyCharm | coding                                                 |
| Postman (or curl)                                                 | any                               | calling the APIs                                       |
| DBeaver or pgAdmin (optional)                                     | any                               | browsing the database visually                         |

Hardware: 8 GB RAM minimum, 16 GB recommended. The four containers use about 1.5 GB together.

## Conventions used in this book

| You see                                                       | It means                                                                                                                                 |
|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| A block labelled <b>TERMINAL, SQL, REDIS-CLI, KAFKA CLI</b>   | something you type or run                                                                                                                |
| A block labelled <b>OUTPUT</b>                                | the real output captured when the starter kit was tested. Your IDs, UUIDs and timestamps will differ.                                    |
| A grey file name above a code block                           | that block is the complete file from the starter kit                                                                                     |
| <b>MUST / SHOULD / MAY</b> in requirements                    | mandatory / strongly recommended / optional (RFC 2119 meaning)                                                                           |
| <code>BR-07</code> , <code>NFR-03</code> , <code>TC-12</code> | business rule, non-functional requirement, test case IDs: used for traceability                                                          |
| A line ending in <code>\</code>                               | the command continues on the next line (bash / Git Bash). In PowerShell, type it on one line or end lines with a backtick <code>`</code> |

### ► YOUR TASK

A box like this is a hands-on step. Do it before reading on.

### TIP

Good practice or a shortcut.

**WATCH OUT**

A common mistake, or something that loses data or money if you get it wrong.

**TRAINER NOTE**

Notes for the trainer: timing, what to check, how to help.

## The starter kit

Everything this book uses is in the `starter_kit` folder next to it:

**LAYOUT**

```
starter_kit/
├── docker-compose.yml          PostgreSQL + Redis + Kafka + Kafka UI
├── db/
│   ├── 01_schema.sql          tables, constraints, indexes, the bank_app login
│   └── 02_seed.sql             3 customers, 5 accounts, 4 transfers (September 2026)
├── sql/
│   ├── 01_open_account.sql     02_transfer.sql          03_statement.sql
│   ├── 04_reconciliation.sql   05_constraints_say_no.sql
│   ├── 06_consumer_idempotency.sql
│   └── 07_race_demo.sh (+ 07_race_safe.pgbench, 07_race_unsafe.pgbench)
└── kafka/
    └── create-topics.sh         the three topics of chapter 07
```

The application code is **yours to write**. Appendix A has reference sketches of the hardest part (the transfer) in all three stacks.

# 01 Project Overview

## 1.1 Business context

**Horizon Bank** is a mid-sized retail bank with 140 branches. Today a customer opens an account at a branch, the officer types the details into an old desktop system, and transfers between Horizon accounts are batch-posted every evening. Statements are printed from the batch output.

The bank's Digital Channels programme is replacing this with an **API-first core accounts service**. The mobile app, the internet-banking site and the branch screens will all call the same service. Phase 1, which is this project, covers the three most-used journeys:

| Journey                             | Today                                        | Target                                       |
|-------------------------------------|----------------------------------------------|----------------------------------------------|
| Open an account                     | paper form, ~20 min, account number next day | one API call, account number at once         |
| Transfer to another Horizon account | posted overnight                             | posted in real time, balance updated at once |
| View a statement                    | printed at the branch                        | any date range, on demand, paginated         |

## 1.2 Problem statement

*Build the **Bank Account System** service that opens accounts, transfers money between accounts and returns account statements. Money must never be created, lost or double-counted, even when requests arrive in parallel, are retried by the client, or a component fails half-way. Account and statement reads must stay fast as traffic grows, and other bank systems (SMS alerts, fraud monitoring, analytics) must learn about every account and transfer event without slowing the transfer itself down.*

The last sentence maps onto the three data-layer technologies:

| Need                                                       | Technology          | Why this one                                     |
|------------------------------------------------------------|---------------------|--------------------------------------------------|
| Money never created, lost or double-counted                | <b>PostgreSQL</b>   | ACID transactions, row locks, constraints        |
| Reads stay fast as traffic grows                           | <b>Redis</b>        | in-memory key-value cache, sub-millisecond reads |
| Other systems learn about events without slowing transfers | <b>Apache Kafka</b> | durable event log, many independent consumers    |

## 1.3 Objectives

### Business objectives

- **BO-1** Open an account in a single request and return the account number immediately.
- **BO-2** Post internal transfers in real time with a unique, traceable reference number.
- **BO-3** Provide statements for any period of up to 90 days, with opening and closing balances.
- **BO-4** Notify the customer of every account opening, debit and credit.

### Learning objectives

When you finish this project you can:

| #    | Learning objective                                                                          | Chapter |
|------|---------------------------------------------------------------------------------------------|---------|
| LO-1 | Design a normalised relational schema with keys, constraints and indexes for a money domain | 05      |
| LO-2 | Store money correctly ( <code>NUMERIC</code> , decimal types, string amounts in JSON)       | 05, 08  |
| LO-3 | Write an ACID transaction that moves money, and explain isolation, row locks and deadlocks  | 05      |
| LO-4 | Prove data integrity with reconciliation queries                                            | 05, 10  |
| LO-5 | Apply the cache-aside pattern with TTLs and correct invalidation in Redis                   | 06      |
| LO-6 | Make a write API idempotent using Redis and a database unique constraint                    | 06      |
| LO-7 | Design Kafka topics, keys and event contracts; produce after commit; consume idempotently   | 07      |
| LO-8 | Expose it all as a clean REST API with validation and a standard error model                | 08      |
| LO-9 | Test functionally, under concurrency and at the data layer, and present the evidence        | 10      |

## 1.4 Scope

| In scope (MUST)                                              | Out of scope (do not build)                        |
|--------------------------------------------------------------|----------------------------------------------------|
| Open account (new or existing customer) with initial deposit | Login, authentication, authorisation (see note)    |
| Account enquiry (details and balance)                        | Any UI (optional, no marks)                        |
| Transfer between two Horizon Bank accounts                   | NEFT / IMPS / UPI / transfers to other banks       |
| Transfer enquiry by reference number                         | Cash deposit / withdrawal, cheques                 |
| Account statement for a date range, paginated                | Interest, fees, loans, cards                       |
| Kafka events + one consumer (notification-service)           | Real SMS / e-mail sending (log + table only)       |
| Redis caching + idempotency                                  | KYC document upload, account closure / freeze APIs |
| Docker Compose environment, test evidence                    | Multi-currency (INR only), cloud deployment        |

### NOTE

Security is out of scope so you can concentrate on the data layer. Do not forget it. Chapter 08 shows where an `Authorization` header would go, and NFR-07 still applies: no passwords in code, a least-privilege database login, and no full PAN in logs.

## 1.5 Assumptions and constraints

- **A-1** Every customer has a valid PAN, already verified by the branch. The system checks only its format.
- **A-2** Accounts in status `FROZEN` or `CLOSED` are set by the operations team directly in the database ( `UPDATE account SET status = ...` ). No API changes status in phase 1.
- **A-3** The bank works in **Indian Standard Time** (IST, `Asia/Kolkata` ). "Today", statement dates and daily limits use IST.
- **A-4** One service instance runs during the course. The design **MUST** still be correct with several instances, so no in-memory locks or counters.
- **C-1** PostgreSQL is the **only** system of record. Redis and Kafka may lose data without any money being lost.
- **C-2** All infrastructure runs in Docker on the participant's laptop using the starter kit.

## 1.6 Deliverables

| #   | Deliverable                                                                             | Format                                |
|-----|-----------------------------------------------------------------------------------------|---------------------------------------|
| D-1 | Source code of the service (and consumer)                                               | Git repository                        |
| D-2 | Database scripts (starter kit schema, plus any changes you make)                        | <code>.sql</code> files in the repo   |
| D-3 | <code>docker-compose.yml</code> that starts the whole data layer                        | in the repo                           |
| D-4 | Postman collection <b>or</b> curl script covering the test cases of chapter 10          | <code>.json</code> / <code>.sh</code> |
| D-5 | Test evidence: test-case sheet, reconciliation output, concurrency-test output          | Markdown or PDF                       |
| D-6 | <code>README.md</code> : how to run, configuration, design decisions, known limitations | Markdown                              |
| D-7 | 10-minute demo and viva                                                                 | live                                  |

## 1.7 Timeline: five days

| Day | Theme                         | Milestones (chapter 09)                    | End-of-day output                                                     |
|-----|-------------------------------|--------------------------------------------|-----------------------------------------------------------------------|
| 1   | Environment and data layer    | M0 Environment · M1 Database labs          | Stack running, all SQL labs done, schema understood                   |
| 2   | Project skeleton and accounts | M2 Skeleton · M3 Account opening & enquiry | <code>POST /accounts</code> , <code>GET /accounts/{no}</code> working |
| 3   | Money movement                | M4 Transfer · M5 Statement                 | Transfers ACID-safe, statement with paging                            |
| 4   | Cache and events              | M6 Redis · M7 Kafka                        | Cache hits visible, events on topics, consumer writing                |
| 5   | Proof                         | M8 Testing, concurrency, evidence          | Demo, test evidence, viva                                             |

## 1.8 Success criteria

The project succeeds when **all** of these are shown in the demo:

1. Every **MUST** test case in chapter 10 passes.
2. After the concurrency test (20 parallel transfers), the reconciliation script returns **zero rows** for R1–R3 and no balance is negative.
3. Repeating a transfer with the same `Idempotency-Key` returns the original response and moves no money.
4. A second `GET` of an account shows `X-Cache: HIT`; after a transfer it shows `MISS` with the new balance.
5. Every transfer produces one event on `bank.transfer.completed.v1`, and the consumer writes exactly two `notification_log` rows, even when the event is consumed twice.

## 02 Business Requirements

### 2.1 Personas

| Persona                    | Who                                                       | Wants                                                |
|----------------------------|-----------------------------------------------------------|------------------------------------------------------|
| <b>Customer</b>            | Retail customer using the app or a branch                 | Open an account quickly, move money, see a statement |
| <b>Branch officer</b>      | Uses the "Account Opening screen" (any client of the API) | One request that creates customer + account          |
| <b>Operations</b>          | Back-office team                                          | Traceable reference numbers, reconciled balances     |
| <b>Notification system</b> | Downstream system (SMS / e-mail gateway)                  | An event for every opening, debit and credit         |
| <b>Auditor</b>             | Internal audit                                            | An immutable ledger that explains every rupee        |

### 2.2 Feature map

| ID | Feature                                        | Endpoint                                       | Priority      |
|----|------------------------------------------------|------------------------------------------------|---------------|
| F1 | Account opening                                | POST /api/v1/accounts                          | MUST          |
| F2 | Account enquiry (details, balance)             | GET /api/v1/accounts/{accountNumber}           | MUST          |
| F3 | Money transfer                                 | POST /api/v1/transfers                         | MUST          |
| F4 | Transfer enquiry                               | GET /api/v1/transfers/{referenceNo}            | SHOULD        |
| F5 | Account statement enquiry                      | GET /api/v1/accounts/{accountNumber}/statement | MUST          |
| F6 | Notifications (async)                          | Kafka consumer notification-service            | MUST          |
| F7 | Record failed transfers + TransferFailed event | —                                              | MAY (stretch) |

### 2.3 User stories and acceptance criteria

#### US-01 · Open an account (F1)

*As a branch officer, I want to open an account for a customer with an initial deposit, so that the customer can start banking immediately.*

## ACCEPTANCE CRITERIA

**Feature:** Account opening**Scenario:** New customer opens a savings account**Given** no customer exists with PAN "CKRPR7788Q"**When** the officer opens a SAVINGS account for "Karthik Rao" with initial deposit "15000.00"**Then** the response is 201 Created with a 12-digit account number starting "5010"**And** the account status is ACTIVE and the balance is "15000.00"**And** the ledger has one CREDIT entry "Initial deposit" of "15000.00"**And** an AccountOpened event is published with the account number as key**Scenario:** Existing customer opens a second account of another type**Given** customer "Rahul Verma" (PAN "AKQPV5678L") has only a SAVINGS account**When** the officer opens a CURRENT account with PAN "AKQPV5678L"**Then** the new account belongs to the existing customer (no duplicate customer row)**Scenario:** Same account type twice is rejected**Given** customer "Priya Sharma" already has a SAVINGS account**When** the officer opens another SAVINGS account for her**Then** the response is 409 with error code DUPLICATE\_ACCOUNT\_TYPE**Scenario:** Initial deposit below the minimum**When** the officer opens a SAVINGS account with initial deposit "500.00"**Then** the response is 400 with error code VALIDATION\_ERROR on field "initialDeposit"**US-02 · View an account (F2)***As a customer, I want to see my account details and current balance.*

## ACCEPTANCE CRITERIA

**Scenario:** First and second read**When** the customer reads account "501000000001" twice**Then** both responses are 200 with the same body**And** the first has header X-Cache: MISS and the second X-Cache: HIT**Scenario:** Unknown account**When** the customer reads account "509999999999"**Then** the response is 404 with error code ACCOUNT\_NOT\_FOUND**US-03 · Transfer money (F3)***As a customer, I want to transfer money to another Horizon Bank account, so that the payee receives it at once.*

## ACCEPTANCE CRITERIA

**Feature:** Fund transfer**Scenario:** Successful transfer

**Given** account "501000000001" has balance "55000.00" and "501000000002" has "28500.00"  
**When** "2500.00" is transferred from "501000000001" to "501000000002" with a new Idempotency-Key  
**Then** the response is 201 with status COMPLETED and a reference number "TRF<yyyyMMdd><6 digits>"  
**And** the balances become "52500.00" and "31000.00"  
**And** the ledger has one DEBIT on the sender and one CREDIT on the receiver  
**And** one TransferCompleted event is published with the sender account number as key

**Scenario:** Client retries the same request

**Given** the transfer above succeeded with Idempotency-Key "K1"  
**When** the identical request is sent again with Idempotency-Key "K1"  
**Then** the response is the original 201 body with header Idempotent-Replayed: true  
**And** no money moves

**Scenario:** Not enough money

**Given** account "501000000004" has balance "13500.00"  
**When** "20000.00" is transferred from "501000000004"  
**Then** the response is 422 with error code INSUFFICIENT\_FUNDS and no balance changes

**Scenario:** Frozen account

**When** money is transferred to "501000000005" (status FROZEN)  
**Then** the response is 422 with error code ACCOUNT\_NOT\_ACTIVE

**Scenario:** Daily limit

**Given** a CURRENT account opened today with "1000000.00" has already sent "500000.00" today  
**When** "1.00" more is transferred from it  
**Then** the response is 422 with error code DAILY\_LIMIT\_EXCEEDED

## US-04 · Account statement (F5)

*As a customer, I want a statement for a date range, so that I can check every debit and credit.*

## ACCEPTANCE CRITERIA

**Scenario:** September statement

**When** the customer asks for the statement of "501000000001" from "2026-09-01" to "2026-09-30"  
**Then** the response shows opening balance "0.00" and closing balance "55000.00"  
**And** total credits "62000.00", total debits "7000.00", 4 transactions, newest first

**Scenario:** Paging

**When** the same statement is requested with page 0 and size 3  
**Then** 3 transactions are returned and totalPages is 2

**Scenario:** Range too long

**When** the customer asks for "2026-01-01" to "2026-09-30"  
**Then** the response is 400 with error code VALIDATION\_ERROR (maximum 90 days)

## US-05 · Notifications (F6)

*As a customer, I want an SMS for every debit and credit on my account.*

## ACCEPTANCE CRITERIA

**Scenario:** Debit and credit alerts

**When** a transfer of "2500.00" from "501000000001" to "501000000002" completes  
**Then** notification-service writes one row for each account in notification\_log

**Scenario:** Duplicate delivery

**When** the same TransferCompleted event is delivered twice  
**Then** notification\_log still has exactly two rows for that event

## 2.4 Business rules

The **Enforced in** column shows the layers that MUST enforce each rule. Money rules are enforced in more than one layer (*defence in depth*). The application gives a friendly error, and the database makes it impossible to break the rule even when the application has a bug.

| ID    | Rule                                                                                                                                                                                                                                     | Enforced in                                                                                              |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| BR-01 | A customer is identified by <b>PAN</b> , format <code>AAAAA9999A</code> , unique. Opening an account with an existing PAN links it to that customer; the request's other customer fields are ignored (profile updates are out of scope). | API, DB ( <code>ck_customer_pan</code> , <code>uq_customer_pan</code> )                                  |
| BR-02 | The customer MUST be at least <b>18 years</b> old on the opening date.                                                                                                                                                                   | API                                                                                                      |
| BR-03 | E-mail is unique and stored in lower case. Phone is a 10-digit Indian mobile number (starts 6–9), unique. A new PAN with an e-mail or phone that belongs to another customer is rejected ( <code>CUSTOMER_CONFLICT</code> ).             | API, DB ( <code>uq_customer_email</code> , <code>uq_customer_phone</code> , <code>ck_customer_*</code> ) |
| BR-04 | Account types are <code>SAVINGS</code> and <code>CURRENT</code> ; a customer holds <b>at most one of each</b> .                                                                                                                          | API, DB ( <code>uq_account_customer_type</code> )                                                        |
| BR-05 | Initial deposit: <code>SAVINGS</code> ≥ ₹1,000.00, <code>CURRENT</code> ≥ ₹5,000.00, maximum ₹10,00,000.00.                                                                                                                              | API                                                                                                      |
| BR-06 | Account number = <code>5010</code> + 8-digit sequence (12 digits), generated by the database, never reused.                                                                                                                              | DB (sequence + default)                                                                                  |
| BR-07 | Currency is <code>INR</code> only.                                                                                                                                                                                                       | DB ( <code>ck_account_currency</code> )                                                                  |
| BR-08 | A balance can <b>never be negative</b> (no overdraft).                                                                                                                                                                                   | Service, DB ( <code>ck_account_balance</code> )                                                          |
| BR-09 | Only <code>ACTIVE</code> accounts can send or receive money.                                                                                                                                                                             | Service                                                                                                  |
| BR-10 | Transfer amount: ₹1.00 to ₹2,00,000.00 per transfer, at most 2 decimal places.                                                                                                                                                           | API, DB ( <code>ck_transfer_amount</code> )                                                              |
| BR-11 | <b>Daily limit</b> : total completed transfers <i>out of</i> one account per IST calendar day ≤ ₹5,00,000.00.                                                                                                                            | Service (indexed query)                                                                                  |
| BR-12 | Sender and receiver accounts MUST differ.                                                                                                                                                                                                | API, DB ( <code>ck_transfer_accounts</code> )                                                            |
| BR-13 | Every transfer request carries an <code>Idempotency-Key</code> header (1–64 characters, UUID recommended). The same key never moves money twice.                                                                                         | Redis, DB ( <code>uq_transfer_idempotency</code> )                                                       |
| BR-14 | A transfer is <b>atomic</b> : the debit, the credit, the transfer row and both ledger entries commit together or not at all.                                                                                                             | DB transaction                                                                                           |
| BR-15 | Every balance change writes an <b>immutable ledger entry</b> ( <code>account_transaction</code> ) holding <code>balance_after</code> . Ledger rows are never updated or deleted.                                                         | Service, DB (privileges)                                                                                 |
| BR-16 | Statement: <code>fromDate ≤ toDate</code> , range ≤ <b>90 days</b> , <code>toDate</code> not in the future. Default range = last 30 days. Page size 1–100 (default 20). Newest first.                                                    | API                                                                                                      |
| BR-17 | A statement shows the opening balance, closing balance, total credits and total debits for the period.                                                                                                                                   | Service (SQL)                                                                                            |
| BR-18 | Reference numbers: <code>TRF</code> + <code>yyyyMMdd</code> + 6-digit sequence, unique. Gaps are allowed.                                                                                                                                | DB (sequence + default)                                                                                  |
| BR-19 | A cached balance is for <b>display only</b> . It is never used to approve a debit.                                                                                                                                                       | Service                                                                                                  |
| BR-20 | Customers are notified of every account opening, debit and credit, asynchronously, within seconds.                                                                                                                                       | Kafka consumer                                                                                           |

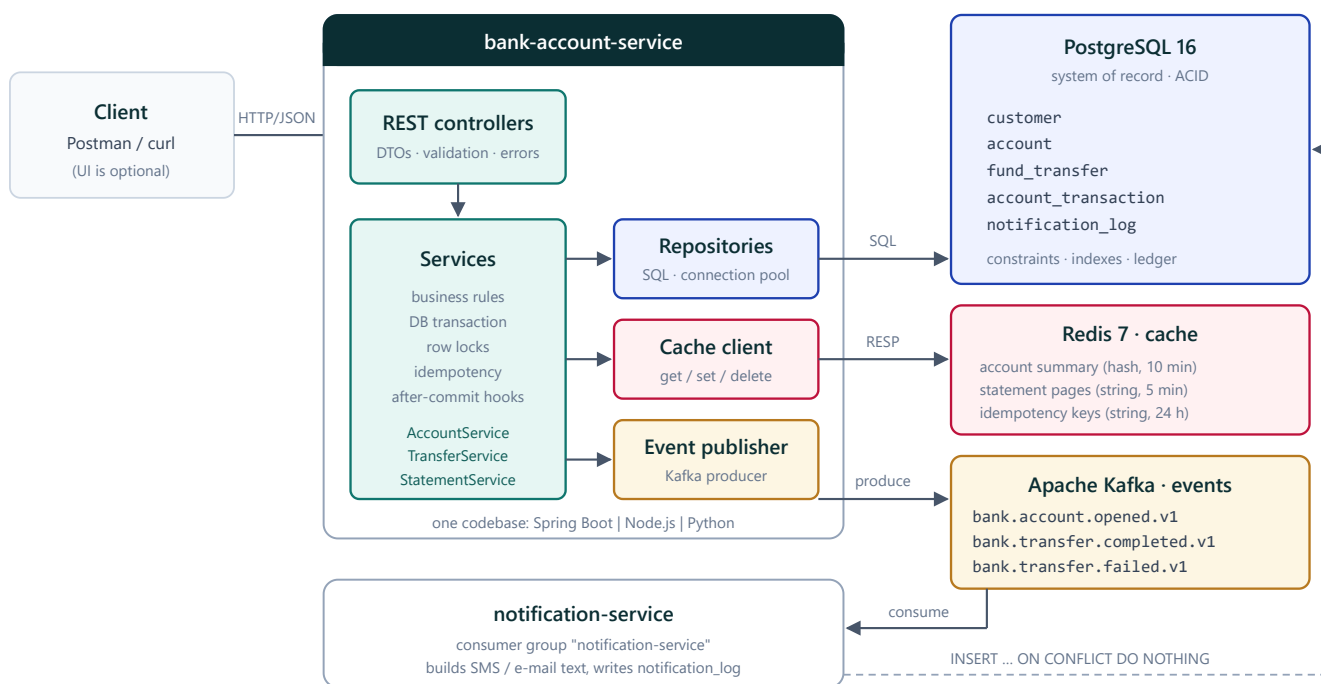
## 2.5 Non-functional requirements

| ID     | Category             | Requirement                                                                                                                                                                      | How it is verified              |
|--------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| NFR-01 | Consistency          | Balances are strongly consistent: a successful transfer is visible to the next read from the database at once.                                                                   | TC-10, TC-26                    |
| NFR-02 | Concurrency          | 20 parallel transfers from one account give no lost update and no negative balance. Total money is unchanged.                                                                    | TC-20 + reconciliation          |
| NFR-03 | Performance (laptop) | <code>GET</code> account p95 < 50 ms on a cache hit, < 200 ms on a miss. Transfer p95 < 300 ms.                                                                                  | Postman runner / timing in logs |
| NFR-04 | Resilience: cache    | If Redis is down, every API still works (slower). Idempotency falls back to the database unique key.                                                                             | TC-29                           |
| NFR-05 | Resilience: events   | If Kafka is down, transfers still succeed. The failed publish is logged with the event payload (an outbox removes this gap; stretch S-1).                                        | TC-30                           |
| NFR-06 | Auditability         | Every money movement is traceable: reference number → transfer row → two ledger rows → event.                                                                                    | Reconciliation R2               |
| NFR-07 | Security basics      | Secrets come from environment variables. The app connects as <code>bank_app</code> (no DDL, no DELETE). No full PAN in logs or responses.                                        | Code review                     |
| NFR-08 | Observability        | Each request logs a correlation ID ( <code>X-Correlation-Id</code> , generated if missing) at INFO, and errors log a stack trace. A health endpoint reports DB, Redis and Kafka. | Demo                            |
| NFR-09 | Maintainability      | Layered code (controller → service → repository); schema scripts in version control; no SQL in controllers.                                                                      | Code review                     |
| NFR-10 | Portability          | <code>docker compose up -d</code> plus one command to start the app is all a reviewer needs.                                                                                     | README walkthrough              |

## 03 Solution Architecture

### 3.1 Overview

The solution is a **modular monolith**: one deployable service (`bank-account-service`) exposes the REST API, and a separate consumer (`notification-service`) reacts to events. They can live in the same codebase as two run modes or two modules. A beginner-friendly design that still has the seams a real bank would later split into microservices.



**Figure 03.1** Components and the data layer. Solid arrows are synchronous calls; the dashed arrow is the consumer's write.

### 3.2 Components

| Component                      | Responsibility                                                                                           | Owns              |
|--------------------------------|----------------------------------------------------------------------------------------------------------|-------------------|
| <b>Client</b> (Postman / curl) | Calls the REST API, sends <code>Idempotency-Key</code> for transfers                                     | —                 |
| <b>REST controllers</b>        | HTTP mapping, request validation, DTO ↔ domain mapping, error responses                                  | no business logic |
| <b>Services</b>                | Business rules (BR-xx), transaction boundaries, locking order, cache and event calls <b>after commit</b> | the use cases     |
| <b>Repositories</b>            | SQL / ORM access to PostgreSQL through a connection pool                                                 | the queries       |
| <b>Cache client</b>            | Redis reads and writes. Every call is wrapped so a Redis failure never fails a request.                  | key naming, TTLs  |
| <b>Event publisher</b>         | Builds the event envelope and sends it to Kafka with the right key                                       | topic contracts   |
| <b>PostgreSQL</b>              | <b>System of record</b> : customers, accounts, transfers, ledger, notification log                       | all money data    |
| <b>Redis</b>                   | Cache for account summaries and statement pages; idempotency keys                                        | nothing permanent |
| <b>Kafka</b>                   | Durable, ordered log of business events                                                                  | events for 7 days |

| Component                   | Responsibility                                                                        | Owns              |
|-----------------------------|---------------------------------------------------------------------------------------|-------------------|
| <b>notification-service</b> | Consumes events, builds alert text, writes <code>notification_log</code> idempotently | notification data |

### 3.3 Technology stack

| Layer       | Technology                                     | Version (tested)                          | Why                                                                          |
|-------------|------------------------------------------------|-------------------------------------------|------------------------------------------------------------------------------|
| Database    | PostgreSQL                                     | 16 ( <code>postgres:16-alpine</code> )    | ACID, row locks, CHECK constraints, <code>NUMERIC</code> , mature tooling    |
| Cache       | Redis                                          | 7 ( <code>redis:7-alpine</code> )         | in-memory speed, TTL per key, atomic <code>SET NX</code> , <code>INCR</code> |
| Messaging   | Apache Kafka (KRaft, no ZooKeeper)             | 4.3.1 ( <code>apache/kafka:4.3.1</code> ) | durable partitioned log, consumer groups, replay                             |
| Kafka UI    | kafbat/kafka-ui                                | latest                                    | browse topics and messages at <code>http://localhost:8085</code>             |
| Containers  | Docker Compose                                 | v2                                        | one command starts the whole data layer                                      |
| Application | Spring Boot <b>or</b> Node.js <b>or</b> Python | see 3.4                                   | participant's choice                                                         |
| API testing | Postman / curl                                 | any                                       | no UI needed                                                                 |

### 3.4 Pick your application stack

All three stacks are equally acceptable. The data layer, API contract and tests are identical.

| Concern           | Java · Spring Boot                                                                            | Node.js                                                       | Python                                                          |
|-------------------|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------|-----------------------------------------------------------------|
| Runtime           | JDK 21, Spring Boot 4.x (3.5 also fine)                                                       | Node.js 22 LTS                                                | Python 3.12+                                                    |
| Web / REST        | <code>spring-boot-starter-webmvc</code>                                                       | Express 5 (or Fastify)                                        | FastAPI + Uvicorn                                               |
| Validation        | <code>spring-boot-starter-validation</code> (Jakarta Bean Validation)                         | zod                                                           | Pydantic v2 (built into FastAPI)                                |
| PostgreSQL access | Spring Data JPA (Hibernate) or <code>JdbcClient</code> + PostgreSQL JDBC driver               | <code>pg</code> (node-postgres) with <code>Pool</code>        | psycopg 3 ( <code>psycopg[binary,pool]</code> ) or SQLAlchemy 2 |
| Transactions      | <code>@Transactional</code>                                                                   | <code>BEGIN</code> / <code>COMMIT</code> on one pooled client | <code>with conn.transaction():</code>                           |
| Row lock          | <code>@Lock(PESSIMISTIC_WRITE)</code> or native <code>FOR UPDATE</code>                       | SQL <code>FOR UPDATE</code>                                   | SQL <code>FOR UPDATE</code>                                     |
| Money type        | <code>BigDecimal</code>                                                                       | keep as <b>string</b> ; let SQL do the arithmetic             | <code>decimal.Decimal</code>                                    |
| Redis             | Spring Data Redis ( <code>StringRedisTemplate</code> )                                        | <code>ioredis</code>                                          | <code>redis</code> (redis-py)                                   |
| Kafka producer    | Spring for Apache Kafka <code>KafkaTemplate</code> ( <code>spring-boot-starter-kafka</code> ) | <code>kafkajs</code>                                          | <code>confluent-kafka</code>                                    |
| Kafka consumer    | <code>@KafkaListener</code>                                                                   | <code>kafkajs</code> consumer                                 | <code>confluent_kafka.Consumer</code>                           |
| Config            | <code>application.yml</code> + env vars                                                       | <code>.env</code> + <code>dotenv</code>                       | <code>.env</code> + <code>pydantic-settings</code>              |

| Concern | Java · Spring Boot                                 | Node.js                        | Python                         |
|---------|----------------------------------------------------|--------------------------------|--------------------------------|
| Tests   | JUnit 5, Testcontainers                            | Jest / Vitest + supertest      | pytest + httpx                 |
| Health  | Spring Boot Actuator <code>/actuator/health</code> | own <code>/health</code> route | own <code>/health</code> route |

**WATCH OUT**

**Never use `float` / `double` / JavaScript `Number` arithmetic for money.** `0.1 + 0.2` is `0.30000000000000004` in all three languages. Use `NUMERIC(15,2)` in the database, `BigDecimal` / `Decimal` in code, and strings in JSON. Node.js has no built-in decimal type, so pass the amount to SQL as a string and let PostgreSQL do `balance - $1`. node-postgres already returns `NUMERIC` columns as strings for this reason.

### 3.5 Code structure

Whatever the language, keep the same layers. A Spring Boot example:

## LAYOUT

```

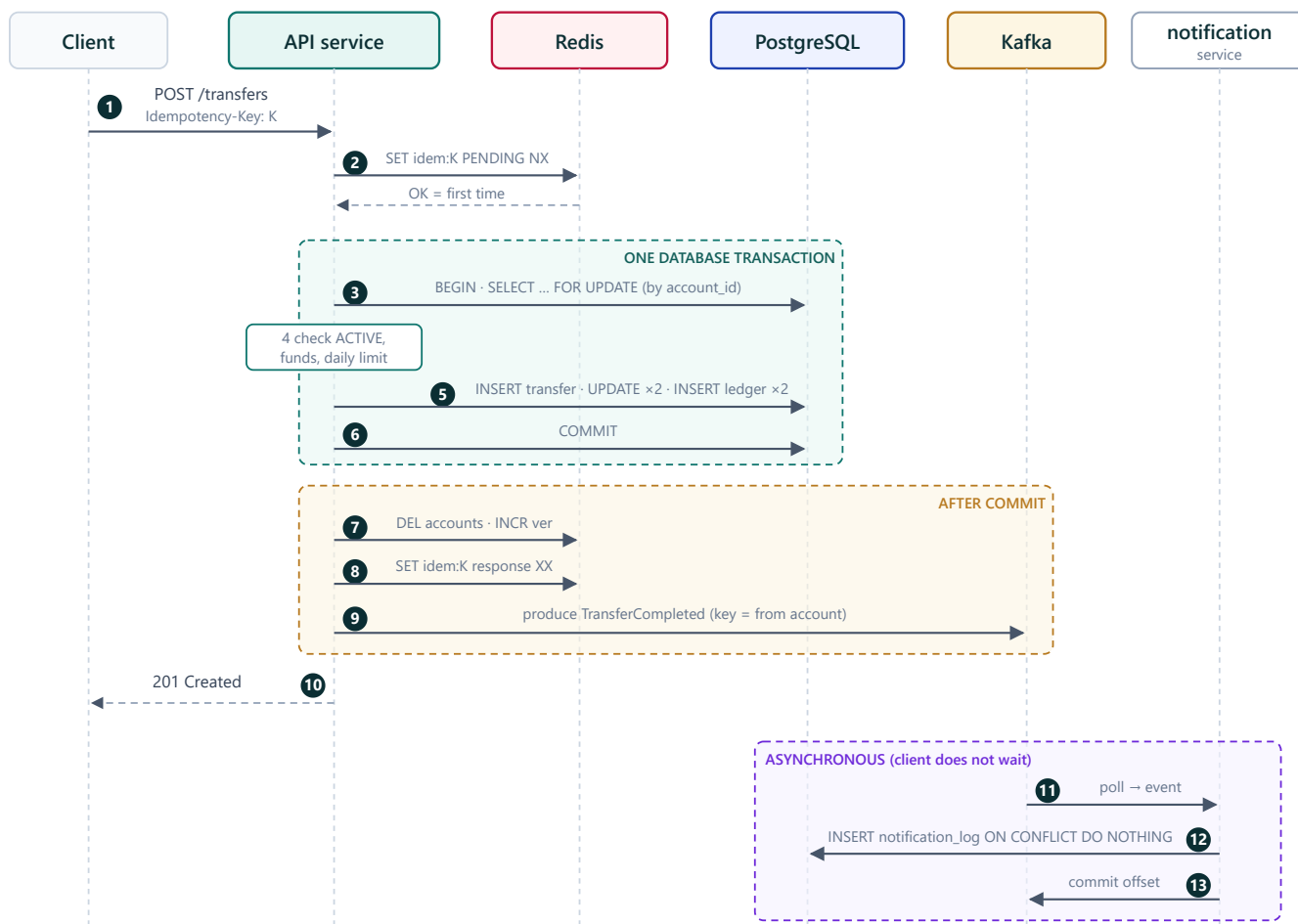
bank-account-service/
├── src/main/java/com/horizonbank/accounts/
│   ├── api/                AccountController, TransferController, StatementController,
│   │                        dto/ (requests, responses), GlobalExceptionHandler
│   ├── service/            AccountService, TransferService, StatementService, IdempotencyService
│   ├── domain/             Customer, Account, FundTransfer, AccountTransaction (entities)
│   ├── repository/         CustomerRepository, AccountRepository, TransferRepository, LedgerRepository
│   ├── cache/              AccountCache, StatementCache (key names + TTLs in one place)
│   ├── events/             EventEnvelope, TransferCompletedEvent, EventPublisher, AfterCommitListener
│   └── notification/       NotificationConsumer (@KafkaListener) - or a separate module
└── src/main/resources/    application.yml

```

Node.js: `src/routes`, `src/services`, `src/repositories`, `src/cache`, `src/events`, `src/consumer`. Python: `app/api`, `app/services`, `app/repositories`, `app/cache`, `app/events`, `app/consumer`.

## 3.6 Key flows

### Fund transfer: the critical path



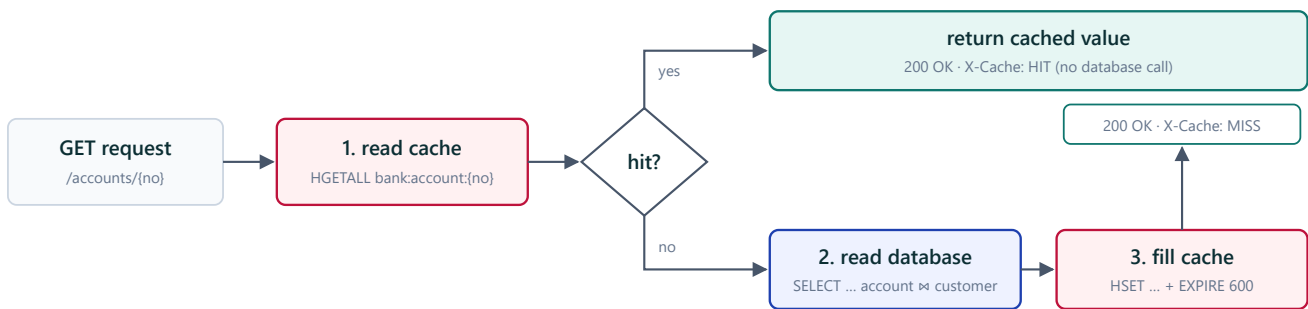
**Figure 03.2** Fund transfer. Steps 3–6 are one database transaction; Redis and Kafka are touched only before it starts or after it commits.

Three rules in this diagram decide whether your transfer is correct:

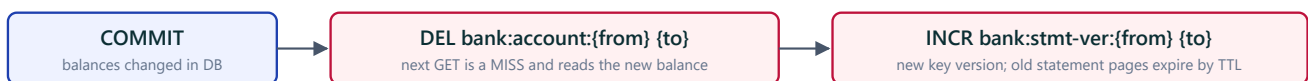
1. **Nothing slow or external inside the database transaction.** Calls to Redis or Kafka inside it hold row locks while waiting on the network, and if the transaction then rolls back, the event has already gone out for a transfer that never happened.
2. **Lock both accounts, always in the same order** (ascending `account_id`). Two transfers A→B and B→A then queue up instead of deadlocking.
3. **Validate after locking.** A balance you read *before* the lock may already be stale.

## Account and statement reads: cache-aside

READ PATH (GET account, GET statement)



WRITE PATH (after a transfer COMMITS)



**Figure 03.3** Cache-aside read path, and how a transfer invalidates cached data after it commits.

## 3.7 Architecture decisions

Real projects record important decisions as ADRs (Architecture Decision Records). These are the ones already made for you. You may challenge them in your README if you can justify it.

| ID     | Decision                                                                                                                     | Alternatives considered                                               | Reason                                                                                                                                      |
|--------|------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| ADR-01 | Keep a current <code>balance</code> column <b>and</b> an append-only ledger with <code>balance_after</code>                  | Compute the balance from the ledger on every read                     | Fast balance reads and a simple non-negative CHECK; the ledger provides the audit trail and statements; reconciliation proves the two agree |
| ADR-02 | <b>Pessimistic locking</b> ( <code>SELECT ... FOR UPDATE</code> ) in ascending <code>account_id</code> order, READ COMMITTED | Optimistic locking with <code>version</code> ; SERIALIZABLE isolation | Simplest correct option for beginners, no retry loop needed; optimistic locking is stretch S-4                                              |
| ADR-03 | Money as <code>NUMERIC(15,2)</code> ; strings in JSON                                                                        | integer parse ( <code>BIGINT</code> )                                 | Readable in SQL and JSON; exact arithmetic; the bank's 2-decimal convention                                                                 |
| ADR-04 | Cache-aside, <b>delete-on-write</b> for account summaries, <b>versioned keys</b> for statement pages                         | write-through; deleting keys by pattern                               | No stale balances after a transfer; <code>KEYS</code> /pattern delete is O(N) and blocks Redis                                              |
| ADR-05 | Idempotency key in Redis ( <code>SET NX EX</code> ) <b>plus</b> a unique constraint in PostgreSQL                            | Redis only; DB only                                                   | Redis answers retries fast; the DB constraint keeps money safe if Redis is flushed or down                                                  |
| ADR-06 | Publish events <b>after commit</b> ; an outbox table is stretch S-1                                                          | publish inside the transaction; transactional outbox                  | Never announces a rolled-back transfer; the known gap (crash between commit and publish) is documented                                      |
| ADR-07 | Business numbers (account, reference) from <b>database sequences</b>                                                         | UUIDs only; app-generated numbers                                     | Human-readable, unique across many app instances, no race                                                                                   |
| ADR-08 | One service + one consumer group (modular monolith)                                                                          | Full microservices                                                    | Right size for a 5-day beginner project; the seams (topics, <code>notification_log</code> ) allow a split later                             |

## 04 Environment Setup

### 4.1 The data layer in one file

The whole data layer (PostgreSQL, Redis, Kafka and a Kafka UI) starts from one Compose file. Your application runs **on your laptop** (from the IDE) and connects to the published ports.

📄 `starter_kit/docker-compose.yml`

YAML

```
# -----
# Horizon Bank | Bank Account System capstone | Local data Layer
#
# PostgreSQL 16 (system of record) + Redis 7 (cache) + Apache Kafka 4.3.1
# (events, single KRaft node) + Kafka UI. Your Spring Boot / Node.js /
# Python application runs on the Laptop and connects to the ports below.
#
# docker compose up -d          # start everything, wait for "healthy"
# docker compose ps            # check status
# docker compose logs -f kafka  # follow one service's log
# docker compose down          # stop, KEEP the data
# docker compose down -v       # stop and DELETE all data
#                               # (db/*.sql run again on the next start)
#
# From your Laptop      -> PostgreSQL localhost:5432  Redis localhost:6379
#                               Kafka      localhost:9092  Kafka UI http://localhost:8085
# From inside the network -> bank-postgres:5432  bank-redis:6379  bank-kafka:19092
# -----
name: bank-capstone

services:

  postgres:
    image: postgres:16-alpine
    container_name: bank-postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: bankdb
      POSTGRES_USER: bank_admin          # superuser: DDL, psql labs
      POSTGRES_PASSWORD: bank_admin_pwd  # Lab-only password
      TZ: Asia/Kolkata
      PGTZ: Asia/Kolkata
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./db:/docker-entrypoint-initdb.d:ro  # 01_schema.sql, 02_seed.sql - FIRST start only
      - ./sql:/sql:ro                        # Lab scripts, run with psql -f /sql/...
    healthcheck:
      # 127.0.0.1 (TCP), so the check stays red while the init scripts run
      test: ["CMD-SHELL", "pg_isready -h 127.0.0.1 -U bank_admin -d bankdb"]
      interval: 5s
      timeout: 5s
      retries: 20

  redis:
    image: redis:7-alpine
    container_name: bank-redis
    restart: unless-stopped
    # pure cache: no persistence, 128 MB cap, evict least-recently-used keys
    command: ["redis-server", "--save", "", "--appendonly", "no",
      "--maxmemory", "128mb", "--maxmemory-policy", "allkeys-lru"]
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
```

```

    interval: 5s
    timeout: 3s
    retries: 20

kafka:
  image: apache/kafka:4.3.1
  container_name: bank-kafka
  restart: unless-stopped
  ports:
    - "9092:9092"
  environment:
    # CLI tools on PATH: docker exec bank-kafka kafka-topics.sh ...
    PATH:
"/opt/kafka/bin:/opt/java/openjdk/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
    KAFKA_HEAP_OPTS: "-Xms256m -Xmx512m"
    # --- single node, broker + KRaft controller in one process (lab only)
    CLUSTER_ID: "bnkCapStone-4k3x9QpLwA"
    KAFKA_NODE_ID: "1"
    KAFKA_PROCESS_ROLES: "broker,controller"
    KAFKA_CONTROLLER_QUORUM_VOTERS: "1@bank-kafka:9093"
    KAFKA_CONTROLLER_LISTENER_NAMES: "CONTROLLER"
    # INTERNAL for containers, EXTERNAL (localhost:9092) for your app
    KAFKA_LISTENERS: "INTERNAL://:19092,CONTROLLER://:9093,EXTERNAL://:9092"
    KAFKA_ADVERTISED_LISTENERS: "INTERNAL://bank-kafka:19092,EXTERNAL://localhost:9092"
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: "CONTROLLER:PLAINTEXT,INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT"
    KAFKA_INTER_BROKER_LISTENER_NAME: "INTERNAL"
    # --- one broker, so every replication factor is 1
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: "1"
    KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: "1"
    KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: "1"
    KAFKA_NUM_PARTITIONS: "3"
    KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false" # topics are created on purpose
    KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: "0"
    KAFKA_LOG_DIRS: "/var/lib/kafka/data"
  volumes:
    - kafkadata:/var/lib/kafka/data
    - ./kafka:/scripts:ro # create-topics.sh
  healthcheck:
    test: ["CMD-SHELL", "/opt/kafka/bin/kafka-broker-api-versions.sh --bootstrap-server localhost:9092 > /dev/null 2>&1 || exit 1"]
    interval: 10s
    timeout: 10s
    retries: 12
    start_period: 20s

kafka-ui:
  image: kfbat/kafka-ui:latest
  container_name: bank-kafka-ui
  restart: unless-stopped
  depends_on:
    kafka:
      condition: service_healthy
  ports:
    - "8085:8080"
  environment:
    KAFKA_CLUSTERS_0_NAME: bank-local
    KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: bank-kafka:19092

volumes:
  pgdata:
  kafkadata:

```

Things to notice:

- **Container names** ( `bank-postgres` , `bank-redis` , `bank-kafka` ) are fixed, so every command in this book works as printed.

- **./db is mounted into /docker-entrypoint-initdb.d**. PostgreSQL runs those scripts *only when its data volume is empty*, which means on the first start. To rebuild the database from scratch, run `docker compose down -v`.
- **Redis is a pure cache**: no persistence, 128 MB maximum, least-recently-used keys evicted. A restart empties it, and the application MUST cope with that (C-1).
- **Kafka has two listeners**. Your app uses `localhost:9092` (EXTERNAL). Containers such as Kafka UI use `bank-kafka:19092` (INTERNAL). `auto.create.topics.enable` is off, so topics are created on purpose (Lab D3.1).

## 4.2 Start and verify

### ► YOUR TASK

Start the stack and wait until PostgreSQL, Redis and Kafka report `healthy`.

```
cd starter_kit
docker compose up -d
docker compose ps
```

TERMINAL

```
NAME                IMAGE                STATUS
bank-kafka          apache/kafka:4.3.1   Up 9 seconds (healthy)
bank-kafka-ui       kafbat/kafka-ui:latest Up 1 second
bank-postgres       postgres:16-alpine   Up 8 seconds (healthy)
bank-redis          redis:7-alpine       Up 8 seconds (healthy)
```

OUTPUT

Then check that each store answers:

```
docker exec bank-postgres pg_isready -U bank_app -d bankdb
docker exec bank-redis redis-cli ping
docker exec bank-kafka kafka-broker-api-versions.sh --bootstrap-server localhost:9092 | head -1
```

TERMINAL

```
> docker exec bank-postgres pg_isready -U bank_app -d bankdb
/var/run/postgresql:5432 - accepting connections
> docker exec bank-redis redis-cli ping
PONG
> docker exec bank-kafka kafka-broker-api-versions.sh --bootstrap-server localhost:9092 | head -1
localhost:9092 (id: 1 rack: null isFenced: false) -> (
```

OUTPUT

Open `http://localhost:8085` to see the Kafka UI (cluster `bank-local`).

## 4.3 Connection details

| Store      | From your application                                      | From another container          | Credentials (lab only)                                                                                                              |
|------------|------------------------------------------------------------|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| PostgreSQL | <code>localhost:5432</code> , database <code>bankdb</code> | <code>bank-postgres:5432</code> | <b>app:</b> <code>bank_app</code> / <code>bank_app_pwd</code> · <b>admin:</b> <code>bank_admin</code> / <code>bank_admin_pwd</code> |
| Redis      | <code>localhost:6379</code>                                | <code>bank-redis:6379</code>    | none                                                                                                                                |
| Kafka      | <code>localhost:9092</code>                                | <code>bank-kafka:19092</code>   | none (PLAINTEXT)                                                                                                                    |
| Kafka UI   | <code>http://localhost:8085</code>                         | —                               | none                                                                                                                                |

**WATCH OUT**

Your application connects as `bank_app`, never as `bank_admin`. `bank_app` cannot create or drop tables, cannot `DELETE` anything, and cannot even `UPDATE` the ledger. A bug or an injection attack can then do far less damage. Use `bank_admin` only for psql labs that need it, and with DBeaver if you want to browse.

Open a SQL prompt inside the container (no local PostgreSQL install needed):

TERMINAL

```
docker exec -it bank-postgres psql -U bank_app -d bankdb
```

## 4.4 Application configuration templates

Use one of these as the starting point for your project. Values come from environment variables with lab defaults, so nothing secret is hard-coded (NFR-07).

**Spring Boot:** `src/main/resources/application.yml`

YAML

```
server:
  port: 8080

spring:
  application:
    name: bank-account-service
  datasource:
    url: jdbc:postgresql://${DB_HOST:localhost}:5432/bankdb
    username: ${DB_USER:bank_app}
    password: ${DB_PASSWORD:bank_app_pwd}
    hikari:
      maximum-pool-size: 10          # 10 connections are plenty for a laptop
  jpa:
    hibernate:
      ddl-auto: validate             # schema comes from db/01_schema.sql - never let Hibernate change it
    open-in-view: false
  data:
    redis:
      host: ${REDIS_HOST:localhost}
      port: 6379
      timeout: 500ms                 # a slow cache must not slow the API down
  kafka:
    bootstrap-servers: ${KAFKA_BOOTSTRAP:localhost:9092}
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.apache.kafka.common.serialization.StringSerializer
      acks: all
      properties:
        enable.idempotence: true
        max.block.ms: 5000           # do not hang the request if Kafka is down
        delivery.timeout.ms: 10000
    consumer:
      group-id: notification-service
      auto-offset-reset: earliest
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.apache.kafka.common.serialization.StringDeserializer
    listener:
      ack-mode: record               # commit the offset after each record is processed
  jackson:
    time-zone: Asia/Kolkata

app:
  zone: Asia/Kolkata
  cache:
    account-ttl: 10m
```

```
statement-ttl: 5m
idempotency-ttl: 24h
topics:
  account-opened: bank.account.opened.v1
  transfer-completed: bank.transfer.completed.v1
  transfer-failed: bank.transfer.failed.v1
```

Node.js: `.env` (node-postgres reads the `PG*` variables by itself)

```
PORT=8080
PGHOST=localhost
PGPORT=5432
PGDATABASE=bankdb
PGUSER=bank_app
PGPASSWORD=bank_app_pwd
REDIS_URL=redis://localhost:6379
KAFKA_BROKERS=localhost:9092
KAFKA_CLIENT_ID=bank-account-service
APP_ZONE=Asia/Kolkata
```

`.ENV`

Python: `.env`

```
DATABASE_URL=postgresql://bank_app:bank_app_pwd@localhost:5432/bankdb
REDIS_URL=redis://localhost:6379/0
KAFKA_BOOTSTRAP_SERVERS=localhost:9092
APP_ZONE=Asia/Kolkata
```

`.ENV`

#### TIP

Add `.env` to `.gitignore` and commit a `.env.example` with the same keys. That is how real teams share configuration without sharing secrets.

## 4.5 Everyday commands

| Task                                           | Command                                                                                         |
|------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Start / stop (keep data)                       | <code>docker compose up -d</code> / <code>docker compose down</code>                            |
| <b>Reset everything</b> (re-run schema + seed) | <code>docker compose down -v</code> then <code>docker compose up -d</code>                      |
| Follow a log                                   | <code>docker compose logs -f kafka</code>                                                       |
| SQL prompt                                     | <code>docker exec -it bank-postgres psql -U bank_app -d bankdb</code>                           |
| Run a lab script                               | <code>docker exec -it bank-postgres psql -U bank_app -d bankdb -f /sql/02_transfer.sql</code>   |
| Redis prompt                                   | <code>docker exec -it bank-redis redis-cli</code>                                               |
| Empty the cache                                | <code>docker exec bank-redis redis-cli FLUSHALL</code>                                          |
| Kafka CLI                                      | <code>docker exec -it bank-kafka kafka-topics.sh --bootstrap-server localhost:9092 -list</code> |

## 4.6 Troubleshooting

| Symptom                                                                         | Cause                                                                           | Fix                                                                                                        |
|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <code>Bind for 0.0.0.0:5432 failed: port is already allocated</code>            | A local PostgreSQL (or another container) uses the port                         | Stop it, or change the left side of the mapping, e.g. <code>"5433:5432"</code> , and use 5433 in your app  |
| Tables missing / seed data missing                                              | The <code>pgdata</code> volume existed before, so the init scripts were skipped | <code>docker compose down -v</code> then <code>up -d</code>                                                |
| <code>psql: error: C:/Program Files/Git/sql/...: No such file</code>            | <b>Git Bash</b> rewrites paths that start with <code>/</code>                   | Run <code>export MSYS_NO_PATHCONV=1</code> first, or use PowerShell                                        |
| <code>sh: /scripts/create-topics.sh: line 2: \${'\r'}: command not found</code> | The <code>.sh</code> file was saved with Windows (CRLF) line endings            | Save it with LF endings (VS Code: bottom-right "CRLF" → "LF"); <code>git config core.autocrlf input</code> |
| <code>the input device is not a TTY</code> (Git Bash)                           | mintty cannot host <code>docker exec -it</code>                                 | Prefix the command with <code>winpty</code> , or use PowerShell / Windows Terminal                         |
| Kafka stays <code>starting</code> / <code>unhealthy</code>                      | Laptop short of memory, or first start still running                            | Wait 30–60 s; give Docker Desktop at least 4 GB RAM                                                        |
| App: <code>Connection to localhost:9092 ... could not be established</code>     | Kafka not healthy yet, or the app is inside a container                         | Wait for <code>healthy</code> ; from a container use <code>bank-kafka:19092</code>                         |
| App: <code>password authentication failed for user "bank_app"</code>            | Old volume from an earlier schema version                                       | <code>docker compose down -v</code> and start again                                                        |
| <code>UNKNOWN_TOPIC_OR_PARTITION</code> when producing                          | Topics not created (auto-create is off)                                         | Run Lab D3.1: <code>docker exec bank-kafka sh /scripts/create-topics.sh</code>                             |

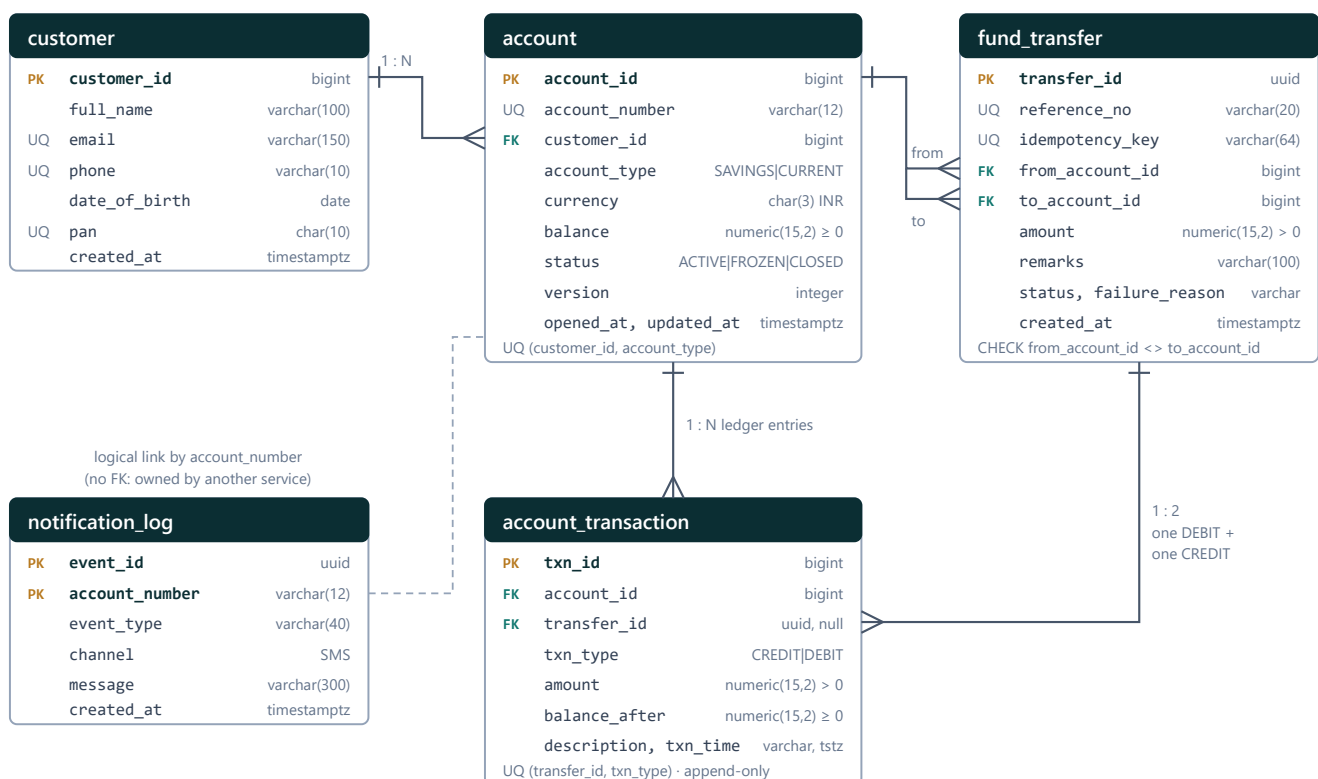
## 05 Data Layer — PostgreSQL

PostgreSQL is the **system of record**. If Redis and Kafka both disappeared, the bank would still know every balance and every transfer. This chapter explains the design, then you run each data-layer operation by hand before writing any application code.

### 5.1 Design principles

1. **Constraints are the last line of defence.** Validate in the API for friendly errors *and* declare the rule in the database ( `CHECK`, `UNIQUE`, `FOREIGN KEY` ) so a bug cannot break it.
2. **Money is `NUMERIC(15,2)`.** It is exact decimal arithmetic up to ₹9,99,99,99,99,999.99, never `REAL` / `DOUBLE`.
3. **Two kinds of keys.** A surrogate primary key ( `account_id`, a `BIGINT` identity) for joins, and a business key ( `account_number`, `reference_no` ) for people and APIs. Never expose the surrogate key.
4. **The ledger is append-only.** `account_transaction` rows are only ever inserted. Corrections would be new, reversing entries, never edits.
5. **Time is `TIMESTAMPZ`.** It stores an absolute instant. The bank's day boundaries (IST) are applied when you query, not when you store.
6. **The database generates business numbers** from sequences, so many app instances can never create the same account number.

### 5.2 Entity relationship diagram



**Figure 05.1** The five tables. PK = primary key, FK = foreign key, UQ = unique.

- A **customer** has one or two **accounts** (at most one SAVINGS and one CURRENT).
- A **fund\_transfer** references two accounts: `from_account_id` and `to_account_id`.

- Every completed transfer writes exactly **two ledger rows** in `account_transaction`: a DEBIT on the sender and a CREDIT on the receiver. The initial deposit is a ledger row with no transfer.
- **notification\_log** belongs to notification-service. It refers to accounts by number only, with no foreign key, because in a microservice split it would live in another database.

## 5.3 Data dictionary

### customer

| Column                     | Type                      | Null | Default / constraint                              | Description                               |
|----------------------------|---------------------------|------|---------------------------------------------------|-------------------------------------------|
| <code>customer_id</code>   | <code>BIGINT</code>       | no   | identity, <b>PK</b>                               | Surrogate key                             |
| <code>full_name</code>     | <code>VARCHAR(100)</code> | no   |                                                   | As on PAN card                            |
| <code>email</code>         | <code>VARCHAR(150)</code> | no   | <b>UQ</b> , must be lower case                    | Contact e-mail                            |
| <code>phone</code>         | <code>VARCHAR(10)</code>  | no   | <b>UQ</b> , <code>^[6-9][0-9]{9}\$</code>         | Indian mobile number                      |
| <code>date_of_birth</code> | <code>DATE</code>         | no   |                                                   | Used for the age $\geq 18$ check (BR-02)  |
| <code>pan</code>           | <code>CHAR(10)</code>     | no   | <b>UQ</b> , <code>^[A-Z]{5}[0-9]{4}[A-Z]\$</code> | Business identity of the customer (BR-01) |
| <code>created_at</code>    | <code>TIMESTAMPZ</code>   | no   | <code>now()</code>                                |                                           |

### account

| Column                                              | Type                       | Null | Default / constraint                                                     | Description                                             |
|-----------------------------------------------------|----------------------------|------|--------------------------------------------------------------------------|---------------------------------------------------------|
| <code>account_id</code>                             | <code>BIGINT</code>        | no   | identity, <b>PK</b>                                                      | Surrogate key, also the <b>lock order</b>               |
| <code>account_number</code>                         | <code>VARCHAR(12)</code>   | no   | <code>'5010'    lpad(nextval(seq), 8)</code> , <b>UQ</b>                 | Business key shown to customers (BR-06)                 |
| <code>customer_id</code>                            | <code>BIGINT</code>        | no   | <b>FK</b> $\rightarrow$ customer                                         | Owner                                                   |
| <code>account_type</code>                           | <code>VARCHAR(10)</code>   | no   | <code>SAVINGS / CURRENT</code> ; <b>UQ</b> with <code>customer_id</code> | BR-04                                                   |
| <code>currency</code>                               | <code>CHAR(3)</code>       | no   | <code>'INR'</code> , CHECK = <code>INR</code>                            | BR-07                                                   |
| <code>balance</code>                                | <code>NUMERIC(15,2)</code> | no   | <code>0</code> , CHECK $\geq 0$                                          | Current balance (BR-08)                                 |
| <code>status</code>                                 | <code>VARCHAR(10)</code>   | no   | <code>'ACTIVE'</code> ; <code>ACTIVE / FROZEN / CLOSED</code>            | BR-09                                                   |
| <code>version</code>                                | <code>INTEGER</code>       | no   | <code>0</code>                                                           | For optimistic locking (stretch S-4)                    |
| <code>opened_at</code> ,<br><code>updated_at</code> | <code>TIMESTAMPZ</code>    | no   | <code>now()</code>                                                       | <code>updated_at</code> changes on every balance update |

### fund\_transfer

| Column                    | Type                     | Null | Default / constraint                                                | Description                              |
|---------------------------|--------------------------|------|---------------------------------------------------------------------|------------------------------------------|
| <code>transfer_id</code>  | <code>UUID</code>        | no   | <code>gen_random_uuid()</code> , <b>PK</b>                          | Technical ID, also the event correlation |
| <code>reference_no</code> | <code>VARCHAR(20)</code> | no   | <code>'TRF'    yyyyMMdd    lpad(nextval(seq), 6)</code> , <b>UQ</b> | Customer-facing reference (BR-18)        |

| Column                       | Type                       | Null | Default / constraint                              | Description                                          |
|------------------------------|----------------------------|------|---------------------------------------------------|------------------------------------------------------|
| <code>idempotency_key</code> | <code>VARCHAR(64)</code>   | no   | <b>UQ</b>                                         | From the <code>Idempotency-Key</code> header (BR-13) |
| <code>from_account_id</code> | <code>BIGINT</code>        | no   | <b>FK</b> → account                               | Sender                                               |
| <code>to_account_id</code>   | <code>BIGINT</code>        | no   | <b>FK</b> → account; CHECK ≠ from                 | Receiver (BR-12)                                     |
| <code>amount</code>          | <code>NUMERIC(15,2)</code> | no   | CHECK > 0                                         | BR-10                                                |
| <code>remarks</code>         | <code>VARCHAR(100)</code>  | yes  |                                                   | Free text from the customer                          |
| <code>status</code>          | <code>VARCHAR(10)</code>   | no   | 'COMPLETED' ; COMPLETED / FAILED                  | FAILED only with stretch F7                          |
| <code>failure_reason</code>  | <code>VARCHAR(40)</code>   | yes  | required exactly when FAILED                      | e.g. INSUFFICIENT_FUNDS                              |
| <code>created_at</code>      | <code>TIMESTAMPZ</code>    | no   | now() ; indexed with <code>from_account_id</code> | Daily-limit query (BR-11)                            |

### account\_transaction (the ledger)

| Column                     | Type                       | Null | Default / constraint                                            | Description                                   |
|----------------------------|----------------------------|------|-----------------------------------------------------------------|-----------------------------------------------|
| <code>txn_id</code>        | <code>BIGINT</code>        | no   | identity, <b>PK</b>                                             | Also the tie-breaker for ordering             |
| <code>account_id</code>    | <code>BIGINT</code>        | no   | <b>FK</b> → account                                             | Whose statement this line belongs to          |
| <code>transfer_id</code>   | <code>UUID</code>          | yes  | <b>FK</b> → fund_transfer; <b>UQ</b> with <code>txn_type</code> | NULL for the initial deposit                  |
| <code>txn_type</code>      | <code>VARCHAR(6)</code>    | no   | CREDIT / DEBIT                                                  |                                               |
| <code>amount</code>        | <code>NUMERIC(15,2)</code> | no   | CHECK > 0                                                       | Always positive; the type gives the direction |
| <code>balance_after</code> | <code>NUMERIC(15,2)</code> | no   | CHECK ≥ 0                                                       | Running balance, shown on the statement       |
| <code>description</code>   | <code>VARCHAR(140)</code>  | no   |                                                                 | e.g. Transfer to 501000000002 - Rent October  |
| <code>txn_time</code>      | <code>TIMESTAMPZ</code>    | no   | now() ; indexed (account_id, txn_time DESC, txn_id DESC)        | Statement order and filter                    |

### notification\_log

| Column                      | Type                      | Null | Default / constraint                          | Description                           |
|-----------------------------|---------------------------|------|-----------------------------------------------|---------------------------------------|
| <code>event_id</code>       | <code>UUID</code>         | no   | <b>PK</b> (with <code>account_number</code> ) | From the Kafka event envelope         |
| <code>account_number</code> | <code>VARCHAR(12)</code>  | no   | <b>PK</b> (with <code>event_id</code> )       | One row per notified account          |
| <code>event_type</code>     | <code>VARCHAR(40)</code>  | no   |                                               | AccountOpened , TransferCompleted ... |
| <code>channel</code>        | <code>VARCHAR(10)</code>  | no   | 'SMS'                                         |                                       |
| <code>message</code>        | <code>VARCHAR(300)</code> | no   |                                               | The alert text                        |
| <code>created_at</code>     | <code>TIMESTAMPZ</code>   | no   | now()                                         |                                       |

## 5.4 The schema script

This is the complete script that ran when the container started. Read the comments: each constraint is named after the rule it protects, and those names show up in error messages.

📄 `starter_kit/db/01_schema.sql`

SQL

```
-- =====
-- Horizon Bank | Bank Account System capstone | PostgreSQL 16 schema
--
-- Runs automatically on the FIRST start of the bank-postgres container
-- (every file in /docker-entrypoint-initdb.d runs in name order).
-- To run it again: docker compose down -v then docker compose up -d
-- =====

-- Business dates are Indian dates: psql sessions show IST by default.
-- (Values are still stored as absolute instants in TIMESTAMPTZ.)
ALTER DATABASE bankdb SET timezone TO 'Asia/Kolkata';
SET timezone TO 'Asia/Kolkata';

-- -----
-- Sequences behind the human-readable business numbers
-- -----

CREATE SEQUENCE account_number_seq START 1; -- 501000000001, 501000000002 ...
CREATE SEQUENCE transfer_ref_seq START 1; -- TRF20261005000001 ...

-- -----
-- customer : one row per KYC-verified person, identified by PAN
-- -----

CREATE TABLE customer (
  customer_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL,
  phone VARCHAR(10) NOT NULL,
  date_of_birth DATE NOT NULL,
  pan CHAR(10) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_customer_pan UNIQUE (pan),
  CONSTRAINT uq_customer_email UNIQUE (email),
  CONSTRAINT uq_customer_phone UNIQUE (phone),
  CONSTRAINT ck_customer_pan CHECK (pan ~ '^[A-Z]{5}[0-9]{4}[A-Z]$'),
  CONSTRAINT ck_customer_phone CHECK (phone ~ '^[6-9][0-9]{9}$'),
  CONSTRAINT ck_customer_email CHECK (email = lower(email)) -- app stores lower case
);

-- -----
-- account : one row per bank account; balance is the current balance
-- -----

CREATE TABLE account (
  account_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  account_number VARCHAR(12) NOT NULL
    DEFAULT ('5010' || lpad(nextval('account_number_seq')::text, 8, '0')),
  customer_id BIGINT NOT NULL REFERENCES customer (customer_id),
  account_type VARCHAR(10) NOT NULL,
  currency CHAR(3) NOT NULL DEFAULT 'INR',
  balance NUMERIC(15,2) NOT NULL DEFAULT 0,
  status VARCHAR(10) NOT NULL DEFAULT 'ACTIVE',
  version INTEGER NOT NULL DEFAULT 0, -- for optimistic locking (stretch)
  opened_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_account_number UNIQUE (account_number),
  CONSTRAINT uq_account_customer_type UNIQUE (customer_id, account_type), -- 1 SAVINGS + 1 CURRENT max
  CONSTRAINT ck_account_type CHECK (account_type IN ('SAVINGS', 'CURRENT')),
  CONSTRAINT ck_account_currency CHECK (currency = 'INR'),
  CONSTRAINT ck_account_status CHECK (status IN ('ACTIVE', 'FROZEN', 'CLOSED')),
  CONSTRAINT ck_account_balance CHECK (balance >= 0) -- Last line of defence
);
```

```

-- -----
-- fund_transfer : one row per transfer request (the business event)
-- -----

CREATE TABLE fund_transfer (
  transfer_id      UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  reference_no     VARCHAR(20)   NOT NULL
                        DEFAULT ('TRF' || to_char(now(), 'YYYYMMDD')
                        || lpad(nextval('transfer_ref_seq')::text, 6, '0')),
  idempotency_key  VARCHAR(64)   NOT NULL,
  from_account_id  BIGINT        NOT NULL REFERENCES account (account_id),
  to_account_id    BIGINT        NOT NULL REFERENCES account (account_id),
  amount           NUMERIC(15,2) NOT NULL,
  remarks          VARCHAR(100),
  status           VARCHAR(10)   NOT NULL DEFAULT 'COMPLETED',
  failure_reason    VARCHAR(40),
  created_at       TIMESTAMPTZ   NOT NULL DEFAULT now(),
  CONSTRAINT uq_transfer_reference  UNIQUE (reference_no),
  CONSTRAINT uq_transfer_idempotency UNIQUE (idempotency_key), -- backstop if Redis Loses the key
  CONSTRAINT ck_transfer_accounts  CHECK (from_account_id <> to_account_id),
  CONSTRAINT ck_transfer_amount    CHECK (amount > 0),
  CONSTRAINT ck_transfer_status    CHECK (status IN ('COMPLETED', 'FAILED')),
  CONSTRAINT ck_transfer_failure   CHECK ((status = 'FAILED') = (failure_reason IS NOT NULL))
);

-- daily-limit query: today's completed transfers out of one account
CREATE INDEX ix_transfer_from_created ON fund_transfer (from_account_id, created_at);

-- -----
-- account_transaction : the ledger. Append-only; one row per balance
-- movement. Every transfer writes exactly one DEBIT and one CREDIT.
-- -----

CREATE TABLE account_transaction (
  txn_id          BIGINT          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  account_id      BIGINT          NOT NULL REFERENCES account (account_id),
  transfer_id     UUID            REFERENCES fund_transfer (transfer_id), -- NULL = initial deposit
  txn_type        VARCHAR(6)      NOT NULL,
  amount          NUMERIC(15,2)   NOT NULL,
  balance_after   NUMERIC(15,2)   NOT NULL,
  description     VARCHAR(140)    NOT NULL,
  txn_time        TIMESTAMPTZ     NOT NULL DEFAULT now(),
  CONSTRAINT ck_txn_type          CHECK (txn_type IN ('CREDIT', 'DEBIT')),
  CONSTRAINT ck_txn_amount        CHECK (amount > 0),
  CONSTRAINT ck_txn_balance       CHECK (balance_after >= 0),
  CONSTRAINT uq_txn_transfer_leg  UNIQUE (transfer_id, txn_type)         -- one DEBIT + one CREDIT
);

-- the statement query: one account, a date range, newest first
CREATE INDEX ix_txn_account_time ON account_transaction (account_id, txn_time DESC, txn_id DESC);

-- -----
-- notification_log : written by the Kafka consumer (notification-service).
-- The primary key makes the consumer idempotent: a re-delivered event
-- hits ON CONFLICT DO NOTHING instead of sending a second SMS.
-- -----

CREATE TABLE notification_log (
  event_id        UUID            NOT NULL,
  account_number  VARCHAR(12)     NOT NULL,
  event_type      VARCHAR(40)     NOT NULL,
  channel         VARCHAR(10)     NOT NULL DEFAULT 'SMS',
  message         VARCHAR(300)    NOT NULL,
  created_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
  CONSTRAINT pk_notification_log PRIMARY KEY (event_id, account_number)
);

-- -----
-- bank_app : the login your application uses. Least privilege:
-- no DDL, no DELETE anywhere, and the ledger cannot even be UPDATED.
-- -----

CREATE ROLE bank_app LOGIN PASSWORD 'bank_app_pwd'; -- Lab-only password

```

```
GRANT CONNECT ON DATABASE bankdb TO bank_app;
GRANT USAGE ON SCHEMA public TO bank_app;
GRANT SELECT, INSERT, UPDATE ON customer, account, fund_transfer TO bank_app;
GRANT SELECT, INSERT ON account_transaction, notification_log TO bank_app;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO bank_app;
```

**TIP**

Name every constraint ( `ck_...`, `uq_...`, `fk_...` ). When the database rejects a row, your code can turn `uq_account_customer_type` into the API error `DUPLICATE_ACCOUNT_TYPE` by looking at the constraint name in the exception, which is much more reliable than parsing the error text.

**The seed data**

`db/02_seed.sql` loads three customers, five accounts and four September transfers, with matching ledger rows, so statements have history from the first minute. Account `501000000005` is `FROZEN` for negative tests.

## 5.5 Lab D1: the data layer by hand

These labs use the same SQL your application will send. Run them in order, right after starting the stack. At the end you **reset** the database so the API tests start from clean seed data.

**Lab D1.1 · Explore****► YOUR TASK**

Open a psql prompt as `bank_app`, list the tables, and look at the seeded accounts.

**TERMINAL**

```
docker exec -it bank-postgres psql -U bank_app -d bankdb
```

**SQL**

```
\dt
SELECT a.account_number, c.full_name, a.account_type, a.balance, a.status
FROM account a JOIN customer c USING (customer_id) ORDER BY a.account_number;
```

**OUTPUT**

```

List of relations
Schema |      Name      | Type | Owner
-----+-----+-----+-----
public | account        | table | bank_admin
public | account_transaction | table | bank_admin
public | customer       | table | bank_admin
public | fund_transfer  | table | bank_admin
public | notification_log | table | bank_admin
(5 rows)
```

**OUTPUT**

```

account_number | full_name | account_type | balance | status
-----+-----+-----+-----+-----
501000000001   | Priya Sharma | SAVINGS      | 55000.00 | ACTIVE
501000000002   | Rahul Verma  | SAVINGS      | 28500.00 | ACTIVE
501000000003   | Rahul Verma  | CURRENT      | 88000.00 | ACTIVE
501000000004   | Ananya Iyer  | SAVINGS      | 13500.00 | ACTIVE
501000000005   | Ananya Iyer  | CURRENT      | 5000.00  | FROZEN
(5 rows)
```

## Lab D1.2 · Open an account

Account opening is **three inserts in one transaction**: the customer (if the PAN is new), the account, and the initial-deposit ledger entry. If any one fails, none of them stays.

📄 [starter\\_kit/sql/01\\_open\\_account.sql](#)

SQL

```
-- =====
-- Lab D1.2 | Open an account by hand - the SQL your API will run
--
-- docker exec -it bank-postgres psql -U bank_app -d bankdb -f /sql/01_open_account.sql
--
-- \gset stores the columns of a RETURNING row in psql variables
-- (:customer_id, :account_id ...), the way your code keeps the values
-- that INSERT ... RETURNING hands back.
-- =====

\set ON_ERROR_STOP on
\set amount 15000.00

BEGIN;

-- 1. new customer (PAN not seen before)
INSERT INTO customer (full_name, email, phone, date_of_birth, pan)
VALUES ('Karthik Rao', 'karthik.rao@example.com', '9876500004', '1995-02-14', 'CKRPR7788Q')
RETURNING customer_id \gset

-- 2. the account: number, currency, status, timestamps come from DEFAULTs
INSERT INTO account (customer_id, account_type, balance)
VALUES (:customer_id, 'SAVINGS', :amount)
RETURNING account_id, account_number \gset

-- 3. the initial deposit is the first ledger entry
INSERT INTO account_transaction (account_id, txn_type, amount, balance_after, description)
VALUES (:account_id, 'CREDIT', :amount, :amount, 'Initial deposit');

COMMIT;

SELECT a.account_number, c.full_name, a.account_type, a.balance, a.status
FROM account a JOIN customer c USING (customer_id)
WHERE a.account_id = :account_id;
```

TERMINAL

```
docker exec -it bank-postgres psql -U bank_app -d bankdb -f /sql/01_open_account.sql
```

OUTPUT

```
BEGIN
INSERT 0 1
INSERT 0 1
INSERT 0 1
COMMIT
 account_number | full_name | account_type | balance | status
-----+-----+-----+-----+-----
 501000000006   | Karthik Rao | SAVINGS      | 15000.00 | ACTIVE
(1 row)
```

The account number `501000000006`, the currency, the status and the timestamps all came from column defaults. The application never computes them.

### NOTE

In the application the customer step is an **upsert by PAN**: `INSERT INTO customer (...) VALUES (...) ON CONFLICT (pan) DO NOTHING RETURNING customer_id`. If it returns no row, the PAN already exists, so `SELECT customer_id FROM customer WHERE pan = ?`. A unique violation on `uq_customer_email` or `uq_customer_phone` means the details belong to another PAN, and the API returns `CUSTOMER_CONFLICT`.

## Lab D1.3 · Transfer money

This is the most important SQL in the project. Read every comment.

📄 [starter\\_kit/sql/02\\_transfer.sql](#)

SQL

```
-- =====
-- Lab D1.3 | One fund transfer, step by step - the SQL your API runs
--
-- docker exec -it bank-postgres psql -U bank_app -d bankdb -f /sql/02_transfer.sql
--
-- Transfer 2,500.00 from 501000000001 (Priya) to 501000000002 (Rahul).
-- The app does the checks in step 2 in code; here you read the rows.
-- =====
\set ON_ERROR_STOP on
\set from_acc '501000000001'
\set to_acc '501000000002'
\set amount 2500.00
\set idem_key 'lab-d1-3-0001'

BEGIN;

-- 1. Lock BOTH rows, always in account_id order. Two transfers that
-- touch the same pair of accounts then queue up instead of deadlocking.
SELECT account_id, account_number, balance, status
FROM account
WHERE account_number IN (:from_acc, :to_acc)
ORDER BY account_id
FOR UPDATE;

-- 2. Checks (in code): both rows found, both ACTIVE, balance >= amount,
-- and today's total + amount within the daily limit (5,00,000.00):
SELECT COALESCE(SUM(amount), 0.00) AS sent_today
FROM fund_transfer
WHERE from_account_id = (SELECT account_id FROM account WHERE account_number = :from_acc)
AND status = 'COMPLETED'
AND created_at >= date_trunc('day', now());

SELECT account_id AS from_id FROM account WHERE account_number = :from_acc \gset
SELECT account_id AS to_id FROM account WHERE account_number = :to_acc \gset

-- 3. The transfer row (reference_no comes from its DEFAULT)
INSERT INTO fund_transfer (idempotency_key, from_account_id, to_account_id, amount, remarks)
VALUES (:idem_key, :from_id, :to_id, :amount, 'Rent October')
RETURNING transfer_id, reference_no \gset

-- 4. Move the money. The database does the arithmetic, exactly.
UPDATE account SET balance = balance - :amount, updated_at = now()
WHERE account_id = :from_id
RETURNING balance AS from_balance \gset

UPDATE account SET balance = balance + :amount, updated_at = now()
WHERE account_id = :to_id
RETURNING balance AS to_balance \gset

-- 5. Two Ledger entries: the DEBIT and the CREDIT
INSERT INTO account_transaction (account_id, transfer_id, txn_type, amount, balance_after, description)
VALUES (:from_id, :transfer_id, 'DEBIT', :amount, :from_balance, 'Transfer to ' || :to_acc || ' -
Rent October'),
(:to_id, :transfer_id, 'CREDIT', :amount, :to_balance, 'Transfer from ' || :from_acc || ' -
Rent October');

COMMIT;

-- 6. AFTER the commit, the app evicts the cache and publishes the Kafka event.

SELECT reference_no, amount, status FROM fund_transfer WHERE transfer_id = :transfer_id;
SELECT account_number, balance FROM account WHERE account_id IN (:from_id, :to_id) ORDER BY account_id;
```

TERMINAL

```
docker exec -it bank-postgres psql -U bank_app -d bankdb -f /sql/02_transfer.sql
```

OUTPUT

```
BEGIN
account_id | account_number | balance | status
-----+-----+-----+-----
          1 | 501000000001  | 55000.00 | ACTIVE
          2 | 501000000002  | 28500.00 | ACTIVE
(2 rows)

sent_today
-----
      0.00
(1 row)

INSERT 0 1
UPDATE 1
UPDATE 1
INSERT 0 2
COMMIT
reference_no | amount | status
-----+-----+-----
TRF20261001000005 | 2500.00 | COMPLETED
(1 row)

account_number | balance
-----+-----
501000000001  | 52500.00
501000000002  | 31000.00
(2 rows)
```

What each step protects:

| Step   | SQL                                                                           | Protects against                                                                                                   |
|--------|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| 1      | <code>SELECT ... ORDER BY account_id FOR UPDATE</code>                        | Lost updates (two transfers reading the same balance) and deadlocks (lock order)                                   |
| 2      | Checks on the <b>locked</b> rows + daily total                                | Overdraft, inactive accounts, limit breach: decided on data nobody else can change now                             |
| 3      | <code>INSERT fund_transfer</code>                                             | Double processing: <code>uq_transfer_idempotency</code> rejects a replayed key                                     |
| 4      | <code>UPDATE ... SET balance = balance ± :amount ... RETURNING balance</code> | Arithmetic in the database is exact; <code>RETURNING</code> gives <code>balance_after</code> without another query |
| 5      | Two ledger <code>INSERT</code> s                                              | Missing audit trail: <code>uq_txn_transfer_leg</code> allows only one DEBIT and one CREDIT per transfer            |
| COMMIT |                                                                               | All of the above become visible together, or none (BR-14)                                                          |

## Lab D1.4 · The database says no

Every statement in this script breaks a business rule. Each one fails, which proves the rules hold even if your application code is wrong.

📄 [starter\\_kit/sql/05\\_constraints\\_say\\_no.sql](#)

SQL

```
-- =====
-- Lab D1.4 | The database says NO - each statement below breaks a
-- business rule and must fail. Bugs in the app cannot corrupt the data.
--
-- docker exec -it bank-postgres psql -U bank_app -d bankdb -f /sql/05_constraints_say_no.sql
-- =====

-- 1. overdraw an account (ck_account_balance)
UPDATE account SET balance = balance - 100000.00 WHERE account_number = '501000000004';

-- 2. a second SAVINGS account for Priya (uq_account_customer_type)
INSERT INTO account (customer_id, account_type, balance)
SELECT customer_id, 'SAVINGS', 1000.00 FROM customer WHERE pan = 'ABCPS1234K';

-- 3. transfer to the same account (ck_transfer_accounts)
INSERT INTO fund_transfer (idempotency_key, from_account_id, to_account_id, amount)
VALUES ('lab-d1-4-0003', 1, 1, 100.00);

-- 4. replay an idempotency key that was already used (uq_transfer_idempotency)
INSERT INTO fund_transfer (idempotency_key, from_account_id, to_account_id, amount)
VALUES ('seed-0001', 1, 2, 5000.00);

-- 5. a PAN in the wrong format (ck_customer_pan)
INSERT INTO customer (full_name, email, phone, date_of_birth, pan)
VALUES ('Test User', 'test.user@example.com', '9876500099', '1990-01-01', 'ABC123');

-- 6. rewrite history in the ledger (no UPDATE/DELETE privilege for bank_app)
DELETE FROM account_transaction WHERE txn_id = 1;
```

OUTPUT

```
psql:/sql/05_constraints_say_no.sql:9: ERROR: new row for relation "account" violates check constraint "ck_account_balance"
DETAIL: Failing row contains (4, 501000000004, 3, SAVINGS, INR, -86500.00, ACTIVE, 0, 2026-09-05 09:15:00+05:30, 2026-09-28 12:10:00+05:30).
psql:/sql/05_constraints_say_no.sql:13: ERROR: duplicate key value violates unique constraint "uq_account_customer_type"
DETAIL: Key (customer_id, account_type)=(1, SAVINGS) already exists.
psql:/sql/05_constraints_say_no.sql:17: ERROR: new row for relation "fund_transfer" violates check constraint "ck_transfer_accounts"
DETAIL: Failing row contains (8e2984c7-04f3-4f30-ae07-85f10be63803, TRF20261001000006, lab-d1-4-0003, 1, 1, 100.00, null, COMPLETED, null, 2026-10-01 10:41:30.32217+05:30).
psql:/sql/05_constraints_say_no.sql:21: ERROR: duplicate key value violates unique constraint "uq_transfer_idempotency"
DETAIL: Key (idempotency_key)=(seed-0001) already exists.
psql:/sql/05_constraints_say_no.sql:25: ERROR: new row for relation "customer" violates check constraint "ck_customer_pan"
DETAIL: Failing row contains (5, Test User, test.user@example.com, 9876500099, 1990-01-01, ABC123, 2026-10-01 10:41:30.329676+05:30).
psql:/sql/05_constraints_say_no.sql:28: ERROR: permission denied for table account_transaction
```

Statement 3 failed, but the reference number `TRF20261001000006` was still consumed. **Sequences never roll back**, so reference numbers are unique but may have gaps (BR-18). Banks accept that, and auditors know it.

## Lab D1.5 · Statement queries

A statement needs four things: a page of transactions, paging metadata, period totals, and opening and closing balances.

📁 `starter_kit/sql/03_statement.sql`

SQL

```
-- =====
-- Lab D1.5 | Account statement queries
--
-- docker exec -it bank-postgres psql -U bank_app -d bankdb -f /sql/03_statement.sql
--
-- Statement for 501000000001, 1-30 September 2026, page 0, size 3.
-- The date range is half-open: from 00:00 IST on fromDate up to (not
-- including) 00:00 IST on the day AFTER toDate. Your code computes the
-- two instants; here they are written out with the +05:30 offset.
-- =====

\set ON_ERROR_STOP on
\set acc      '501000000001'
\set from_ts  '2026-09-01 00:00:00+05:30'
\set to_ts    '2026-10-01 00:00:00+05:30'
\set size     3
```

```

\set page 0

SELECT account_id AS acc_id FROM account WHERE account_number = :'acc' \gset

-- Q1. one page of transactions, newest first (txn_id breaks ties)
SELECT t.txn_id,
       to_char(t.txn_time, 'YYYY-MM-DD HH24:MI') AS txn_time,
       t.txn_type, t.amount, t.balance_after, t.description,
       ft.reference_no
FROM account_transaction t
LEFT JOIN fund_transfer ft ON ft.transfer_id = t.transfer_id
WHERE t.account_id = :acc_id
      AND t.txn_time >= :'from_ts' AND t.txn_time < :'to_ts'
ORDER BY t.txn_time DESC, t.txn_id DESC
LIMIT :size OFFSET (:page * :size);

-- Q2. paging metadata and period totals in one pass
SELECT count(*) AS total_elements,
       ceil(count(*)::numeric / :size) AS total_pages,
       COALESCE(SUM(amount) FILTER (WHERE txn_type = 'CREDIT'), 0) AS total_credits,
       COALESCE(SUM(amount) FILTER (WHERE txn_type = 'DEBIT'), 0) AS total_debits
FROM account_transaction
WHERE account_id = :acc_id
      AND txn_time >= :'from_ts' AND txn_time < :'to_ts';

-- Q3. opening balance = balance after the last entry BEFORE the period (0 if none)
--      closing balance = balance after the last entry INSIDE or before the period
SELECT COALESCE((SELECT balance_after FROM account_transaction
                  WHERE account_id = :acc_id AND txn_time < :'from_ts'
                  ORDER BY txn_time DESC, txn_id DESC LIMIT 1), 0.00) AS opening_balance,
       COALESCE((SELECT balance_after FROM account_transaction
                  WHERE account_id = :acc_id AND txn_time < :'to_ts'
                  ORDER BY txn_time DESC, txn_id DESC LIMIT 1), 0.00) AS closing_balance;

-- Q4. proof that the statement query uses the index, not a full scan
SET enable_seqscan = off; -- tiny table: force the planner to show the index path
EXPLAIN (COSTS OFF)
SELECT txn_id FROM account_transaction
WHERE account_id = :acc_id AND txn_time >= :'from_ts' AND txn_time < :'to_ts'
ORDER BY txn_time DESC, txn_id DESC LIMIT 3;

```

## TERMINAL

```
docker exec -it bank-postgres psql -U bank_app -d bankdb -f /sql/03_statement.sql
```

## OUTPUT

| txn_id | txn_time         | txn_type | amount   | balance_after | description                               | reference_no      |
|--------|------------------|----------|----------|---------------|-------------------------------------------|-------------------|
| 12     | 2026-09-28 12:10 | DEBIT    | 2000.00  | 55000.00      | Transfer to 501000000004 - Birthday gift  | TRF20260928000004 |
| 9      | 2026-09-15 14:00 | CREDIT   | 12000.00 | 57000.00      | Transfer from 501000000003 - Invoice 1043 | TRF20260915000002 |
| 6      | 2026-09-10 09:30 | DEBIT    | 5000.00  | 45000.00      | Transfer to 501000000002 - Rent share     | TRF20260910000001 |

(3 rows)

| total_elements | total_pages | total_credits | total_debits |
|----------------|-------------|---------------|--------------|
| 4              | 2           | 62000.00      | 7000.00      |

(1 row)

| opening_balance | closing_balance |
|-----------------|-----------------|
| 0.00            | 55000.00        |

(1 row)

SET

## QUERY PLAN

```

Limit
-> Index Only Scan using ix_txn_account_time on account_transaction
    Index Cond: ((account_id = 1) AND (txn_time >= '2026-09-01 00:00:00+05:30'::timestamp with time zone) AND (txn_time <
'2026-10-01 00:00:00+05:30'::timestamp with time zone))
(3 rows)

```

Change the two timestamps to October ( `2026-10-01 ... 2026-11-01` ) and the opening balance becomes September's closing balance, while the closing balance includes Lab D1.3's transfer:

| opening_balance | closing_balance |
|-----------------|-----------------|
| 55000.00        | 52500.00        |
| (1 row)         |                 |

OUTPUT

The plan in Q4 says **Index Only Scan using ix\_txn\_account\_time**: the composite index ( `account_id, txn_time DESC, txn_id DESC` ) filters the account, the date range and the sort order in one pass. Without it, a statement for one customer would read the whole ledger of the bank.

#### WATCH OUT

**Use half-open date ranges.** `txn_time >= from AND txn_time < (toDate + 1 day)` includes every instant of the last day. `BETWEEN '2026-09-01' AND '2026-09-30'` silently drops everything after midnight on the 30th. Compute both boundaries at 00:00 IST in your code ( `LocalDate.atStartOfDay(ZoneId.of("Asia/Kolkata"))` and its equivalents) and pass them as `TIMESTAMPZ` parameters.

## Lab D1.6 · Reconciliation

Reconciliation queries check **invariants**, rules that must always be true whatever happened before. Banks run them every night. You run them after every test session.

📄 [starter\\_kit/sql/04\\_reconciliation.sql](#)

SQL

```
-- =====
-- Lab D1.6 | Reconciliation - checks that must ALWAYS hold.
-- Run them after every test session; R1-R3 must return zero rows.
--
-- docker exec -it bank-postgres psql -U bank_app -d bankdb -f /sql/04_reconciliation.sql
-- =====

-- R1. account.balance = credits - debits in the Ledger = last balance_after
SELECT a.account_number, a.balance,
       SUM(CASE t.txn_type WHEN 'CREDIT' THEN t.amount ELSE -t.amount END) AS ledger_sum,
       (ARRAY_AGG(t.balance_after ORDER BY t.txn_time DESC, t.txn_id DESC))[1] AS last_balance_after
FROM account a
JOIN account_transaction t USING (account_id)
GROUP BY a.account_id, a.account_number, a.balance
HAVING a.balance <> SUM(CASE t.txn_type WHEN 'CREDIT' THEN t.amount ELSE -t.amount END)
      OR a.balance <> (ARRAY_AGG(t.balance_after ORDER BY t.txn_time DESC, t.txn_id DESC))[1];

-- R2. every COMPLETED transfer has exactly one DEBIT on the sender and
--      one CREDIT on the receiver, both for the transfer amount
SELECT ft.reference_no
FROM fund_transfer ft
LEFT JOIN account_transaction d ON d.transfer_id = ft.transfer_id AND d.txn_type = 'DEBIT'
LEFT JOIN account_transaction c ON c.transfer_id = ft.transfer_id AND c.txn_type = 'CREDIT'
WHERE ft.status = 'COMPLETED'
      AND (d.account_id IS DISTINCT FROM ft.from_account_id OR d.amount IS DISTINCT FROM ft.amount
           OR c.account_id IS DISTINCT FROM ft.to_account_id OR c.amount IS DISTINCT FROM ft.amount);

-- R3. transfers move money, they never create it:
--      total balances must equal total initial deposits
SELECT total_balances, total_deposits
FROM (SELECT SUM(balance) AS total_balances FROM account) b,
     (SELECT SUM(amount) AS total_deposits FROM account_transaction WHERE transfer_id IS NULL) d
WHERE total_balances <> total_deposits;

-- R4. summary for the test report
SELECT (SELECT count(*) FROM account) AS accounts,
```

```
(SELECT count(*) FROM fund_transfer WHERE status = 'COMPLETED') AS transfers,
(SELECT count(*) FROM account_transaction) AS ledger_rows,
(SELECT SUM(balance) FROM account) AS money_in_bank;
```

OUTPUT

```
account_number | balance | ledger_sum | last_balance_after
-----+-----+-----+-----
(0 rows)

reference_no
-----
(0 rows)

total_balances | total_deposits
-----+-----
(0 rows)

accounts | transfers | ledger_rows | money_in_bank
-----+-----+-----+-----
6 | 5 | 16 | 205000.00
(1 row)
```

R1–R3 return **zero rows**: every balance matches its ledger, every transfer has its two legs, and no money was created. The total of ₹2,05,000.00 is the seed's ₹1,90,000.00 plus Karthik's ₹15,000.00 deposit, so the transfer moved money without creating any.

### Lab D1.7 · Why we lock

This lab shows the bug that row locks prevent. Ten sessions each debit ₹100.00 twenty times from a wallet holding ₹1,00,000.00, so the result must be ₹80,000.00. `pgbench` (shipped with PostgreSQL) runs the sessions in parallel.

📄 `starter_kit/sql/07_race_unsafe.pgbench`

SQL

```
-- read-modify-write WITHOUT a lock: two sessions can read the same balance
BEGIN;
SELECT balance AS bal FROM demo_wallet WHERE id = 1 \gset
UPDATE demo_wallet SET balance = :bal - 100 WHERE id = 1;
COMMIT;
```

📄 `starter_kit/sql/07_race_safe.pgbench`

SQL

```
-- the same read-modify-write, but FOR UPDATE makes the others wait their turn
BEGIN;
SELECT balance AS bal FROM demo_wallet WHERE id = 1 FOR UPDATE \gset
UPDATE demo_wallet SET balance = :bal - 100 WHERE id = 1;
COMMIT;
```

TERMINAL

```
docker exec bank-postgres sh /sql/07_race_demo.sh
```

OUTPUT

```
unsafe: expected 80000.00, got 98000.00
safe: expected 80000.00, got 80000.00
```

Without the lock, many debits were **lost**: two sessions read the same balance, both subtract 100, and the second write overwrites the first. With `FOR UPDATE`, the second session waits until the first commits, then reads the new balance. Your number for the unsafe run will differ on every run. That randomness is exactly what makes race conditions dangerous: they pass most tests.

## ► YOUR TASK

**Two-terminal lock exercise.** Open two psql prompts (A and B) as `bank_app`.

1. **A:** `BEGIN; SELECT balance FROM account WHERE account_number = '50100000003' FOR UPDATE;`
2. **B:** `BEGIN; UPDATE account SET balance = balance - 1 WHERE account_number = '50100000003';` and notice that **B hangs**: it is waiting for A's row lock.
3. **A:** `ROLLBACK;` and B's `UPDATE` finishes immediately.
4. **B:** `ROLLBACK;` so the balance still matches the ledger.

Now try a deadlock: A locks `...001` then `...002`, while B locks `...002` then `...001` (each with `SELECT ... FOR UPDATE`, one row at a time). Within a second PostgreSQL detects the cycle and aborts one session with `ERROR: deadlock detected`. Locking in ascending `account_id` order makes this impossible.

## ► YOUR TASK

**Reset before the API work.** The labs changed balances. Start the API milestones from clean seed data: `docker compose down -v` then `docker compose up -d`.

## 5.6 Transaction design for your application

| Topic                          | Rule for this project                                                                                                                                                                                                                                                            |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Isolation level</b>         | Default <b>READ COMMITTED</b> plus explicit row locks. Do not switch to <b>SERIALIZABLE</b> ; it would need a retry loop for serialization failures.                                                                                                                             |
| <b>Transaction boundary</b>    | Exactly one transaction per use case, opened in the <b>service</b> layer ( <code>@Transactional</code> on the service method, <code>BEGIN</code> / <code>COMMIT</code> around the service function). Never in the controller.                                                    |
| <b>What goes inside</b>        | SQL only. No Redis calls, no Kafka sends, no HTTP calls, no <code>Thread.sleep</code> .                                                                                                                                                                                          |
| <b>Lock order</b>              | <code>SELECT ... WHERE account_number IN (?, ?) ORDER BY account_id FOR UPDATE</code> : both rows in one statement, ascending ID.                                                                                                                                                |
| <b>Lock timeout</b>            | <code>SET LOCAL lock_timeout = '3s'</code> at the start of the transfer transaction. A request that cannot get the lock in 3 s fails with a clear error instead of piling up.                                                                                                    |
| <b>Deadlock / lock timeout</b> | Map SQLSTATE <code>40P01</code> (deadlock) and <code>55P03</code> (lock not available) to HTTP 503 <code>SERVICE_BUSY</code> with <code>Retry-After: 1</code> .                                                                                                                  |
| <b>Connection pool</b>         | 10 connections (HikariCP default; <code>pg.Pool({max: 10})</code> ; <code>pyscopg ConnectionPool(max_size=10)</code> ). Always return connections, even on error ( <code>finally</code> / <code>try-with-resources</code> ).                                                     |
| <b>Arithmetic</b>              | <code>balance = balance - :amount</code> in SQL, not <code>balance = :newBalanceComputedInCode</code> .                                                                                                                                                                          |
| <b>Mapping errors</b>          | Map by constraint name: <code>ck_account_balance</code> → <code>INSUFFICIENT_FUNDS</code> (if the code check was missed), <code>uq_transfer_idempotency</code> → <code>replay</code> , <code>uq_account_customer_type</code> → <code>DUPLICATE_ACCOUNT_TYPE</code> .             |
| <b>ORM users</b>               | With JPA: <code>ddl-auto: validate</code> , a repository method with <code>@Lock(LockModeType.PESSIMISTIC_WRITE)</code> , and <code>@Query</code> ordering by id. Check the SQL log once to see <code>for no key update</code> / <code>for update</code> in the generated query. |

## 06 Data Layer — Redis Cache

### 6.1 What Redis does in this project

Redis is an in-memory key-value store: reads and writes take well under a millisecond. Here it has **three jobs**, and none of them makes Redis a system of record:

| # | Job                          | Why                                                                                 | If Redis loses the data                                                                    |
|---|------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| 1 | <b>Account summary cache</b> | <code>GET /accounts/{no}</code> is the most frequent call (every app home screen)   | Next read goes to PostgreSQL. Slower, still correct.                                       |
| 2 | <b>Statement page cache</b>  | Statements are expensive (4 queries) and often re-requested (paging back and forth) | Same: recomputed from PostgreSQL                                                           |
| 3 | <b>Idempotency store</b>     | Answer a retried transfer instantly with the original response                      | The DB unique key <code>uq_transfer_idempotency</code> still blocks the duplicate (ADR-05) |

#### WATCH OUT

**BR-19: never approve a debit from the cache.** The cached balance may be up to 10 minutes old if an operator changed something directly in the database. The transfer always reads the balance from PostgreSQL **under a row lock**. The cache only serves display reads.

### 6.2 Key naming conventions

- Pattern: `bank:<entity>:<id>[:<qualifier>...]`. Colons separate levels; tools such as RedisInsight show them as folders.
- Always the **business key** (`501000000001`), never the internal `account_id`.
- **Every key has a TTL**. A key without a TTL is a memory leak waiting to happen.
- Keep all key names and TTLs in **one class or module** (`CacheKeys`). No string concatenation scattered across services.

### 6.3 Key catalogue

| Key pattern                                                   | Type             | Value                                                                                                                                                                                   | TTL                          | Written by                                 | Removed by                                          |
|---------------------------------------------------------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|--------------------------------------------|-----------------------------------------------------|
| <code>bank:account:{accountNumber}</code>                     | Hash             | <code>accountNumber</code> ,<br><code>customerName</code> , <code>accountType</code> ,<br><code>currency</code> , <code>balance</code> , <code>status</code> ,<br><code>openedAt</code> | 600 s                        | <code>GET</code> account on a miss         | <code>DEL</code> after a transfer commits; TTL      |
| <code>bank:stmt-ver:{accountNumber}</code>                    | String (integer) | statement version, starts at 0 (missing = 0)                                                                                                                                            | none (tiny, one per account) | <code>INCR</code> after a transfer commits | never                                               |
| <code>bank:stmt:{acc}:v{ver}:{from}:{to}:{page}:{size}</code> | String (JSON)    | the full statement response body                                                                                                                                                        | 300 s                        | <code>GET</code> statement on a miss       | TTL only (old versions are simply never read again) |
| <code>bank:idem:{idempotencyKey}</code>                       | String           | <code>PENDING</code> , then the stored response JSON                                                                                                                                    | 86 400 s (24 h)              | <code>POST /transfers</code>               | TTL; <code>DEL</code> if processing crashed         |

| Key pattern                                                     | Type                | Value                 | TTL  | Written by           | Removed by |
|-----------------------------------------------------------------|---------------------|-----------------------|------|----------------------|------------|
| bank:r1:<br>{accountNumber}:<br>{yyyyMMddHHmm} (stretch<br>S-6) | String<br>(integer) | transfers this minute | 60 s | INCR per<br>transfer | TTL        |

## 6.4 Pattern 1: cache-aside for the account summary

Cache-aside ("lazy loading"): the application looks in the cache first. On a miss it reads the database and fills the cache. The cache never talks to the database itself.

### PSEUDOCODE

```
function getAccount(accountNumber):
    key = "bank:account:" + accountNumber
    try:
        cached = redis.HGETALL(key)                // empty map = miss
        if cached is not empty:
            return (cached, header X-Cache = HIT)
    catch RedisException e:
        log.warn("cache read failed, using DB", e)    // NFR-04: never fail because of the cache

    row = db.query("SELECT a.*, c.full_name FROM account a JOIN customer c ... WHERE a.account_number = ?")
    if row is null: throw AccountNotFound(accountNumber)    // 404 - do NOT cache misses

    try:
        redis.HSET(key, toMap(row)); redis.EXPIRE(key, 600)    // pipeline / MULTI in real code
    catch RedisException e:
        log.warn("cache write failed", e)
    return (row, header X-Cache = MISS)
```

### Lab D2.1 · Cache-aside by hand

#### ► YOUR TASK

Play the application: look up the key (miss), fill it as the app would after reading PostgreSQL, then read it again (hit) and check its TTL.

### TERMINAL

```
docker exec -it bank-redis redis-cli
```

### OUTPUT

```
127.0.0.1:6379> HGETALL bank:account:501000000001
(empty array)
127.0.0.1:6379> HSET bank:account:501000000001 accountNumber 501000000001 customerName "Priya Sharma" accountType SAVINGS currency INR balance 52500.00
status ACTIVE
(integer) 6
127.0.0.1:6379> EXPIRE bank:account:501000000001 600
(integer) 1
127.0.0.1:6379> HGETALL bank:account:501000000001
1) "accountNumber"
2) "501000000001"
3) "customerName"
4) "Priya Sharma"
5) "accountType"
6) "SAVINGS"
7) "currency"
8) "INR"
9) "balance"
10) "52500.00"
11) "status"
12) "ACTIVE"
127.0.0.1:6379> TTL bank:account:501000000001
(integer) 599
```

**HSET** returned 6 (six fields written). **TTL** counts down from 600. When it reaches 0, Redis deletes the key by itself and the next read is a miss.

## 6.5 Pattern 2: invalidation after a write

Cache invalidation is famously one of the hardest parts of caching: a cache that is fast but wrong is worse than no cache. The rules for this project:

1. **Invalidate after COMMIT, never before.** If you delete first and the transaction then waits for a lock, a concurrent read can put the *old* balance back into the cache.
2. **Account summary: delete-on-write.** After a transfer commits, `DEL` both account keys. The next `GET` misses and loads the new balance.
3. **Statement pages: versioned keys.** A statement key contains the date range, page and size, so one account may have dozens of cached pages. Deleting them would need a pattern search ( `KEYS` or `SCAN` ), which is slow and, with `KEYS`, blocks Redis for everyone. Instead, `INCR` a **version number** that is part of every statement key. Old pages are never read again and expire through their TTL.

### Lab D2.2 · Invalidate by hand

#### ► YOUR TASK

Cache a statement page at version 0, then do what the app does after a transfer commits: delete both account summaries and bump both statement versions. Check that the next statement lookup (version 1) misses while the old page quietly ages out.

#### OUTPUT

```
127.0.0.1:6379> GET bank:stmt-ver:501000000001
(nil)
127.0.0.1:6379> SET bank:stmt:501000000001:v0:2026-09-01:2026-09-30:0:20 '{"openingBalance":"0.00","closingBalance":"55000.00"}' EX 300
OK
127.0.0.1:6379> DEL bank:account:501000000001 bank:account:501000000002
(integer) 1
127.0.0.1:6379> INCR bank:stmt-ver:501000000001
(integer) 1
127.0.0.1:6379> INCR bank:stmt-ver:501000000002
(integer) 1
127.0.0.1:6379> EXISTS bank:stmt:501000000001:v1:2026-09-01:2026-09-30:0:20
(integer) 0
127.0.0.1:6379> TTL bank:stmt:501000000001:v0:2026-09-01:2026-09-30:0:20
(integer) 298
```

`DEL` returned 1: only the `...001` summary was cached; deleting a missing key is not an error. `INCR` created the version keys at 1. A key that does not exist counts as 0, so no initialisation is needed.

## 6.6 Pattern 3: idempotency keys

Networks fail. A mobile app that sends a transfer and gets a timeout cannot tell whether the money moved, so it **retries**. Without idempotency, the customer pays twice. The client sends a unique `Idempotency-Key` header (a UUID) with each *logical* transfer and reuses it on retries.

| State of <code>bank:idem: {K}</code> | Meaning                                         | API behaviour                                                                                            |
|--------------------------------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| does not exist                       | first time this key is seen                     | <code>SET K PENDING NX EX 86400</code> → <code>OK</code> ; process the transfer                          |
| <code>PENDING</code>                 | the first request is still running (or crashed) | <code>409 REQUEST_IN_PROGRESS</code> ; the client retries a little later                                 |
| <code>{"httpStatus":201,...}</code>  | finished                                        | return the stored status and body + header <code>Idempotent-Replayed: true</code> ; move <b>no</b> money |

The flow in the service:

## PSEUDOCODE

```
function transfer(request, idemKey):
  claimed = redis.SET("bank:idem:" + idemKey, "PENDING", NX, EX 86400) // atomic
  if not claimed:
    stored = redis.GET("bank:idem:" + idemKey)
    if stored == "PENDING": throw RequestInProgress() // 409
    return replay(stored) // same status + body
  try:
    response = doTransferInDbTransaction(request, idemKey) // Lab D1.3
    // business failures (422) are final answers too: store them as well
  catch BusinessException e:
    response = ErrorResponse(e)
  catch Exception e:
    redis.DEL("bank:idem:" + idemKey) // unexpected crash: let the client retry
    throw e
  redis.SET("bank:idem:" + idemKey, toJson(response), XX, KEPTTL)
  return response
```

If Redis is down, skip the claim and rely on the database. A replayed key then fails the insert with `uq_transfer_idempotency`. Catch that and return the original transfer by looking it up with `SELECT ... FROM fund_transfer WHERE idempotency_key = ?`.

### Lab D2.3 · Idempotency by hand

## OUTPUT

```
127.0.0.1:6379> SET bank:idem:7c9e6679-7425-40de-944b-e07fc1f90ae7 PENDING NX EX 86400
OK
127.0.0.1:6379> SET bank:idem:7c9e6679-7425-40de-944b-e07fc1f90ae7 PENDING NX EX 86400
(nil)
127.0.0.1:6379> GET bank:idem:7c9e6679-7425-40de-944b-e07fc1f90ae7
"PENDING"
127.0.0.1:6379> SET bank:idem:7c9e6679-7425-40de-944b-e07fc1f90ae7 '{"httpStatus":201,"referenceNo":"TRF20261001000007"}' XX KEPTTL
OK
127.0.0.1:6379> GET bank:idem:7c9e6679-7425-40de-944b-e07fc1f90ae7
'{"httpStatus":201,"referenceNo":"TRF20261001000007"}'
127.0.0.1:6379> TTL bank:idem:7c9e6679-7425-40de-944b-e07fc1f90ae7
(integer) 86398
```

The second `SET ... NX` returned `(nil)`: the key was already claimed, so a concurrent duplicate is detected **atomically**, with no read-then-write race. `XX KEPTTL` replaces the value only if the key exists and keeps the original 24-hour expiry.

## 6.7 Memory policy and safe commands

## OUTPUT

```
127.0.0.1:6379> CONFIG GET maxmemory-policy
1) "maxmemory-policy"
2) "allkeys-lru"
```

`allkeys-lru` lets Redis evict the least-recently-used keys when it reaches 128 MB, which is right for a cache. (It could evict an idempotency key early. The database constraint covers that case.)

To look at what is in Redis, use `SCAN`. It walks the keyspace in small steps:

## OUTPUT

```
127.0.0.1:6379> SCAN 0 MATCH bank:* COUNT 100
1) "0"
2) 1) "bank:idem:7c9e6679-7425-40de-944b-e07fc1f90ae7"
    2) "bank:stmt-ver:501000000002"
    3) "bank:stmt-ver:501000000001"
    4) "bank:stmt:501000000001:v0:2026-09-01:2026-09-30:0:20"
```

**WATCH OUT**

Never call `KEYS *` (or `KEYS bank:*`) in application code or on a shared server. It scans every key in one blocking step. With millions of keys it freezes Redis for every client. `SCAN` gives the same answer in small, non-blocking batches. (The first `1` "`0`" in the reply is the cursor: `0` means the scan is complete.)

## 6.8 Client cheat sheet

| Operation            | Spring ( <code>StringRedisTemplate t</code> )                                                                       | Node.js ( <code>ioredis</code> )                                                                | Python ( <code>redis-py</code> )                                                                          |
|----------------------|---------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Read hash            | <code>t.opsForHash().entries(key)</code>                                                                            | <code>await redis.hgetall(key)</code>                                                           | <code>r.hgetall(key)</code>                                                                               |
| Write hash<br>+ TTL  | <code>t.opsForHash().putAll(key, map);</code><br><code>t.expire(key, ttl)</code>                                    | <code>redis.multi().hset(key,</code><br><code>obj).expire(key, 600).exec()</code>               | <code>r.pipeline().hset(key,</code><br><code>mapping=m).expire(key,</code><br><code>600).execute()</code> |
| Delete               | <code>t.delete(List.of(k1, k2))</code>                                                                              | <code>await redis.del(k1, k2)</code>                                                            | <code>r.delete(k1, k2)</code>                                                                             |
| Version<br>bump      | <code>t.opsForValue().increment(key)</code>                                                                         | <code>await redis.incr(key)</code>                                                              | <code>r.incr(key)</code>                                                                                  |
| Claim key            | <code>t.opsForValue().setIfAbsent(key,</code><br><code>"PENDING", Duration.ofHours(24))</code>                      | <code>await redis.set(key,</code><br><code>'PENDING', 'EX', 86400,</code><br><code>'NX')</code> | <code>r.set(key, "PENDING",</code><br><code>nx=True, ex=86400)</code>                                     |
| Replace,<br>keep TTL | <code>t.execute(...)</code> with <code>SET key val XX</code><br><code>KEEPTTL</code> (or re-set with remaining TTL) | <code>await redis.set(key, json,</code><br><code>'XX', 'KEEPTTL')</code>                        | <code>r.set(key, json, xx=True,</code><br><code>keepttl=True)</code>                                      |

**TIP**

Set a **short Redis timeout** (500 ms in the template) and wrap every cache call so an exception is logged and swallowed. Test it: stop Redis ( `docker stop bank-redis` ) and check that every API still answers (TC-29). Start it again with `docker start bank-redis`.

## 07 Data Layer — Kafka Messaging

### 7.1 Why events

When a transfer completes, several other things should happen: an SMS to both customers, a fraud check, an update to the analytics warehouse. If the transfer API called each of these systems directly, it would be slow, and it would fail whenever any of them was down.

Instead the service **publishes one event** ("TransferCompleted") to Kafka and returns. Every interested system reads the event at its own pace through its own **consumer group**. In this project you build one consumer, `notification-service`. The design lets the bank add more later without touching the transfer code.

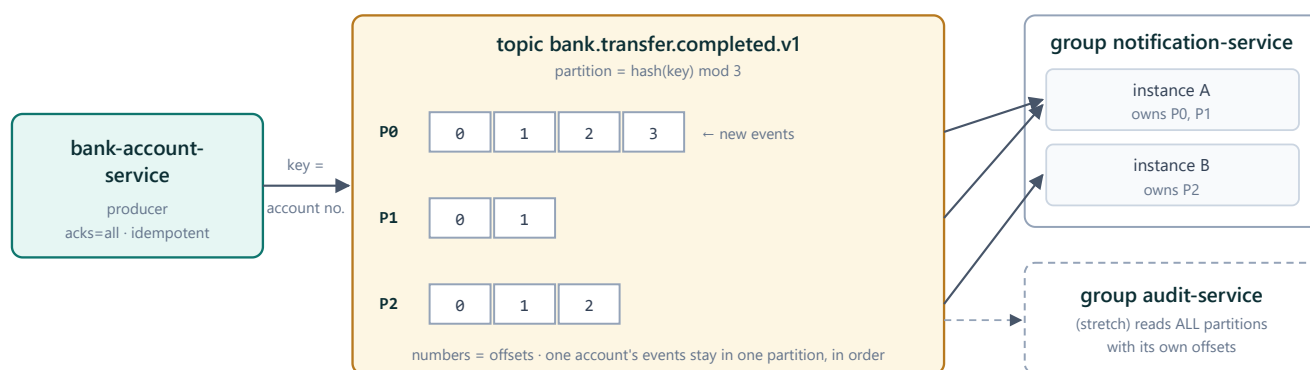
Kafka is a **durable, ordered, partitioned log**. Events are not deleted when they are read; they stay for the retention period (7 days here). A new consumer, or a consumer that fixed a bug, can **replay** them.

### 7.2 Topic catalogue

| Topic                                   | Event                                       | Key                             | Partitions | Retention | Producer             | Consumer groups                                          |
|-----------------------------------------|---------------------------------------------|---------------------------------|------------|-----------|----------------------|----------------------------------------------------------|
| <code>bank.account.opened.v1</code>     | <code>AccountOpened</code>                  | account number                  | 3          | 7 days    | bank-account-service | notification-service                                     |
| <code>bank.transfer.completed.v1</code> | <code>TransferCompleted</code>              | <b>sender</b><br>account number | 3          | 7 days    | bank-account-service | notification-service · ( <i>audit-service, stretch</i> ) |
| <code>bank.transfer.failed.v1</code>    | <code>TransferFailed</code><br>(stretch F7) | sender<br>account number        | 3          | 7 days    | bank-account-service | notification-service                                     |

**Naming:** `<domain>.<entity>.<past-tense event>.v<major version>`. Use lower case and dots only, never mixed with underscores (Kafka warns about this; you will see the warning in Lab D3.1). The version suffix allows a breaking change later: publish `v2` to a new topic and keep `v1` running until every consumer has moved.

### 7.3 Keys, partitions and ordering



**Figure 07.1** The producer picks the partition from the key; each partition is read by exactly one instance in a consumer group.

- Kafka guarantees order **only within a partition**. The producer picks the partition as `hash(key) mod partitions`, so **all events with the same key land in the same partition, in order**. Keying by account number keeps every account's events ordered: its debit alert can never overtake the opening alert.
- A consumer group shares the partitions among its instances. With 3 partitions, up to 3 instances work in parallel; a 4th would sit idle. That is why the partition count is a capacity decision.
- Two **different** groups (notification-service, audit-service) each get **every** event and keep their own offsets.

## 7.4 Event contract

Every event uses the same **envelope**, so consumers can route, de-duplicate and trace events without knowing the payload:

| Field                      | Type                 | Rule                                                                                      |
|----------------------------|----------------------|-------------------------------------------------------------------------------------------|
| <code>eventId</code>       | UUID string          | New random UUID per event. Consumers de-duplicate on it.                                  |
| <code>eventType</code>     | string               | <code>AccountOpened</code> , <code>TransferCompleted</code> , <code>TransferFailed</code> |
| <code>eventVersion</code>  | integer              | Schema version of <code>data</code> ; <code>1</code> for this project                     |
| <code>occurredAt</code>    | ISO-8601 with offset | When the business fact happened (the commit), not when it was sent                        |
| <code>source</code>        | string               | <code>bank-account-service</code>                                                         |
| <code>correlationId</code> | string               | The request's <code>X-Correlation-Id</code> , so logs across services can be joined       |
| <code>data</code>          | object               | The event-specific payload below                                                          |

### WATCH OUT

**Amounts are strings in events** (`"2500.00"`), never JSON numbers. A consumer written in JavaScript would otherwise parse them as binary floating point. The same rule applies to the REST API (chapter 08).

`AccountOpened` (topic `bank.account.opened.v1`, key = `"501000000006"`)

JSON

```
{
  "eventId": "9b1f4f7e-6d0c-4b7a-8a53-0c2f4e9d1a21",
  "eventType": "AccountOpened",
  "eventVersion": 1,
  "occurredAt": "2026-10-05T11:20:31.412+05:30",
  "source": "bank-account-service",
  "correlationId": "3f2a9c4e-58b1-4f0e-9d7a-1c6b2e8f4a90",
  "data": {
    "accountNumber": "501000000006",
    "customerId": 4,
    "customerName": "Karthik Rao",
    "accountType": "SAVINGS",
    "currency": "INR",
    "initialDeposit": "15000.00",
    "phone": "9876500004",
    "email": "karthik.rao@example.com"
  }
}
```

**TransferCompleted** (topic `bank.transfer.completed.v1`, key = sender account)

JSON

```
{
  "eventId": "0b6f7a52-3c1e-4d0a-9a51-2f8e6c1d9b47",
  "eventType": "TransferCompleted",
  "eventVersion": 1,
  "occurredAt": "2026-10-01T10:29:12.940+05:30",
  "source": "bank-account-service",
  "correlationId": "lab-d1-3-0001",
  "data": {
    "transferId": "4e1c32a1-7b63-4dc7-a3a2-de2e1f59f4da",
    "referenceNo": "TRF20261001000005",
    "fromAccountNumber": "501000000001",
    "toAccountNumber": "501000000002",
    "amount": "2500.00",
    "currency": "INR",
    "fromBalanceAfter": "52500.00",
    "toBalanceAfter": "31000.00",
    "remarks": "Rent October"
  }
}
```

**TransferFailed** (stretch F7; topic `bank.transfer.failed.v1`, key = sender account)

JSON

```
{
  "eventId": "e4c1d2b3-7a8f-4e6d-9c0b-1a2f3e4d5c6b",
  "eventType": "TransferFailed",
  "eventVersion": 1,
  "occurredAt": "2026-10-05T12:02:41.511+05:30",
  "source": "bank-account-service",
  "correlationId": "c7d8e9f0-1a2b-4c3d-8e4f-5a6b7c8d9e0f",
  "data": {
    "referenceNo": "TRF20261005000009",
    "fromAccountNumber": "501000000004",
    "toAccountNumber": "501000000002",
    "amount": "20000.00",
    "currency": "INR",
    "reason": "INSUFFICIENT_FUNDS"
  }
}
```

#### NOTE

`AccountOpened` carries the phone number and e-mail because notification-service needs them. That makes the topic **PII** (personal data). In production it would have access control (ACLs) and, often, field-level encryption. Never log full event payloads that contain PII at INFO level.

## 7.5 Producer design

| Setting                                          | Value                       | Why                                                                                            |
|--------------------------------------------------|-----------------------------|------------------------------------------------------------------------------------------------|
| <code>acks</code>                                | <code>all</code>            | The broker confirms only when the event is safely written                                      |
| <code>enable.idempotence</code>                  | <code>true</code>           | Producer retries never write the same event twice in the log                                   |
| <code>key</code> / <code>value</code> serializer | String / String (JSON text) | Simple, readable in Kafka UI. Schema Registry + Avro is stretch S-8                            |
| <code>max.block.ms</code>                        | 5000                        | If Kafka is down, <code>send()</code> gives up after 5 s instead of hanging the request thread |

| Setting             | Value                                   | Why                                                           |
|---------------------|-----------------------------------------|---------------------------------------------------------------|
| <b>When to send</b> | <b>after the DB transaction commits</b> | Never announce a transfer that might still roll back (ADR-06) |

How each stack publishes after commit:

| Stack       | Mechanism                                                                                                                                                                                                                                          |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Spring Boot | Publish a Spring <code>ApplicationEvent</code> inside the <code>@Transactional</code> method; a <code>@TransactionalEventListener(phase = AFTER_COMMIT)</code> method evicts the cache and calls <code>kafkaTemplate.send(topic, key, json)</code> |
| Node.js     | <code>await client.query('COMMIT')</code> , then <code>client.release()</code> , then the cache calls and <code>await producer.send(...)</code> , in that order, in the service function                                                           |
| Python      | Leave the <code>with conn.transaction():</code> block (commits), then the cache calls and <code>producer.produce(...)</code> + <code>producer.poll(0)</code>                                                                                       |

If the send fails after the commit, the transfer has still happened. Log the full event at ERROR with the reference number so operations can re-publish it (NFR-05). Stretch S-1 (transactional outbox) removes this gap: the event row is written **in the same DB transaction** and a relay publishes it.

## 7.6 Consumer design (notification-service)

| Decision                       | Choice                                                                                                                                                                                     |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Group ID                       | <code>notification-service</code>                                                                                                                                                          |
| Subscribed topics              | all three                                                                                                                                                                                  |
| Start position for a new group | <code>auto.offset.reset = earliest</code>                                                                                                                                                  |
| Delivery semantics             | <b>At-least-once</b> : process the event, <i>then</i> commit the offset. A crash in between means the event comes again.                                                                   |
| Making duplicates harmless     | <code>INSERT ... ON CONFLICT (event_id, account_number) DO NOTHING</code> ; the primary key of <code>notification_log</code> de-duplicates (Lab D3.4)                                      |
| Messages per event             | <code>AccountOpened</code> → 1 row · <code>TransferCompleted</code> → 2 rows (debit alert to sender, credit alert to receiver) · <code>TransferFailed</code> → 1 row                       |
| Message text                   | Mask account numbers to the last 4 digits: <code>A/c XX0001</code>                                                                                                                         |
| Bad message (unparseable JSON) | Log it at ERROR with topic/partition/offset and <b>skip</b> it (commit). One poison message must not block the partition. Dead-letter topic <code>&lt;topic&gt;.dlt</code> is stretch S-7. |

| Semantics            | How                                                  | Risk                                 | Used here?                            |
|----------------------|------------------------------------------------------|--------------------------------------|---------------------------------------|
| At-most-once         | commit offset, then process                          | crash = event lost                   | no                                    |
| <b>At-least-once</b> | process, then commit                                 | crash = event processed twice        | <b>yes</b> , plus an idempotent write |
| Exactly-once         | Kafka transactions (read-process-write inside Kafka) | complex; does not cover the DB write | no                                    |

## 7.7 Lab D3: Kafka by hand

### Lab D3.1 · Create the topics

starter\_kit/kafka/create-topics.sh

TERMINAL

```
#!/bin/sh
# -----
# Create the capstone's Kafka topics (safe to run again).
#
#     docker exec bank-kafka sh /scripts/create-topics.sh
#
# 3 partitions each: events with the same key (an account number) always
# land in the same partition, so they stay in order. Replication factor 1
# because the Lab has a single broker - production would use 3.
# -----
set -e
BOOTSTRAP=localhost:9092

for topic in bank.account.opened.v1 bank.transfer.completed.v1 bank.transfer.failed.v1; do
    kafka-topics.sh --bootstrap-server "$BOOTSTRAP" --create --if-not-exists \
        --topic "$topic" --partitions 3 --replication-factor 1 \
        --config retention.ms=604800000 # keep events for 7 days
done

kafka-topics.sh --bootstrap-server "$BOOTSTRAP" --list
```

TERMINAL

```
docker exec bank-kafka sh /scripts/create-topics.sh
```

OUTPUT

```
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but
not both.
Created topic bank.account.opened.v1.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but
not both.
Created topic bank.transfer.completed.v1.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but
not both.
Created topic bank.transfer.failed.v1.
bank.account.opened.v1
bank.transfer.completed.v1
bank.transfer.failed.v1
```

The **WARNING** lines are informational. Our names use only dots, so nothing can collide.

### Lab D3.2 · Inspect a topic

TERMINAL

```
docker exec bank-kafka kafka-topics.sh --bootstrap-server localhost:9092 \
    --describe --topic bank.transfer.completed.v1
```

OUTPUT

```
Topic: bank.transfer.completed.v1      TopicId: _jRYVK1aQditRE8Vd35Rwg PartitionCount: 3      ReplicationFactor: 1      Configs:
min.insync.replicas=1,retention.ms=604800000
  Topic: bank.transfer.completed.v1    Partition: 0      Leader: 1      Replicas: 1      Isr: 1 Elr:      LastKnownElr:
  Topic: bank.transfer.completed.v1    Partition: 1      Leader: 1      Replicas: 1      Isr: 1 Elr:      LastKnownElr:
  Topic: bank.transfer.completed.v1    Partition: 2      Leader: 1      Replicas: 1      Isr: 1 Elr:      LastKnownElr:
```

Three partitions, each with a leader on broker 1 and one replica (the lab has one broker; production uses three replicas and `min.insync.replicas=2`).

## Lab D3.3 · Produce and consume a keyed event

### ► YOUR TASK

Play bank-account-service: publish the `TransferCompleted` event of Lab D1.3, keyed by the sender account. Start the console producer, paste the line below (key | JSON on **one** line), press Enter, then Ctrl+C.

#### TERMINAL

```
docker exec -it bank-kafka kafka-console-producer.sh --bootstrap-server localhost:9092 \
  --topic bank.transfer.completed.v1 \
  --reader-property parse.key=true --reader-property 'key.separator=|'
```

#### PASTE AS ONE LINE

```
501000000001|{"eventId":"0b6f7a52-3c1e-4d0a-9a51-2f8e6c1d9b47","eventType":"TransferCompleted","eventVersion":1,"occurredAt":"2026-10-01T10:29:12.940+05:30","source":"bank-account-service","correlationId":"lab-d1-3-0001","data":{"transferId":"4e1c32a1-7b63-4dc7-a3a2-de2e1f59f4da","referenceNo":"TRF20261001000005","fromAccountNumber":"501000000001","toAccountNumber":"501000000002","amount":"2500.00","currency":"INR","fromBalanceAfter":"52500.00","toBalanceAfter":"31000.00","remarks":"Rent October"}}
```

Now play notification-service: read it as the group `notification-service`, printing partition, offset and key:

#### TERMINAL

```
docker exec -it bank-kafka kafka-console-consumer.sh --bootstrap-server localhost:9092 \
  --topic bank.transfer.completed.v1 --group notification-service --from-beginning \
  --formatter-property print.key=true --formatter-property print.partition=true \
  --formatter-property print.offset=true --max-messages 1
```

#### OUTPUT

```
Partition:0      Offset:0      501000000001  {"eventId":"0b6f7a52-3c1e-4d0a-9a51-2f8e6c1d9b47","eventType":"TransferCompleted","eventVersion":1,"occurredAt":"2026-10-01T10:29:12.940+05:30","source":"bank-account-service","correlationId":"lab-d1-3-0001","data":{"transferId":"4e1c32a1-7b63-4dc7-a3a2-de2e1f59f4da","referenceNo":"TRF20261001000005","fromAccountNumber":"501000000001","toAccountNumber":"501000000002","amount":"2500.00","currency":"INR","fromBalanceAfter":"52500.00","toBalanceAfter":"31000.00","remarks":"Rent October"}}
```

The key `501000000001` hashed to partition 0. Every later event for this account goes to partition 0 too. Now look at the group's progress:

#### TERMINAL

```
docker exec bank-kafka kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --describe --group notification-service
```

#### OUTPUT

| GROUP                | TOPIC                      | PARTITION | CURRENT-OFFSET | LOG-END-OFFSET | LAG | CONSUMER-ID | HOST | CLIENT-ID |
|----------------------|----------------------------|-----------|----------------|----------------|-----|-------------|------|-----------|
| notification-service | bank.transfer.completed.v1 | 0         | 1              | 1              | 0   | -           | -    | -         |
| notification-service | bank.transfer.completed.v1 | 1         | 0              | 0              | 0   | -           | -    | -         |
| notification-service | bank.transfer.completed.v1 | 2         | 0              | 0              | 0   | -           | -    | -         |

`CURRENT-OFFSET 1 = LOG-END-OFFSET 1`, so `LAG 0`: the group has processed everything. **Lag** is the number one metric for consumers in production. Growing lag means the consumers cannot keep up.

## Lab D3.4 · Make the consumer idempotent

What notification-service writes for that event, delivered **twice**:

📄 [starter\\_kit/sql/06\\_consumer\\_idempotency.sql](#)

SQL

```
-- =====
-- Lab D3.4 | Idempotent consumer - what notification-service writes
-- for ONE TransferCompleted event: a debit alert and a credit alert.
--
-- docker exec -it bank-postgres psql -U bank_app -d bankdb -f /sql/06_consumer_idempotency.sql
--
-- Kafka delivers at-least-once: after a crash or rebalance the same
-- event can arrive again. The run below inserts the same event twice;
-- the second insert writes nothing, so nobody gets a second SMS.
-- =====
\set event_id '0b6f7a52-3c1e-4d0a-9a51-2f8e6c1d9b47'

-- first delivery
INSERT INTO notification_log (event_id, account_number, event_type, message)
VALUES (:event_id, '501000000001', 'TransferCompleted',
        'Rs 2500.00 debited from A/c XX0001 to A/c XX0002. Ref TRF20261001000005. Avl bal Rs 52500.00'),
        (:event_id, '501000000002', 'TransferCompleted',
        'Rs 2500.00 credited to A/c XX0002 from A/c XX0001. Ref TRF20261001000005. Avl bal Rs 31000.00')
ON CONFLICT (event_id, account_number) DO NOTHING;

-- re-delivery of the very same event
INSERT INTO notification_log (event_id, account_number, event_type, message)
VALUES (:event_id, '501000000001', 'TransferCompleted', 'duplicate'),
        (:event_id, '501000000002', 'TransferCompleted', 'duplicate')
ON CONFLICT (event_id, account_number) DO NOTHING;

SELECT account_number, left(message, 48) AS message FROM notification_log
WHERE event_id = :event_id ORDER BY account_number;
```

OUTPUT

```
INSERT 0 2
INSERT 0 0
 account_number | message
-----+-----
501000000001    | Rs 2500.00 debited from A/c XX0001 to A/c XX0002
501000000002    | Rs 2500.00 credited to A/c XX0002 from A/c XX000
(2 rows)
```

`INSERT 0 2`, then `INSERT 0 0`: the re-delivery wrote nothing, so there is no duplicate SMS. This is the whole trick of at-least-once delivery with exactly-once *effect*.

## Lab D3.5 · Replay

A new consumer, or a fixed bug, sometimes needs to re-read history. With the group idle (no consumer running), move its offsets back to the start:

TERMINAL

```
docker exec bank-kafka kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --group notification-service --topic bank.transfer.completed.v1 \
  --reset-offsets --to-earliest --execute
```

OUTPUT

| GROUP                | TOPIC                      | PARTITION | NEW-OFFSET |
|----------------------|----------------------------|-----------|------------|
| notification-service | bank.transfer.completed.v1 | 1         | 0          |
| notification-service | bank.transfer.completed.v1 | 2         | 0          |
| notification-service | bank.transfer.completed.v1 | 0         | 0          |

When your notification-service starts, it re-reads every event from offset 0. Because of Lab D3.4, `notification_log` does not change. Try it in M7 (test TC-28).

**TIP**

Open `http://localhost:8085` → *Topics* → `bank.transfer.completed.v1` → *Messages* to see the same event with its key, partition and offset, and *Consumers* to see the group lag.

# 08 API Specification

## 8.1 Conventions

| Item             | Convention                                                                                                                                                                                                                                                                                                      |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Base URL         | <code>http://localhost:8080/api/v1</code> . The version is in the path; a breaking change becomes <code>/api/v2</code> .                                                                                                                                                                                        |
| Format           | JSON, UTF-8, <code>Content-Type: application/json</code> ; errors use <code>application/problem+json</code>                                                                                                                                                                                                     |
| Field names      | <code>camelCase</code>                                                                                                                                                                                                                                                                                          |
| <b>Amounts</b>   | <b>Strings</b> with up to 2 decimals in requests ( <code>"2500"</code> , <code>"2500.5"</code> , <code>"2500.50"</code> ), always exactly 2 in responses ( <code>"2500.00"</code> ). Same convention as Open Banking APIs. Bind to <code>BigDecimal</code> / <code>Decimal</code> ; in Node.js keep the string. |
| Dates            | <code>yyyy-MM-dd</code> ( <code>"2026-09-30"</code> ), interpreted in IST                                                                                                                                                                                                                                       |
| Timestamps       | ISO-8601 with offset, IST: <code>"2026-10-01T10:29:12.940+05:30"</code>                                                                                                                                                                                                                                         |
| Request headers  | <code>Idempotency-Key</code> (required on <code>POST /transfers</code> ), <code>X-Correlation-Id</code> (optional, echoed back; generated if missing)                                                                                                                                                           |
| Response headers | <code>X-Correlation-Id</code> (always) · <code>X-Cache: HIT \   MISS</code> (cached GETs) · <code>Idempotent-Replayed: true</code> (replays) · <code>Location</code> (on 201)                                                                                                                                   |
| Security         | Out of scope for phase 1. A real deployment adds <code>Authorization: Bearer &lt;token&gt;</code> and checks that the caller owns the account.                                                                                                                                                                  |

## 8.2 Endpoint summary

| # | Method and path                                              | Purpose                               | Success | Data layer                               |
|---|--------------------------------------------------------------|---------------------------------------|---------|------------------------------------------|
| 1 | <code>POST /accounts</code>                                  | Open an account                       | 201     | PG (1 tx) → Kafka                        |
| 2 | <code>GET /accounts/{accountNumber}</code>                   | Account details and balance           | 200     | Redis → PG on miss → Redis               |
| 3 | <code>POST /transfers</code>                                 | Transfer money                        | 201     | Redis (idem) → PG (1 tx) → Redis → Kafka |
| 4 | <code>GET /transfers/{referenceNo}</code>                    | Transfer enquiry                      | 200     | PG                                       |
| 5 | <code>GET /accounts/{accountNumber}/statement</code>         | Account statement                     | 200     | Redis → PG on miss → Redis               |
| 6 | <code>GET /actuator/health</code> (or <code>/health</code> ) | Liveness of the app, DB, Redis, Kafka | 200     | all three                                |

## 8.3 `POST /accounts`: open an account

### Request body

| Field                          | Type   | Req. | Rules                                                                |
|--------------------------------|--------|------|----------------------------------------------------------------------|
| <code>customer.fullName</code> | string | yes  | 3–100 characters; letters, spaces, <code>.</code> and <code>'</code> |
| <code>customer.email</code>    | string | yes  | valid e-mail, ≤ 150 characters; stored lower case                    |

| Field                             | Type   | Req. | Rules                                                                   |
|-----------------------------------|--------|------|-------------------------------------------------------------------------|
| <code>customer.phone</code>       | string | yes  | <code>^[6-9][0-9]{9}\$</code>                                           |
| <code>customer.dateOfBirth</code> | date   | yes  | age ≥ 18 today (BR-02)                                                  |
| <code>customer.pan</code>         | string | yes  | <code>^[A-Z]{5}[0-9]{4}[A-Z]\$</code> ; convert to upper case first     |
| <code>accountType</code>          | string | yes  | <code>SAVINGS</code> or <code>CURRENT</code>                            |
| <code>initialDeposit</code>       | amount | yes  | <code>SAVINGS ≥ 1000.00, CURRENT ≥ 5000.00, ≤ 1000000.00 (BR-05)</code> |

TERMINAL

```
curl -i -X POST http://localhost:8080/api/v1/accounts \
-H "Content-Type: application/json" \
-d '{
  "customer": {
    "fullName": "Karthik Rao",
    "email": "karthik.rao@example.com",
    "phone": "9876500004",
    "dateOfBirth": "1995-02-14",
    "pan": "CKRPR7788Q"
  },
  "accountType": "SAVINGS",
  "initialDeposit": "15000.00"
}'
```

**201 Created**, header `Location: /api/v1/accounts/501000000006`

JSON

```
{
  "accountNumber": "501000000006",
  "customerId": 4,
  "customerName": "Karthik Rao",
  "newCustomer": true,
  "accountType": "SAVINGS",
  "currency": "INR",
  "balance": "15000.00",
  "status": "ACTIVE",
  "openedAt": "2026-10-05T11:20:31.398+05:30"
}
```

| Status | errorCode                           | When                                                                                       |
|--------|-------------------------------------|--------------------------------------------------------------------------------------------|
| 400    | <code>VALIDATION_ERROR</code>       | any field rule fails (all failing fields listed in <code>errors</code> )                   |
| 409    | <code>DUPLICATE_ACCOUNT_TYPE</code> | the customer already has an account of this type ( <code>uq_account_customer_type</code> ) |
| 409    | <code>CUSTOMER_CONFLICT</code>      | new PAN, but the e-mail or phone belongs to another customer                               |

**Data layer:** one PostgreSQL transaction: upsert customer by PAN, insert account, insert the "Initial deposit" CREDIT (Lab D1.2). After commit, publish `AccountOpened` (key = account number). No cache write: the first `GET` fills the cache.

## 8.4 `GET /accounts/{accountNumber}`: account enquiry

TERMINAL

```
curl -i http://localhost:8080/api/v1/accounts/501000000001
```

**200 OK**, header `X-Cache: MISS` (first call) or `HIT` (later calls within 10 minutes)

JSON

```
{
  "accountNumber": "501000000001",
  "customerName": "Priya Sharma",
  "accountType": "SAVINGS",
  "currency": "INR",
  "balance": "55000.00",
  "status": "ACTIVE",
  "openedAt": "2026-09-01T10:00:00+05:30"
}
```

| Status | errorCode         | When                                |
|--------|-------------------|-------------------------------------|
| 400    | VALIDATION_ERROR  | the path is not 12 digits           |
| 404    | ACCOUNT_NOT_FOUND | no such account. <b>Not cached.</b> |

**Data layer:** `HGETALL bank:account:{no}`; on a miss, `SELECT` account  $\bowtie$  customer, then `HSET` + `EXPIRE 600` (Lab D2.1).

## 8.5 `POST /transfers`: transfer money

**Headers:** `Idempotency-Key: <1-64 chars of A-Z a-z 0-9 - _>` (**required**; send a new UUID per logical transfer and the same one on retries).

### Request body

| Field                          | Type   | Req. | Rules                                                            |
|--------------------------------|--------|------|------------------------------------------------------------------|
| <code>fromAccountNumber</code> | string | yes  | 12 digits                                                        |
| <code>toAccountNumber</code>   | string | yes  | 12 digits, different from <code>fromAccountNumber</code> (BR-12) |
| <code>amount</code>            | amount | yes  | 1.00 – 200000.00, at most 2 decimals (BR-10)                     |
| <code>remarks</code>           | string | no   | $\leq$ 100 characters                                            |

TERMINAL

```
curl -i -X POST http://localhost:8080/api/v1/transfers \
-H "Content-Type: application/json" \
-H "Idempotency-Key: 7c9e6679-7425-40de-944b-e07fc1f90ae7" \
-d '{
  "fromAccountNumber": "501000000001",
  "toAccountNumber": "501000000002",
  "amount": "2500.00",
  "remarks": "Rent October"
}'
```

**201 Created**, header `Location: /api/v1/transfers/TRF20261001000005`

JSON

```
{
  "transferId": "4e1c32a1-7b63-4dc7-a3a2-de2e1f59f4da",
  "referenceNo": "TRF20261001000005",
  "status": "COMPLETED",
  "fromAccountNumber": "501000000001",
  "toAccountNumber": "501000000002",
  "amount": "2500.00",
  "currency": "INR",
  "remarks": "Rent October",
  "fromBalanceAfter": "52500.00",
  "createdAt": "2026-10-01T10:29:12.940+05:30"
}
```

Sending the identical request again with the same key returns the **same status and body** plus `Idempotent-Replayed: true`, and moves no money.

| Status | errorCode               | When                                                                        |
|--------|-------------------------|-----------------------------------------------------------------------------|
| 400    | VALIDATION_ERROR        | body rule fails                                                             |
| 400    | MISSING_IDEMPOTENCY_KEY | header missing or malformed                                                 |
| 404    | ACCOUNT_NOT_FOUND       | either account does not exist ( <code>detail</code> says which)             |
| 409    | REQUEST_IN_PROGRESS     | the same key is still being processed                                       |
| 422    | SAME_ACCOUNT_TRANSFER   | from = to                                                                   |
| 422    | ACCOUNT_NOT_ACTIVE      | either account is FROZEN or CLOSED                                          |
| 422    | INSUFFICIENT_FUNDS      | sender balance < amount. Do not reveal the balance in <code>detail</code> . |
| 422    | DAILY_LIMIT_EXCEEDED    | today's total + amount > 500000.00                                          |
| 503    | SERVICE_BUSY            | lock timeout or deadlock; header <code>Retry-After: 1</code>                |

**Processing order (MUST):** (1) validate the request → (2) claim the idempotency key in Redis → (3) one DB transaction: lock both rows in `account_id` order, check status / funds / daily limit, insert the transfer, update both balances, insert both ledger rows → (4) **after commit**: delete both account cache keys, `INCR` both statement versions, store the response under the idempotency key, publish `TransferCompleted` → (5) respond 201. Business failures (422) are stored under the idempotency key too: they are final answers.

## 8.6 GET /transfers/{referenceNo} : transfer enquiry

TERMINAL

```
curl -i http://localhost:8080/api/v1/transfers/TRF20261001000005
```

**200 OK:** the same body as the 201 above **without** `fromBalanceAfter` (a reference number must not reveal a balance).

**404** `TRANSFER_NOT_FOUND` if unknown.

## 8.7 GET /accounts/{accountNumber}/statement : account statement

Query parameters

| Parameter             | Type    | Default         | Rules (BR-16)                           |
|-----------------------|---------|-----------------|-----------------------------------------|
| <code>fromDate</code> | date    | today – 29 days | $\leq$ <code>toDate</code>              |
| <code>toDate</code>   | date    | today (IST)     | not in the future; range $\leq$ 90 days |
| <code>page</code>     | integer | 0               | $\geq$ 0                                |
| <code>size</code>     | integer | 20              | 1–100                                   |

## TERMINAL

```
curl -i "http://localhost:8080/api/v1/accounts/501000000001/statement?fromDate=2026-09-01&toDate=2026-09-30&page=0&size=3"
```

**200 OK**, header `X-Cache: MISS | HIT`. The values are the seed data, matching Lab D1.5:

## JSON

```
{
  "accountNumber": "501000000001",
  "customerName": "Priya Sharma",
  "currency": "INR",
  "fromDate": "2026-09-01",
  "toDate": "2026-09-30",
  "openingBalance": "0.00",
  "closingBalance": "55000.00",
  "totalCredits": "62000.00",
  "totalDebits": "7000.00",
  "page": 0,
  "size": 3,
  "totalElements": 4,
  "totalPages": 2,
  "transactions": [
    { "txnId": 12, "txnTime": "2026-09-28T12:10:00+05:30", "type": "DEBIT", "amount": "2000.00",
      "balanceAfter": "55000.00", "description": "Transfer to 501000000004 - Birthday gift",
      "referenceNo": "TRF20260928000004" },
    { "txnId": 9, "txnTime": "2026-09-15T14:00:00+05:30", "type": "CREDIT", "amount": "12000.00",
      "balanceAfter": "57000.00", "description": "Transfer from 501000000003 - Invoice 1043",
      "referenceNo": "TRF20260915000002" },
    { "txnId": 6, "txnTime": "2026-09-10T09:30:00+05:30", "type": "DEBIT", "amount": "5000.00",
      "balanceAfter": "45000.00", "description": "Transfer to 501000000002 - Rent share",
      "referenceNo": "TRF20260910000001" }
  ]
}
```

Page 1 holds the remaining entry, the initial deposit, with `"referenceNo": null`.

| Status | errorCode                      | When                                                                         |
|--------|--------------------------------|------------------------------------------------------------------------------|
| 400    | <code>VALIDATION_ERROR</code>  | bad dates, range > 90 days, <code>toDate</code> in the future, bad page/size |
| 404    | <code>ACCOUNT_NOT_FOUND</code> | no such account                                                              |

**Data layer:** `GET bank:stmt-ver:{no}` (missing = 0) → `GET bank:stmt:{no}:v{ver}:{from}:{to}:{page}:{size}`; on a miss, the four queries of Lab D1.5, then `SET ... EX 300`.

## 8.8 Error model

All errors use **RFC 9457 Problem Details** (`application/problem+json`), extended with an `errorCode` that clients can switch on, the correlation ID and, for validation, the failing fields.

JSON

```
{
  "type": "https://api.horizonbank.example/problems/validation-error",
  "title": "Request validation failed",
  "status": 400,
  "detail": "2 fields are invalid",
  "instance": "/api/v1/transfers",
  "errorCode": "VALIDATION_ERROR",
  "correlationId": "c7d8e9f0-1a2b-4c3d-8e4f-5a6b7c8d9e0f",
  "timestamp": "2026-10-05T12:02:41.511+05:30",
  "errors": [
    { "field": "amount", "message": "must have at most 2 decimal places" },
    { "field": "toAccountNumber", "message": "must be 12 digits" }
  ]
}
```

| HTTP | errorCode                                                                           | Meaning                                                            |
|------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| 400  | VALIDATION_ERROR, MISSING_IDEMPOTENCY_KEY                                           | The request itself is wrong; retrying it unchanged will fail again |
| 404  | ACCOUNT_NOT_FOUND, TRANSFER_NOT_FOUND                                               | The resource does not exist                                        |
| 409  | DUPLICATE_ACCOUNT_TYPE, CUSTOMER_CONFLICT, REQUEST_IN_PROGRESS                      | Conflicts with the current state                                   |
| 422  | SAME_ACCOUNT_TRANSFER, ACCOUNT_NOT_ACTIVE, INSUFFICIENT_FUNDS, DAILY_LIMIT_EXCEEDED | Well-formed, but a business rule says no                           |
| 500  | INTERNAL_ERROR                                                                      | Bug. Log the stack trace; never send it to the client.             |
| 503  | SERVICE_BUSY, SERVICE_UNAVAILABLE                                                   | Temporary (lock timeout, database down);<br>Retry-After header     |

**WATCH OUT**

Never leak internals in an error: no SQL, no stack traces, no table or constraint names, no balances of other people. Map them to an `errorCode` and log the details server-side with the correlation ID.

## 09 Implementation Roadmap

Build the system in **nine milestones**. Each one ends with a *Definition of Done* (DoD) you can demonstrate. Do not start a milestone before the previous DoD is met. Most beginner projects fail by building everything at once and debugging it all at the end.

| Milestone           | Day | Effort | Output                                                        |
|---------------------|-----|--------|---------------------------------------------------------------|
| M0 Environment      | 1   | 1 h    | Stack healthy, repo created                                   |
| M1 Database labs    | 1   | 5 h    | Labs D1.1–D1.7 done, schema understood                        |
| M2 Project skeleton | 2   | 2 h    | App connects to PostgreSQL, health + error model              |
| M3 Accounts         | 2   | 5 h    | <code>POST /accounts</code> , <code>GET /accounts/{no}</code> |
| M4 Transfer         | 3   | 4 h    | <code>POST /transfers</code> , ACID-safe                      |
| M5 Statement        | 3   | 3 h    | Statement + transfer enquiry                                  |
| M6 Redis            | 4   | 4 h    | Cache-aside, invalidation, idempotency                        |
| M7 Kafka            | 4   | 4 h    | Events + notification-service                                 |
| M8 Proof            | 5   | 6 h    | Concurrency test, evidence, README, demo                      |

### M0 · Environment

- Install the prerequisites (chapter 00). Create a Git repository `bank-account-system`.
- Copy `starter_kit/` into it as `infra/`. Start it (chapter 04) and check all services are `healthy`.
- Add `.gitignore` (`target/`, `node_modules/`, `.venv/`, `.env`, IDE folders) and a `README.md` stub.

**DoD:** `docker compose ps` shows three healthy services, the first commit is pushed, and the README says how to start the stack.

### M1 · Database labs

- Do Labs **D1.1 – D1.7** (chapter 05) in order, and run the two-terminal lock exercise.
- Write, in your own words in the README, why the transfer locks both rows in `account_id` order.
- Reset the database at the end (`docker compose down -v`, `up -d`).

**DoD:** you can explain every step of `02_transfer.sql` to the trainer without notes, and reconciliation returns zero rows.

#### TRAINER NOTE

Spend time here. Ask each participant to explain the race demo output. Participants who understand Lab D1.3 and D1.7 write a correct transfer service on Day 3. Participants who skip them write `balance = newBalance` and lose money under load.

### M2 · Project skeleton

- **Spring Boot:** `start.spring.io`: Java 21, dependencies *Spring Web MVC*, *Validation*, *Spring Data JPA* (or *JDBC*), *PostgreSQL Driver*, *Spring Data Redis*, *Spring for Apache Kafka*, *Actuator*. Use the `application.yml` from 4.4.

- **Node.js:** `npm init -y; npm i express zod pg ioredis kafkajs dotenv pino; dev: jest supertest nodemon.`
- **Python:** `python -m venv .venv; pip install fastapi uvicorn[standard] "psycopg[binary,pool]" redis confluent-kafka pydantic-settings; dev: pytest httpx.`
- Create the layers of 3.5. Read configuration from environment variables.
- Add a **correlation-ID filter/middleware**: take `X-Correlation-Id` or generate a UUID, put it in the log context (MDC / AsyncLocalStorage / contextvars) and echo it in the response.
- Add a **global error handler** that turns exceptions into the problem+json body of 8.8.
- Add the health endpoint (Actuator gives DB, Redis and Kafka health for free in Spring).

**DoD:** the app starts, `GET /actuator/health` (or `/health`) shows the database `UP`, and an unknown URL returns a problem+json 404 with a correlation ID.

### M3 · Account opening and enquiry

- Request/response DTOs with validation annotations or schemas (8.3); amounts as strings → decimal.
- `AccountService.open()`: **one transaction**: customer upsert by PAN (5.5 note), account insert (`RETURNING account_number`), initial-deposit ledger row.
- Map constraint violations by name: `uq_account_customer_type` → 409 `DUPLICATE_ACCOUNT_TYPE`, `uq_customer_email` / `uq_customer_phone` → 409 `CUSTOMER_CONFLICT`.
- `AccountService.get()`: account ⇄ customer by account number (no cache yet), 404 if missing.

**DoD:** TC-01 to TC-07 and TC-09 pass; the new account appears in psql with its ledger row.

### M4 · Fund transfer

Implement exactly the algorithm of Lab D1.3. In pseudocode:

PSEUDOCODE

```
// controller: validate body, require Idempotency-Key header, then call the service
// service - ONE database transaction:
function doTransfer(req, idempKey):
  db.execute("SET LOCAL lock_timeout = '3s'")
  rows = accountRepo.lockBoth(req.from, req.to) // ... ORDER BY account_id FOR UPDATE
  from = rows.find(req.from) or throw AccountNotFound(req.from) // 404
  to = rows.find(req.to) or throw AccountNotFound(req.to) // 404
  if from.status != ACTIVE or to.status != ACTIVE: throw AccountNotActive // 422
  if from.balance < req.amount: throw InsufficientFunds // 422
  sentToday = transferRepo.sumCompletedSince(from.id, startOfTodayIST())
  if sentToday + req.amount > 500000.00: throw DailyLimitExceeded // 422
  t = transferRepo.insert(idempKey, from.id, to.id, req.amount, req.remarks)
  fromBal = accountRepo.debit(from.id, req.amount) // UPDATE ... RETURNING balance
  toBal = accountRepo.credit(to.id, req.amount)
  ledgerRepo.insert(from.id, t.id, DEBIT, req.amount, fromBal, "Transfer to " + to.number + suffix)
  ledgerRepo.insert(to.id, t.id, CREDIT, req.amount, toBal, "Transfer from " + from.number + suffix)
  afterCommit(TransferCompleted(t, from, to, fromBal, toBal)) // M6/M7 hook it up
  return TransferResponse(t, fromBal)
// any exception thrown above rolls the whole transaction back - nothing is half-written
```

- Same-account check (422) happens in the controller/service **before** the transaction.
- Until M6, a repeated `Idempotency-Key` fails on `uq_transfer_idempotency`: catch it, load the existing transfer by key, and return it with `Idempotent-Replayed: true`.
- Map SQLSTATE `55P03` / `40P01` to 503 `SERVICE_BUSY`.

**DoD:** TC-10 and TC-12 to TC-19 pass, and reconciliation (Lab D1.6) returns zero rows afterwards.

## M5 · Statement and transfer enquiry

- Parse and default `fromDate`, `toDate`, `page`, `size`; enforce BR-16.
- Convert dates to instants: `from = fromDate 00:00 IST`, `toExclusive = (toDate + 1) 00:00 IST`.
- Run the four queries of Lab D1.5 (inside one read-only transaction so the totals and the page agree); build the response of 8.7.
- `GET /transfers/{referenceNo}`.

**DoD:** TC-21 to TC-25 pass with the exact numbers in 8.7.

## M6 · Redis

- First do **Labs D2.1 – D2.3** (chapter 06).
- Create a cache component with all key names and TTLs; wrap every call in try/catch + log (NFR-04).
- Account enquiry: cache-aside + `X-Cache` header.
- Statement: versioned keys + `X-Cache` header.
- Transfer: idempotency claim / replay (6.6); **after commit**: `DEL` both account keys, `INCR` both statement versions, store the final response under the idempotency key.

**DoD:** TC-08, TC-11, TC-26 and TC-29 pass; `redis-cli` shows the keys of 6.3 with TTLs.

## M7 · Kafka

- First do **Labs D3.1 – D3.5** (chapter 07).
- Producer with the settings of 7.5; an `EventEnvelope` builder (new `eventId`, `occurredAt`, `correlationId`).
- Publish `AccountOpened` and `TransferCompleted` **after commit**, keyed by account number; log failures with the full event.
- notification-service: subscribe as group `notification-service`, build the messages of 7.6, `INSERT ... ON CONFLICT DO NOTHING`, commit offsets after the insert; skip and log poison messages.

**DoD:** TC-27, TC-28 and TC-30 pass; Kafka UI shows the events and lag 0 for the group.

## M8 · Proof

- Run the **concurrency test** (10.5) and the reconciliation script; save both outputs.
- Finish the Postman collection (or curl script) for all test cases; run it top to bottom on a freshly reset stack.
- Complete the README (11.2) and the test-evidence sheet (10.7). Rehearse the demo (11.4).

**DoD:** the submission checklist in chapter 11 is fully ticked.

# 10 Testing & Verification

## 10.1 Test strategy

| Level            | What                                                                                     | Tool                                                                 | Who               |
|------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------|-------------------|
| Unit             | Validation rules, amount parsing, IST date boundaries, message formatting                | JUnit / Jest / pytest                                                | every participant |
| Integration      | Repositories and the transfer transaction against <b>real</b> PostgreSQL / Redis / Kafka | Testcontainers (Java, Node, Python all have it) or the Compose stack | every participant |
| API (functional) | Test cases TC-01 – TC-30 below                                                           | Postman collection or curl script                                    | every participant |
| Concurrency      | 20 parallel transfers from one account (TC-20)                                           | <code>xargs -P</code> + curl, or the Postman runner                  | every participant |
| Data             | Reconciliation R1 – R3 after every run                                                   | <code>04_reconciliation.sql</code>                                   | every participant |

### WATCH OUT

**Do not mock the database for transfer tests.** A mock cannot show a race condition, a lock, a constraint or a rollback, and those are exactly what this project is about. Use the real PostgreSQL.

## 10.2 Before every full test run

1. Reset the data layer: `docker compose down -v`, `docker compose up -d`, wait for healthy, run `create-topics.sh`.
2. Start the application (and notification-service).
3. Run the cases **in order**. Some depend on earlier ones (TC-11 replays TC-10, for example). Expected numbers assume a fresh seed.

Seed accounts: `...001` Priya SAVINGS 55,000.00 · `...002` Rahul SAVINGS 28,500.00 · `...003` Rahul CURRENT 88,000.00 · `...004` Ananya SAVINGS 13,500.00 · `...005` Ananya CURRENT 5,000.00 **FROZEN**.

## 10.3 Test case catalogue

### Account opening and enquiry

| ID    | Pri. | Scenario                   | Input                                                                    | Expected                                                                                                                         |
|-------|------|----------------------------|--------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| TC-01 | MUST | Open SAVINGS, new customer | Karthik Rao, PAN <code>CKRPR7788Q</code> , <code>"15000.00"</code> (8.3) | 201; <code>501000000006</code> ; <code>newCustomer: true</code> ; ledger row "Initial deposit"; <code>AccountOpened</code> event |
| TC-02 | MUST | Deposit below minimum      | SAVINGS, <code>"500.00"</code>                                           | 400 <code>VALIDATION_ERROR</code> , field <code>initialDeposit</code>                                                            |
| TC-03 | MUST | Invalid PAN                | <code>"pan": "ABC123"</code>                                             | 400 <code>VALIDATION_ERROR</code> , field <code>customer.pan</code>                                                              |
| TC-04 | MUST | Under 18                   | <code>"dateOfBirth": "2010-01-01"</code>                                 | 400 <code>VALIDATION_ERROR</code> , field <code>customer.dateOfBirth</code>                                                      |

| ID    | Pri.   | Scenario                      | Input                                                                          | Expected                                                                                                                                                   |
|-------|--------|-------------------------------|--------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TC-05 | MUST   | Second SAVINGS                | Priya's PAN <code>ABCP51234K</code> , SAVINGS                                  | 409 <code>DUPLICATE_ACCOUNT_TYPE</code> ; no new rows                                                                                                      |
| TC-06 | MUST   | Existing customer, other type | Priya's PAN, CURRENT, <code>"5000.00"</code>                                   | 201; <code>customerId: 1</code> ; <code>newCustomer: false</code> ; still 4 customers in DB (with Karthik)                                                 |
| TC-07 | SHOULD | E-mail of another customer    | new PAN <code>DLMPK1122R</code> , e-mail <code>priya.sharma@example.com</code> | 409 <code>CUSTOMER_CONFLICT</code>                                                                                                                         |
| TC-08 | MUST   | Account enquiry + cache       | <code>GET /accounts/501000000001</code> twice                                  | 200 twice, balance <code>"55000.00"</code> ; X-Cache: MISS then HIT; HGETALL <code>bank:account:501000000001</code> shows the hash, <code>TTL ≤ 600</code> |
| TC-09 | MUST   | Unknown account               | <code>GET /accounts/509999999999</code>                                        | 404 <code>ACCOUNT_NOT_FOUND</code> ; no Redis key created                                                                                                  |

## Fund transfer

| ID    | Pri. | Scenario                | Input                                                                                                                                                                                                                                    | Expected                                                                                                                                                       |
|-------|------|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TC-10 | MUST | Successful transfer     | <code>...001 → ...002</code> , <code>"2500.00"</code> , key <code>K1</code>                                                                                                                                                              | 201 <code>COMPLETED</code> ; <code>fromBalanceAfter "52500.00"</code> ; DB balances 52,500.00 / 31,000.00; 2 ledger rows; <code>TransferCompleted</code> event |
| TC-11 | MUST | Idempotent replay       | repeat TC-10 exactly, key <code>K1</code>                                                                                                                                                                                                | Same status and body + <code>Idempotent-Replayed: true</code> ; balance still 52,500.00; still one <code>fund_transfer</code> row for <code>K1</code>          |
| TC-12 | MUST | Missing key             | TC-10 body, no <code>Idempotency-Key</code>                                                                                                                                                                                              | 400 <code>MISSING_IDEMPOTENCY_KEY</code>                                                                                                                       |
| TC-13 | MUST | Insufficient funds      | <code>...004 → ...002</code> , <code>"20000.00"</code>                                                                                                                                                                                   | 422 <code>INSUFFICIENT_FUNDS</code> ; both balances unchanged                                                                                                  |
| TC-14 | MUST | Same account            | <code>...001 → ...001</code> , <code>"10.00"</code>                                                                                                                                                                                      | 422 <code>SAME_ACCOUNT_TRANSFER</code>                                                                                                                         |
| TC-15 | MUST | Frozen receiver         | <code>...001 → ...005</code> , <code>"10.00"</code>                                                                                                                                                                                      | 422 <code>ACCOUNT_NOT_ACTIVE</code>                                                                                                                            |
| TC-16 | MUST | Bad amounts             | <code>"0.00"</code> , <code>"-5.00"</code> , <code>"10.555"</code> , <code>"abc"</code> , missing                                                                                                                                        | 400 <code>VALIDATION_ERROR</code> for each                                                                                                                     |
| TC-17 | MUST | Over per-transfer limit | <code>"200000.01"</code>                                                                                                                                                                                                                 | 400 <code>VALIDATION_ERROR</code>                                                                                                                              |
| TC-18 | MUST | Daily limit             | Open CURRENT for Meera Nair (PAN <code>EMNPN3344T</code> ) with <code>"100000.00"</code> ; from it send <code>"200000.00"</code> , <code>"200000.00"</code> , <code>"100000.00"</code> to <code>...003</code> , then <code>"1.00"</code> | first three 201; the fourth 422 <code>DAILY_LIMIT_EXCEEDED</code>                                                                                              |
| TC-19 | MUST | Unknown receiver        | <code>...001 → 509999999999</code>                                                                                                                                                                                                       | 404 <code>ACCOUNT_NOT_FOUND</code>                                                                                                                             |
| TC-20 | MUST | <b>Concurrency</b>      | 20 parallel × <code>"1000.00"</code> , <code>...004 → ...002</code> (10.5)                                                                                                                                                               | exactly 13 × 201 and 7 × 422; <code>...004</code> = 500.00; reconciliation zero rows                                                                           |

## Statement

| ID    | Pri. | Scenario                       | Input                                                                                                        | Expected                                                                                                                                               |
|-------|------|--------------------------------|--------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| TC-21 | MUST | September statement            | <code>...001</code> , 2026-09-01 → 2026-09-30, size 20                                                       | opening <code>0.00</code> , closing <code>55000.00</code> , credits <code>62000.00</code> , debits <code>7000.00</code> , 4 rows, newest first         |
| TC-22 | MUST | Current month                  | <code>...001</code> , 1st of this month → today                                                              | opening <code>55000.00</code> ; contains the TC-10 debit; closing = the account's current balance                                                      |
| TC-23 | MUST | Dates reversed                 | from 2026-09-30, to 2026-09-01                                                                               | 400 <code>VALIDATION_ERROR</code>                                                                                                                      |
| TC-24 | MUST | Range > 90 days                | 2026-01-01 → 2026-09-30                                                                                      | 400 <code>VALIDATION_ERROR</code>                                                                                                                      |
| TC-25 | MUST | Paging                         | TC-21 with size 3, page 0 then page 1                                                                        | page 0: 3 rows, <code>totalPages 2</code> (exactly 8.7); page 1: 1 row (initial deposit, <code>referenceNo null</code> )                               |
| TC-26 | MUST | Statement cache + invalidation | TC-21 twice; then a transfer from <code>...001</code> ; TC-21 again; <code>GET /accounts/501000000001</code> | <code>MISS</code> , <code>HIT</code> ; after the transfer <code>MISS</code> (new version) and the account shows the new balance with <code>MISS</code> |

## Events and resilience

| ID    | Pri.   | Scenario           | Input                                                                       | Expected                                                                                                                                                                                               |
|-------|--------|--------------------|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TC-27 | MUST   | Events published   | after TC-01 and TC-10                                                       | <code>AccountOpened</code> key <code>501000000006</code> ; <code>TransferCompleted</code> key <code>501000000001</code> , amount <code>"2500.00"</code> ; <code>notification_log</code> has 1 + 2 rows |
| TC-28 | SHOULD | Replay is harmless | stop the consumer; reset offsets to earliest (Lab D3.5); start it           | consumer re-reads everything; <code>SELECT count(*) FROM notification_log</code> unchanged                                                                                                             |
| TC-29 | SHOULD | Redis down         | <code>docker stop bank-redis</code> ; TC-08, a new transfer, then replay it | all 200/201 ( <code>X-Cache: MISS</code> ); replay still moves no money (DB key); then <code>docker start bank-redis</code>                                                                            |
| TC-30 | SHOULD | Kafka down         | <code>docker stop bank-kafka</code> ; a new transfer                        | 201 within about 5 s; ERROR log with the full event; <code>docker start bank-kafka</code>                                                                                                              |

## 10.4 Data-layer checks after a transfer

An API that returns 201 is not proof. Check all three stores:

SQL

```
-- PostgreSQL: the transfer and its two legs
SELECT reference_no, amount, status, idempotency_key FROM fund_transfer
ORDER BY created_at DESC LIMIT 1;

SELECT a.account_number, t.txn_type, t.amount, t.balance_after
FROM account_transaction t JOIN account a USING (account_id)
WHERE t.transfer_id = (SELECT transfer_id FROM fund_transfer ORDER BY created_at DESC LIMIT 1);
```

In `redis-cli`: the account key is gone (`EXISTS` returns 0, evicted after commit), the statement version went up by 1, and the idempotency key holds the stored response JSON.

REDIS-CLI

```
EXISTS bank:account:501000000001
GET bank:stmt-ver:501000000001
GET bank:idem:<your Idempotency-Key>
```

KAFKA CLI

```
# the last event on the topic
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic bank.transfer.completed.v1 \
  --from-beginning --formatter-property print.key=true --timeout-ms 5000
```

## 10.5 Concurrency test (TC-20)

Run this from Git Bash (or WSL / macOS / Linux) on a stack where `...004` still holds 13,500.00:

TERMINAL

```
#!/usr/bin/env bash
# 20 transfers of 1000.00 at the same moment from 501000000004 (13,500.00)
RUN=$(date +%s)
seq 1 20 | xargs -P 20 -I{} curl -s -o /dev/null -w "%{http_code}\n" \
  -X POST http://localhost:8080/api/v1/transfers \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: conc-$RUN-{}" \
  -d '{"fromAccountNumber":"501000000004","toAccountNumber":"501000000002","amount":"1000.00","remarks":"load test"}' \
  | sort | uniq -c
```

**Pass criteria:** the counts are exactly  $13 \times 201$  and  $7 \times 422$ , `...004` shows **500.00**, and `04_reconciliation.sql` returns zero rows for R1–R3. Any other result (for example 14 successes, a negative balance, a 500 error, or R1 rows) means the locking is wrong. Go back to Lab D1.3 and D1.7.

### TIP

Run it three times (reset in between). A race condition can pass once by luck. It rarely passes three times.

## 10.6 Postman tips

- One **collection** with folders *Accounts*, *Transfers*, *Statements*, *Resilience*; one **environment** with `baseUr1 = http://localhost:8080/api/v1`.
- For TC-10, a pre-request script creates the key once; TC-11 reuses it:

NODE.JS

```
// TC-10 > Pre-request
pm.collectionVariables.set("idemKey", pm.variables.replaceIn("{{$guid}}}");
// header in TC-10 and TC-11: Idempotency-Key: {{$idemKey}}
```

NODE.JS

```
// TC-10 > Post-response tests
pm.test("201 Created", () => pm.response.to.have.status(201));
const body = pm.response.json();
pm.test("balance after", () => pm.expect(body.fromBalanceAfter).to.eql("52500.00"));
pm.collectionVariables.set("refNo", body.referenceNo);
```

- Use the **Collection Runner** to run everything in order and export the run results as evidence.

### NOTE

In Windows PowerShell, `curl` is an alias for `Invoke-WebRequest` and JSON quoting is painful. Use **Git Bash**, `curl.exe` with a body file (`-d @body.json`), or Postman.

## 10.7 Test evidence sheet

Submit a table like this (one row per test case) with the outputs attached:

| ID    | Result | Evidence                                                              | Notes       |
|-------|--------|-----------------------------------------------------------------------|-------------|
| TC-10 | PASS   | Postman run #3, screenshot <code>tc10.png</code> , SQL output of 10.4 | —           |
| TC-20 | PASS   | <code>uniq -c</code> output, reconciliation output                    | ran 3 times |
| ...   |        |                                                                       |             |

# 11 Evaluation & Submission

## 11.1 Submission checklist

| ✓                        | Item                                                                                                               |
|--------------------------|--------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> | Git repository link; <code>main</code> branch builds and runs from a fresh clone                                   |
| <input type="checkbox"/> | <code>infra/docker-compose.yml</code> + <code>infra/db/*.sql</code> (+ any schema changes as new numbered scripts) |
| <input type="checkbox"/> | Application source with the layers of 3.5; no secrets committed ( <code>.env.example</code> only)                  |
| <input type="checkbox"/> | notification-service (same repo is fine)                                                                           |
| <input type="checkbox"/> | Postman collection + environment <b>or</b> <code>tests/api.sh</code> covering TC-01 – TC-30                        |
| <input type="checkbox"/> | <code>tests/concurrency.sh</code> (10.5) and its output, run on a fresh stack                                      |
| <input type="checkbox"/> | Reconciliation output after the full test run (R1–R3 empty)                                                        |
| <input type="checkbox"/> | Test evidence sheet (10.7)                                                                                         |
| <input type="checkbox"/> | <code>README.md</code> complete (11.2)                                                                             |

## 11.2 README contents

1. **What it is:** one paragraph.
2. **How to run:** prerequisites, `docker compose up -d`, topic creation, how to start the app and the consumer, the URL.  
A reviewer must get it running in under 10 minutes.
3. **Configuration:** every environment variable, its default and meaning.
4. **API:** a link to the collection, or a short endpoint table.
5. **Design decisions:** your own ADRs. Which of 3.7 did you keep or change, and why? How does the transfer avoid lost updates and deadlocks? What happens when Redis or Kafka is down?
6. **Known limitations:** be honest (for example "event lost if the app crashes between commit and publish; outbox not implemented").
7. **Test evidence:** where to find it.

## 11.3 Assessment rubric (100 points)

| Area                           | Pts | Excellent (full marks)                                                                                                               | Meets (about 60 %)                             | Below (about 20 %)                                                                   |
|--------------------------------|-----|--------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|--------------------------------------------------------------------------------------|
| <b>Data model and SQL</b>      | 20  | Constraints named and mapped to API errors; statement query uses the index (EXPLAIN shown); exact money types end to end             | Starter schema used correctly; queries correct | <code>float</code> / <code>double</code> for money; missing constraints; N+1 queries |
| <b>Transaction correctness</b> | 20  | TC-20 passes 3 of 3; lock order; <code>lock_timeout</code> ; deadlock/timeout mapped to 503; nothing external inside the transaction | TC-20 passes; locks present                    | Lost update, negative balance, or partial writes possible                            |
| <b>Redis</b>                   | 15  | Cache-aside with after-commit invalidation; versioned statement keys; idempotency with DB fallback; TC-29 passes                     | Cache-aside and idempotency work               | Stale balance after a transfer; keys without TTL; <code>KEYS *</code> in code        |

| Area                           | Pts | Excellent (full marks)                                                                   | Meets (about 60 %)            | Below (about 20 %)                                           |
|--------------------------------|-----|------------------------------------------------------------------------------------------|-------------------------------|--------------------------------------------------------------|
| <b>Kafka</b>                   | 15  | After-commit publish; correct keys; envelope; idempotent consumer; replay proven (TC-28) | Events published and consumed | Publish inside the transaction; no key; duplicates on replay |
| <b>API and validation</b>      | 10  | Contract of chapter 08 exactly; problem+json; correlation IDs                            | Main paths right              | Inconsistent errors; stack traces or SQL in responses        |
| <b>Testing and evidence</b>    | 10  | All test cases automated; evidence sheet; reconciliation after each run                  | Most MUST cases               | Manual only; no evidence                                     |
| <b>Code quality and README</b> | 5   | Clean layers; config by environment; README with ADRs and limitations                    | Runs from the README          | Cannot be run by a reviewer                                  |
| <b>Demo and viva</b>           | 5   | Explains trade-offs confidently, answers "what if" questions                             | Answers most questions        | Cannot explain own code                                      |

**Pass:** 60 points or more **and** not "Below" in *Transaction correctness*. A system that can lose money does not pass, whatever else it does well.

## 11.4 Demo script (10 minutes)

| Min  | Show                                                                                                                                            |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| 0–1  | Architecture diagram; <code>docker compose ps</code> ; health endpoint                                                                          |
| 1–3  | <b>Open an account</b> → the rows in psql (customer, account, ledger) → the event in Kafka UI → the <code>notification_log</code> row           |
| 3–5  | <b>Transfer</b> → 201 → <b>replay</b> the same key → <code>GET</code> account <code>MISS</code> then <code>HIT</code> → the Redis keys and TTLs |
| 5–7  | <b>Concurrency test live</b> (10.5) → <code>13 × 201, 7 × 422</code> → reconciliation empty                                                     |
| 7–8  | <b>Statement</b> for September with paging; the same numbers as Lab D1.5                                                                        |
| 8–10 | <b>Failure:</b> stop Redis, show the API still works; start it again. Then questions.                                                           |

## 11.5 Stretch goals (bonus, max +10)

| ID   | Stretch goal                                                                                                                                         | Bonus |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| S-1  | <b>Transactional outbox:</b> an <code>outbox_event</code> table written in the transfer transaction, plus a relay that publishes and marks rows sent | +4    |
| S-2  | Schema migrations with Flyway / Liquibase / Alembic / node-pg-migrate (run as <code>bank_admin</code> )                                              | +2    |
| S-3  | Feature F7: store <code>FAILED</code> transfers (separate transaction after rollback) and publish <code>TransferFailed</code>                        | +2    |
| S-4  | Optimistic locking on <code>account.version</code> with a retry loop; compare with the pessimistic version under TC-20                               | +3    |
| S-5  | Testcontainers integration tests, including the concurrency test, run by <code>mvn verify</code> / <code>npm test</code> / <code>pytest</code>       | +3    |
| S-6  | Redis rate limit: at most 5 transfers per minute per account ( <code>INCR</code> + <code>EXPIRE</code> , key <code>bank:rl:...</code> ) → 429        | +2    |
| S-7  | Dead-letter topic <code>&lt;topic&gt;.dlq</code> for messages the consumer cannot process                                                            | +2    |
| S-8  | JSON Schema or Avro event contracts with a schema registry                                                                                           | +2    |
| S-9  | Keyset (cursor) pagination for statements instead of <code>OFFSET</code>                                                                             | +2    |
| S-10 | <code>audit-service</code> : a second consumer group writing an audit table; show both groups' offsets                                               | +2    |

| ID   | Stretch goal                                                                                                               | Bonus |
|------|----------------------------------------------------------------------------------------------------------------------------|-------|
| S-11 | Containerise the application and add it to the Compose file ( <code>bank-kafka:19092</code> , <code>bank-postgres</code> ) | +2    |
| S-12 | OpenAPI 3 document generated from the code (springdoc / swagger-jsdoc / FastAPI's <code>/docs</code> )                     | +1    |

# 12 Trainer Guide

## 12.1 Running the five days

| Day | Morning (3.5 h)                                                                                         | Afternoon (3.5 h)                                                                  | Checkpoint at 17:00                                    |
|-----|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|--------------------------------------------------------|
| 1   | Kick-off: chapters 01–03 walkthrough (60 min); environment setup with the group (45 min); Lab D1.1–D1.3 | Labs D1.4–D1.7; whiteboard the race condition; two-terminal lock exercise in pairs | Each participant explains <code>02_transfer.sql</code> |
| 2   | Stack choice and skeleton (M2); short demo of a correlation-ID filter and a problem+json handler        | M3 accounts                                                                        | TC-01 – TC-09 (without cache)                          |
| 3   | M4 transfer. Start with a 20-minute recap of lock order.                                                | M5 statement; IST date boundaries demo                                             | TC-10, TC-12 – TC-19, TC-21 – TC-25                    |
| 4   | Labs D2.1–D2.3, then M6                                                                                 | Labs D3.1–D3.5, then M7                                                            | Cache HIT/MISS visible, events in Kafka UI             |
| 5   | M8: concurrency test, evidence, README                                                                  | Demos (10 min each) + viva (5 min each)                                            | Scores recorded                                        |

### TRAINER NOTE

Group participants **by stack** for help sessions (the Spring group, the Node group, the Python group), but keep the data-layer discussions **mixed**: the SQL, Redis keys and topics are the same for everybody, and explaining them across stacks deepens understanding.

## 12.2 Common mistakes, and how to spot them

| Symptom you see                                                             | Root cause                                                                                                             | Coach them to                                                                                                                        |
|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| TC-20 gives 14+ successes or a negative balance                             | Read balance → compute in code → <code>UPDATE SET balance = :new</code> without a lock                                 | Lock with <code>FOR UPDATE</code> and use <code>balance = balance - :amount</code> (Lab D1.7)                                        |
| Occasional <code>deadlock detected</code> under load                        | Locking sender first, then receiver                                                                                    | Lock both in one <code>ORDER BY account_id</code> query                                                                              |
| Transfer works, but sometimes only the debit is saved                       | No transaction, or <code>@Transactional</code> on a private method / self-invocation in Spring                         | Put the transaction on the public service method called from outside the bean                                                        |
| <code>0.30000000000000004</code> , or ₹2,500.5 shown as <code>2500.5</code> | <code>double</code> / JS <code>Number</code> / Python <code>float</code> for money; formatting not fixed to 2 decimals | <code>BigDecimal</code> / <code>Decimal</code> / string end to end; <code>setScale(2)</code> / <code>toFixed</code> only for display |
| Balance shown after a transfer is old                                       | Cache evicted <b>before</b> commit, or not at all                                                                      | Evict in the after-commit hook                                                                                                       |
| Event announces a transfer that then failed                                 | <code>send()</code> inside the transaction                                                                             | Publish after commit (7.5)                                                                                                           |
| The same SMS row twice after a restart                                      | Consumer inserts without <code>ON CONFLICT</code> and commits after                                                    | Use the primary key + <code>ON CONFLICT DO NOTHING</code> (Lab D3.4)                                                                 |
| Statement misses the last evening of the month                              | <code>BETWEEN</code> with dates, or a UTC day boundary                                                                 | Half-open range at 00:00 IST (5.5 warning)                                                                                           |

| Symptom you see                              | Root cause                                                                                     | Coach them to                                                                   |
|----------------------------------------------|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Every request slow when Redis is stopped     | No client timeout; exception not caught                                                        | 500 ms timeout + try/catch around every cache call                              |
| <code>permission denied for table ...</code> | App connects as <code>bank_app</code> but tries DDL (Hibernate <code>ddl-auto: update</code> ) | <code>ddl-auto: validate</code> ; schema comes from SQL scripts                 |
| Kafka consumer reads nothing                 | Topic not created, wrong group reused, <code>auto.offset.reset=latest</code>                   | Lab D3.1; check the group with <code>kafka-consumer-groups.sh --describe</code> |

## 12.3 Code-review checklist (Day 3 and Day 5)

- ☐ Money: `NUMERIC` ↔ `BigDecimal` / `Decimal` / string everywhere; no floats in DTOs, entities or events.
- ☐ One transaction per use case, in the service layer; nothing but SQL inside it.
- ☐ Both accounts locked in one ordered query; validation after locking.
- ☐ `lock_timeout` set; SQLSTATE 40P01 / 55P03 mapped to 503.
- ☐ Constraint names mapped to error codes; no SQL text in responses.
- ☐ Cache: all keys/TTLs in one place; every call guarded; eviction after commit.
- ☐ Idempotency: claim with `SET NX EX`; replay returns the stored response; DB fallback.
- ☐ Kafka: after-commit publish, keyed by account number, envelope with `eventId`; consumer idempotent.
- ☐ Config from environment; app connects as `bank_app`.
- ☐ Logs carry the correlation ID; no full PAN or event payloads with PII at INFO.

## 12.4 Viva questions with model answers

| # | Question                                                                            | A good answer mentions                                                                                                                                                         |
|---|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Why <code>NUMERIC(15,2)</code> and not <code>DOUBLE PRECISION</code> ?              | Binary floating point cannot represent 0.10 exactly; rounding errors accumulate; <code>NUMERIC</code> is exact decimal                                                         |
| 2 | What does <code>SELECT ... FOR UPDATE</code> do?                                    | Takes a row lock until commit/rollback; other writers and lockers of that row wait; plain readers are not blocked (MVCC)                                                       |
| 3 | Why lock in <code>account_id</code> order?                                          | Two transfers A→B and B→A would each hold one lock and wait for the other (deadlock); a global order makes a cycle impossible                                                  |
| 4 | We already have <code>CHECK (balance &gt;= 0)</code> . Why check funds in code too? | The code gives a clean 422 and a stored idempotent answer; the constraint is the safety net for bugs. Defence in depth.                                                        |
| 5 | What isolation level do you use, and what would <code>SERIALIZABLE</code> change?   | READ COMMITTED + explicit locks; <code>SERIALIZABLE</code> detects conflicts itself but aborts transactions (40001) that must then be retried                                  |
| 6 | Why keep both <code>balance</code> and the ledger?                                  | Fast balance reads and a simple constraint vs. a full audit trail and statements; reconciliation proves they agree (ADR-01)                                                    |
| 7 | What is cache-aside? What is the stale-data risk?                                   | App reads cache, falls back to DB, fills cache with TTL; risk is stale data until eviction, handled by after-commit deletes + TTL                                              |
| 8 | Why versioned statement keys instead of deleting them?                              | Many pages per account; deleting by pattern needs <code>KEYS</code> / <code>SCAN</code> (slow/blocking); a version bump makes old keys unreachable and the TTL cleans up       |
| 9 | Walk me through a retried transfer.                                                 | Same Idempotency-Key → <code>SET NX</code> fails → stored response returned with <code>Idempotent-Replayed</code> ; if Redis lost the key, the DB unique constraint catches it |

| #  | Question                                         | A good answer mentions                                                                                                                                 |
|----|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| 10 | Why publish to Kafka after commit, not before?   | Before commit you could announce a transfer that rolls back; after commit the only gap is a crash before publish (outbox fixes it)                     |
| 11 | Why is the key the account number?               | Same key → same partition → per-account ordering; spreads load across partitions                                                                       |
| 12 | What does at-least-once mean for your consumer?  | Events can arrive twice (crash before offset commit, rebalance, replay); consumer must be idempotent: PK + <code>ON CONFLICT DO NOTHING</code>         |
| 13 | What happens if Redis is down? If Kafka is down? | Redis: slower, still correct (DB fallback for reads and idempotency). Kafka: transfer succeeds, event logged for re-publishing (or kept in the outbox) |
| 14 | How do you know no money was created or lost?    | Reconciliation: R1 balance = ledger, R2 each transfer has both legs, R3 total balances = total deposits                                                |
| 15 | How would this scale to 10 app instances?        | Stateless app; DB locks and sequences coordinate; Redis shared; Kafka consumer group spreads partitions (max instances = partitions)                   |

## 12.5 Variations for other batches

- **Shorter (3 days):** skip the statement cache and TC-28–30; provide a skeleton project with M2 done.
- **Stronger batch:** make S-1 (outbox) and S-5 (Testcontainers) mandatory; add a second consumer group (S-10); require optimistic locking as a comparison (S-4).
- **Mixed-stack team project:** one team per stack builds the same API, and a final "contract test" runs one Postman collection against all three, showing that the data layer and the contract make the stack interchangeable.
- **Assessment variant:** give a buggy reference implementation (no lock, float money, evict before commit) and ask participants to find and fix the defects using the tests of chapter 10.

# A Reference Code Sketches

These sketches show the **hardest part**, the transfer, in each stack, so you can check your approach. They are deliberately incomplete: no controllers, DTO validation, error handler or configuration classes. The SQL in them is the SQL of Lab D1.3 and was run against the starter-kit database. Write your own version first, then compare.

## WATCH OUT

Copying these sketches will not get you a passing project, and the viva (12.4) will show it. Read them, understand the order of operations, and write your own.

## A.1 Java · Spring Boot (JdbcClient)

`JdbcClient` (Spring 6.1+) keeps the SQL visible, which suits a data-layer project. If you prefer JPA, the locking query is the repository method at the end of this section.

JAVA

```
@Service
public class TransferService {

    private static final BigDecimal DAILY_LIMIT = new BigDecimal("500000.00");
    private static final ZoneId IST = ZoneId.of("Asia/Kolkata");

    private final JdbcClient jdbc;
    private final ApplicationEventPublisher events;

    public TransferService(JdbcClient jdbc, ApplicationEventPublisher events) {
        this.jdbc = jdbc;
        this.events = events;
    }

    record Locked(long id, String number, BigDecimal balance, String status) {}
    record TransferRow(UUID id, String referenceNo, OffsetDateTime createdAt) {}

    @Transactional // ONE database transaction; any exception rolls it all back
    public TransferResult transfer(TransferRequest req, String idemKey) {
        jdbc.sql("SET LOCAL lock_timeout = '3s'").update();

        // 1. Lock both rows, ascending account_id
        List<Locked> rows = jdbc.sql("""
            SELECT account_id, account_number, balance, status
            FROM account
            WHERE account_number IN (:from, :to)
            ORDER BY account_id
            FOR UPDATE""")
            .param("from", req.fromAccountNumber())
            .param("to", req.toAccountNumber())
            .query((rs, n) -> new Locked(rs.getLong(1), rs.getString(2), rs.getBigDecimal(3),
rs.getString(4)))
            .list();
        Locked from = find(rows, req.fromAccountNumber()); // throws ACCOUNT_NOT_FOUND
        Locked to = find(rows, req.toAccountNumber());

        // 2. business rules - on the LOCKED rows
        if (!"ACTIVE".equals(from.status()) || !"ACTIVE".equals(to.status()))
            throw new BusinessException(ErrorCode.ACCOUNT_NOT_ACTIVE);
        if (from.balance().compareTo(req.amount()) < 0)
            throw new BusinessException(ErrorCode.INSUFFICIENT_FUNDS);
        BigDecimal sentToday = jdbc.sql("""
            SELECT COALESCE(SUM(amount), 0) FROM fund_transfer
            WHERE from_account_id = :id AND status = 'COMPLETED' AND created_at >= :since""")
            .param("id", from.id())
```

```

        .param("since", LocalDate.now(IST).atStartOfDay(IST).toOffsetDateTime())
        .query(BigDecimal.class).single();
    if (sentToday.add(req.amount()).compareTo(DAILY_LIMIT) > 0)
        throw new BusinessException(ErrorCode.DAILY_LIMIT_EXCEEDED);

    // 3. the transfer row - reference_no comes from its DEFAULT
    TransferRow t = jdbc.sql("""
        INSERT INTO fund_transfer (idempotency_key, from_account_id, to_account_id, amount, remarks)
        VALUES (:key, :from, :to, :amount, :remarks)
        RETURNING transfer_id, reference_no, created_at""")
        .param("key", idemKey).param("from", from.id()).param("to", to.id())
        .param("amount", req.amount()).param("remarks", req.remarks())
        .query((rs, n) -> new TransferRow(rs.getObject(1, UUID.class), rs.getString(2),
            rs.getObject(3, OffsetDateTime.class)))
        .single();

    // 4. move the money - the database does the arithmetic
    BigDecimal fromBal = move(from.id(), req.amount().negate());
    BigDecimal toBal = move(to.id(), req.amount());

    // 5. one DEBIT + one CREDIT
    String suffix = req.remarks() == null ? "" : " - " + req.remarks();
    ledger(from.id(), t.id(), "DEBIT", req.amount(), fromBal, "Transfer to " + to.number() + suffix);
    ledger(to.id(), t.id(), "CREDIT", req.amount(), toBal, "Transfer from " + from.number() + suffix);

    // 6. cache + Kafka happen AFTER COMMIT, in the Listener below
    TransferResult result = new TransferResult(t.id(), t.referenceNo(), t.createdAt(),
        from.number(), to.number(), req.amount(), req.remarks(), fromBal, toBal);
    events.publishEvent(new TransferCommitted(result));
    return result;
}

private BigDecimal move(long accountId, BigDecimal delta) {
    return jdbc.sql("""
        UPDATE account SET balance = balance + :delta, updated_at = now()
        WHERE account_id = :id
        RETURNING balance""")
        .param("delta", delta).param("id", accountId)
        .query(BigDecimal.class).single();
}

private void ledger(long accountId, UUID transferId, String type, BigDecimal amount,
    BigDecimal balanceAfter, String description) {
    jdbc.sql("""
        INSERT INTO account_transaction
        (account_id, transfer_id, txn_type, amount, balance_after, description)
        VALUES (:acc, :tid, :type, :amount, :after, :descr)""")
        .param("acc", accountId).param("tid", transferId).param("type", type)
        .param("amount", amount).param("after", balanceAfter).param("descr", description)
        .update();
}

private static Locked find(List<Locked> rows, String number) {
    return rows.stream().filter(r -> r.number().equals(number)).findFirst()
        .orElseThrow(() -> new BusinessException(ErrorCode.ACCOUNT_NOT_FOUND, number));
}
}

```

The after-commit listener runs only if the transaction committed:

JAVA

```

@Component
public class TransferAfterCommit {

    private final AccountCache cache;
    private final EventPublisher publisher;

    public TransferAfterCommit(AccountCache cache, EventPublisher publisher) {
        this.cache = cache;
        this.publisher = publisher;
    }

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void on(TransferCommitted e) {
        TransferResult r = e.result();
        cache.evictAccounts(r.fromAccountNumber(), r.toAccountNumber()); // DEL, guarded
        cache.bumpStatementVersions(r.fromAccountNumber(), r.toAccountNumber()); // INCR, guarded
        publisher.transferCompleted(r); // key = sender account
    }
}

```

Publishing does not block the request. A failure is logged with the full event:

JAVA

```

public void transferCompleted(TransferResult r) {
    String json = toJson(envelope("TransferCompleted", Map.of(
        "transferId", r.transferId().toString(),
        "referenceNo", r.referenceNo(),
        "fromAccountNumber", r.fromAccountNumber(),
        "toAccountNumber", r.toAccountNumber(),
        "amount", r.amount().toPlainString(), // money as a string
        "currency", "INR",
        "fromBalanceAfter", r.fromBalanceAfter().toPlainString(),
        "toBalanceAfter", r.toBalanceAfter().toPlainString())));
    kafka.send(topics.transferCompleted(), r.fromAccountNumber(), json)
        .whenComplete((ok, ex) -> {
            if (ex != null) log.error("Kafka publish failed - re-publish manually: {}", json, ex);
        });
}

```

The idempotency wrapper runs **around** the transactional call, never inside it:

JAVA

```

public StoredResponse execute(String idemKey, Supplier<StoredResponse> work) {
    String key = "bank:idem:" + idemKey;
    Boolean claimed;
    try {
        claimed = redis.opsForValue().setIfAbsent(key, "PENDING", Duration.ofHours(24)); // SET NX EX
    } catch (RuntimeException redisDown) {
        log.warn("Redis unavailable - relying on uq_transfer_idempotency", redisDown);
        return work.get();
    }
    if (Boolean.FALSE.equals(claimed)) {
        String stored = redis.opsForValue().get(key);
        if ("PENDING".equals(stored)) throw new BusinessException(ErrorCode.REQUEST_IN_PROGRESS);
        return StoredResponse.replayOf(stored); // same status + body, replay header
    }
    StoredResponse response;
    try {
        response = work.get(); // the @Transactional transfer
    } catch (BusinessException e) {
        response = StoredResponse.of(e); // 404 / 422 are final answers too
    } catch (RuntimeException e) {
        redis.delete(key); // unexpected failure: allow a retry
        throw e;
    }
    redis.opsForValue().setIfPresent(key, response.toJson(), Duration.ofHours(24)); // SET XX EX
    return response;
}

```

The notification consumer. Its write is idempotent:

JAVA

```

@KafkaListener(topics = {"bank.account.opened.v1", "bank.transfer.completed.v1"},
                groupId = "notification-service")
public void onEvent(String json) {
    JsonNode e;
    try {
        e = mapper.readTree(json);
    } catch (Exception poison) { // Jackson 3 (Boot 4) throws unchecked, Jackson 2 checked
        log.error("Unparseable event skipped: {}", json);
        return; // offset is committed - the partition keeps moving
    }
    UUID eventId = UUID.fromString(e.get("eventId").asText());
    JsonNode d = e.get("data");
    switch (e.get("eventType").asText()) {
        case "TransferCompleted" -> {
            save(eventId, d.get("fromAccountNumber").asText(), "TransferCompleted",
                "Rs %s debited from A/c %s to A/c %s. Ref %s. Avl bal Rs %s".formatted(
                    d.get("amount").asText(), mask(d.get("fromAccountNumber").asText()),
                    mask(d.get("toAccountNumber").asText()), d.get("referenceNo").asText(),
                    d.get("fromBalanceAfter").asText()));
            save(eventId, d.get("toAccountNumber").asText(), "TransferCompleted",
                "Rs %s credited to A/c %s from A/c %s. Ref %s. Avl bal Rs %s".formatted(
                    d.get("amount").asText(), mask(d.get("toAccountNumber").asText()),
                    mask(d.get("fromAccountNumber").asText()), d.get("referenceNo").asText(),
                    d.get("toBalanceAfter").asText()));
        }
        case "AccountOpened" -> save(eventId, d.get("accountNumber").asText(), "AccountOpened",
            "Welcome to Horizon Bank. Your %s A/c %s is open. Bal Rs %s".formatted(
                d.get("accountType").asText(), mask(d.get("accountNumber").asText()),
                d.get("initialDeposit").asText()));
        default -> log.warn("Unknown eventType {} skipped", e.get("eventType").asText());
    }
}

private void save(UUID eventId, String accountNumber, String type, String message) {
    jdbc.sql("""
        INSERT INTO notification_log (event_id, account_number, event_type, message)
        VALUES (:id, :acc, :type, :msg)
    """);
}

```

```

        ON CONFLICT (event_id, account_number) DO NOTHING""")
        .param("id", eventId).param("acc", accountNumber).param("type", type).param("msg", message)
        .update();
    }

    private static String mask(String accountNumber) {                // 501000000001 -> XX0001
        return "XX" + accountNumber.substring(accountNumber.length() - 4);
    }

```

JPA alternative for step 1:

```

public interface AccountRepository extends JpaRepository<Account, Long> {
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select a from Account a where a.accountNumber in :numbers order by a.id")
    List<Account> lockByNumbers(@Param("numbers") Collection<String> numbers);
}

```

JAVA

## A.2 Node.js · Express + pg + ioredis + kafkajs

Amounts stay **strings** from request to SQL. PostgreSQL does every comparison and every sum.

```

import { pool } from '../db.js';                // new pg.Pool({ max: 10 })
import { redis } from '../cache.js';            // new Redis(process.env.REDIS_URL, { commandTimeout: 500 })
import { producer } from '../kafka.js';         // kafka.producer({ idempotent: true }), connected at start-up
import { envelope } from '../events.js';
import { ApiError } from '../errors.js';
import { logger } from '../logger.js';

const DAILY_LIMIT = '500000.00';

export async function transfer(req, idemKey, correlationId) {
    const { fromAccountNumber, toAccountNumber, amount, remarks } = req;    // amount: "2500.00"
    const client = await pool.connect();
    let result;
    try {
        await client.query('BEGIN');
        await client.query("SET LOCAL lock_timeout = '3s'");

        // 1. Lock both rows, ascending account_id
        const { rows } = await client.query(
            `SELECT account_id, account_number, balance, status FROM account
             WHERE account_number = ANY($1) ORDER BY account_id FOR UPDATE`,
            [[fromAccountNumber, toAccountNumber]]);
        const from = rows.find((r) => r.account_number === fromAccountNumber);
        const to = rows.find((r) => r.account_number === toAccountNumber);
        if (!from) throw new ApiError(404, 'ACCOUNT_NOT_FOUND', `Account ${fromAccountNumber} not found`);
        if (!to) throw new ApiError(404, 'ACCOUNT_NOT_FOUND', `Account ${toAccountNumber} not found`);
        if (from.status !== 'ACTIVE' || to.status !== 'ACTIVE') throw new ApiError(422, 'ACCOUNT_NOT_ACTIVE');

        // 2. money checks in SQL - never with JavaScript numbers
        const { rows: [chk] } = await client.query(
            `SELECT $1::numeric <= $2::numeric AS enough,
             (SELECT COALESCE(SUM(amount), 0) FROM fund_transfer
              WHERE from_account_id = $3 AND status = 'COMPLETED'
              AND created_at >= date_trunc('day', now()) AT TIME ZONE 'Asia/Kolkata') AT TIME ZONE
             'Asia/Kolkata'
             + $1::numeric <= $4::numeric AS within_limit`,
            [amount, from.balance, from.account_id, DAILY_LIMIT]);
        if (!chk.enough) throw new ApiError(422, 'INSUFFICIENT_FUNDS');
        if (!chk.within_limit) throw new ApiError(422, 'DAILY_LIMIT_EXCEEDED');

        // 3-5. transfer row, balances, ledger
        const { rows: [t] } = await client.query(
            `INSERT INTO fund_transfer (idempotency_key, from_account_id, to_account_id, amount, remarks)
             VALUES ($1, $2, $3, $4, $5) RETURNING transfer_id, reference_no, created_at`,

```

NODE.JS

```

    [idemKey, from.account_id, to.account_id, amount, remarks ?? null]);
const fromBal = (await client.query(
  'UPDATE account SET balance = balance - $2, updated_at = now() WHERE account_id = $1 RETURNING balance',
  [from.account_id, amount])).rows[0].balance;
const toBal = (await client.query(
  'UPDATE account SET balance = balance + $2, updated_at = now() WHERE account_id = $1 RETURNING balance',
  [to.account_id, amount])).rows[0].balance;
const suffix = remarks ? ` - ${remarks}` : '';
await client.query(
  `INSERT INTO account_transaction (account_id, transfer_id, txn_type, amount, balance_after, description)
  VALUES ($1, $3, 'DEBIT', $4, $5, $7), ($2, $3, 'CREDIT', $4, $6, $8)\`,
  [from.account_id, to.account_id, t.transfer_id, amount, fromBal, toBal,
  `Transfer to ${toAccountNumber}${suffix}`, `Transfer from ${fromAccountNumber}${suffix}`]);

await client.query('COMMIT');
result = { transferId: t.transfer_id, referenceNo: t.reference_no, status: 'COMPLETED',
  fromAccountNumber, toAccountNumber, amount, currency: 'INR', remarks,
  fromBalanceAfter: fromBal, toBalanceAfter: toBal, createdAt: t.created_at };
} catch (err) {
  await client.query('ROLLBACK').catch(() => {});
  throw err; // error middleware maps ApiError / SQLSTATE
} finally {
  client.release(); // always give the connection back
}

// ---- after COMMIT: cache, then event (never inside the transaction) ----
try {
  await redis.multi()
    .del(`bank:account:${fromAccountNumber}`, `bank:account:${toAccountNumber}`)
    .incr(`bank:stmt-ver:${fromAccountNumber}`).incr(`bank:stmt-ver:${toAccountNumber}`)
    .exec();
} catch (err) {
  logger.warn({ err }, 'cache invalidation failed');
}
const { toBalanceAfter, ...publicResult } = result;
const event = envelope('TransferCompleted', correlationId, { ...publicResult, toBalanceAfter });
try {
  await producer.send({ topic: 'bank.transfer.completed.v1',
    messages: [{ key: fromAccountNumber, value: JSON.stringify(event) }] });
} catch (err) {
  logger.error({ err, event }, 'Kafka publish failed - re-publish manually');
}
return publicResult; // the API response has no toBalanceAfter
}

```

### A.3 Python · FastAPI + psycopg 3 + redis-py + confluent-kafka

`NUMERIC` columns arrive as `decimal.Decimal`; keep every amount a `Decimal` (Pydantic does this for fields typed `Decimal`).

PYTHON

```

import json
import logging
from datetime import datetime
from decimal import Decimal
from zoneinfo import ZoneInfo

import redis
from confluent_kafka import Producer
from psycopg.rows import dict_row
from psycopg_pool import ConnectionPool

from .config import settings
from .errors import ApiError
from .events import envelope

log = logging.getLogger(__name__)

```

```

IST = ZoneInfo("Asia/Kolkata")
DAILY_LIMIT = Decimal("500000.00")

pool = ConnectionPool(settings.database_url, max_size=10, kwargs={"row_factory": dict_row})
cache = redis.Redis.from_url(settings.redis_url, socket_timeout=0.5, decode_responses=True)
producer = Producer({"bootstrap.servers": settings.kafka_bootstrap_servers,
                    "enable.idempotence": True, "acks": "all"})

def transfer(req, idem_key: str, correlation_id: str) -> dict:
    since = datetime.now(IST).replace(hour=0, minute=0, second=0, microsecond=0)
    with pool.connection() as conn:
        with conn.transaction():
            # BEGIN ... COMMIT, ROLLBACK on exception
            conn.execute("SET LOCAL lock_timeout = '3s'")
            rows = conn.execute(
                """SELECT account_id, account_number, balance, status FROM account
                   WHERE account_number = ANY(%s) ORDER BY account_id FOR UPDATE""",
                ([req.from_account_number, req.to_account_number],)).fetchall()
            by_no = {r["account_number"]: r for r in rows}
            src, dst = by_no.get(req.from_account_number), by_no.get(req.to_account_number)
            if src is None:
                raise ApiError(404, "ACCOUNT_NOT_FOUND", f"Account {req.from_account_number} not found")
            if dst is None:
                raise ApiError(404, "ACCOUNT_NOT_FOUND", f"Account {req.to_account_number} not found")
            if src["status"] != "ACTIVE" or dst["status"] != "ACTIVE":
                raise ApiError(422, "ACCOUNT_NOT_ACTIVE")
            if src["balance"] < req.amount:
                raise ApiError(422, "INSUFFICIENT_FUNDS")
            sent_today = conn.execute(
                """SELECT COALESCE(SUM(amount), 0) AS s FROM fund_transfer
                   WHERE from_account_id = %s AND status = 'COMPLETED' AND created_at >= %s""",
                (src["account_id"], since)).fetchone()["s"]
            if sent_today + req.amount > DAILY_LIMIT:
                raise ApiError(422, "DAILY_LIMIT_EXCEEDED")

            t = conn.execute(
                """INSERT INTO fund_transfer (idempotency_key, from_account_id, to_account_id, amount,
remarks)
                VALUES (%s, %s, %s, %s, %s) RETURNING transfer_id, reference_no, created_at""",
                (idem_key, src["account_id"], dst["account_id"], req.amount, req.remarks)).fetchone()
            from_bal = conn.execute(
                "UPDATE account SET balance = balance - %s, updated_at = now() "
                "WHERE account_id = %s RETURNING balance", (req.amount, src["account_id"])).fetchone()
["balance"]
            to_bal = conn.execute(
                "UPDATE account SET balance = balance + %s, updated_at = now() "
                "WHERE account_id = %s RETURNING balance", (req.amount, dst["account_id"])).fetchone()
["balance"]
            suffix = f" - {req.remarks}" if req.remarks else ""
            conn.execute(
                """INSERT INTO account_transaction
                   (account_id, transfer_id, txn_type, amount, balance_after, description)
                   VALUES (%s, %s, 'DEBIT', %s, %s, %s), (%s, %s, 'CREDIT', %s, %s, %s)""",
                (src["account_id"], t["transfer_id"], req.amount, from_bal,
                 f"Transfer to {req.to_account_number}{suffix}",
                 dst["account_id"], t["transfer_id"], req.amount, to_bal,
                 f"Transfer from {req.from_account_number}{suffix}"))
            # Leaving the transaction block COMMITTED

            # ---- after COMMIT ----
            try:
                p = cache.pipeline()
                p.delete(f"bank:account:{req.from_account_number}", f"bank:account:{req.to_account_number}")
                p.incr(f"bank:stmt-ver:{req.from_account_number}")
                p.incr(f"bank:stmt-ver:{req.to_account_number}")
                p.execute()
            except redis.RedisError as exc:
                log.warning("cache invalidation failed: %s", exc)

```

```

data = {"transferId": str(t["transfer_id"]), "referenceNo": t["reference_no"],
        "fromAccountNumber": req.from_account_number, "toAccountNumber": req.to_account_number,
        "amount": f"{req.amount:.2f}", "currency": "INR",
        "fromBalanceAfter": f"{from_bal:.2f}", "toBalanceAfter": f"{to_bal:.2f}",
        "remarks": req.remarks}
event = envelope("TransferCompleted", correlation_id, data)

def on_delivery(err, _msg):
    if err is not None:
        log.error("Kafka publish failed - re-publish manually: %s", json.dumps(event))

producer.produce("bank.transfer.completed.v1", key=req.from_account_number,
                 value=json.dumps(event), on_delivery=on_delivery)
producer.poll(0) # serve delivery callbacks

return {"**data", "status": "COMPLETED", "createdAt": t["created_at"].isoformat()}

```

**NOTE**

Call `producer.flush(10)` when the application shuts down (FastAPI lifespan) so buffered events are not lost. In Node.js, call `await producer.disconnect()`; in Spring Boot the `KafkaTemplate` is closed for you.

# B Glossary & References

## B.1 Glossary

| Term                          | Meaning in this project                                                                                                                                                                                                                        |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ACID</b>                   | Atomicity, Consistency, Isolation, Durability: the guarantees a database transaction gives. A transfer is all-or-nothing (A), keeps constraints true (C), is not disturbed by parallel transfers (I), and survives a crash once committed (D). |
| <b>At-least-once</b>          | A delivery guarantee: every event arrives, possibly more than once. Needs an idempotent consumer.                                                                                                                                              |
| <b>Cache-aside</b>            | Read the cache; on a miss read the database and fill the cache. The application, not the cache, talks to the database.                                                                                                                         |
| <b>Consumer group</b>         | A set of consumer instances that share a topic's partitions; each event is processed by one instance of the group. Different groups each get every event.                                                                                      |
| <b>Correlation ID</b>         | An ID passed through a request, its logs and its events, so one business action can be traced across systems.                                                                                                                                  |
| <b>Deadlock</b>               | Two transactions each wait for a lock the other holds. PostgreSQL aborts one ( <code>40P01</code> ). Avoided by a global lock order.                                                                                                           |
| <b>Defence in depth</b>       | Enforcing a rule in more than one layer (API validation <i>and</i> a DB constraint).                                                                                                                                                           |
| <b>Event envelope</b>         | The standard outer fields of every event ( <code>eventId</code> , <code>eventType</code> , <code>occurredAt</code> ...) around the event-specific <code>data</code> .                                                                          |
| <b>Idempotency</b>            | Doing an operation twice has the same effect as doing it once. Here: the same <code>Idempotency-Key</code> never moves money twice.                                                                                                            |
| <b>Invariant</b>              | A condition that must always be true, e.g. "total balances = total deposits". Checked by reconciliation.                                                                                                                                       |
| <b>IST</b>                    | Indian Standard Time, UTC+05:30, zone ID <code>Asia/Kolkata</code> . The bank's business time zone.                                                                                                                                            |
| <b>Isolation level</b>        | How much concurrent transactions can see of each other. PostgreSQL default: READ COMMITTED.                                                                                                                                                    |
| <b>Ledger</b>                 | The append-only list of balance movements ( <code>account_transaction</code> ), each with its <code>balance_after</code> .                                                                                                                     |
| <b>Lost update</b>            | Two transactions read the same value and both write a result based on it; one write is silently lost (Lab D1.7).                                                                                                                               |
| <b>MVCC</b>                   | Multi-Version Concurrency Control: PostgreSQL readers see a snapshot and never block writers. That is why <code>FOR UPDATE</code> is needed for read-modify-write.                                                                             |
| <b>Offset</b>                 | The position of a message in a Kafka partition. A consumer group stores the next offset it will read.                                                                                                                                          |
| <b>Optimistic locking</b>     | No lock while reading; the update checks a <code>version</code> column and fails if someone else changed the row, so the caller retries.                                                                                                       |
| <b>Outbox (transactional)</b> | Writing the event to an <code>outbox</code> table in the same DB transaction as the business change; a relay publishes it later. Closes the commit-then-crash gap.                                                                             |
| <b>Partition</b>              | A Kafka topic is split into partitions; order is guaranteed only within one. The key decides the partition.                                                                                                                                    |
| <b>PAN</b>                    | Permanent Account Number: India's 10-character tax ID ( <code>AAAAA9999A</code> ). Used as the customer's business identity here.                                                                                                              |
| <b>Pessimistic locking</b>    | Locking rows before changing them ( <code>SELECT ... FOR UPDATE</code> ) so others wait.                                                                                                                                                       |
| <b>Problem Details</b>        | RFC 9457 standard JSON error format ( <code>type</code> , <code>title</code> , <code>status</code> , <code>detail</code> , <code>instance</code> ).                                                                                            |

| Term                    | Meaning in this project                                                                                                           |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>Reconciliation</b>   | Queries that compare independent records (balance vs. ledger, transfers vs. ledger legs) to prove integrity.                      |
| <b>Replay</b>           | Re-reading old Kafka events by moving a consumer group's offsets back.                                                            |
| <b>Sequence</b>         | A database object that hands out unique increasing numbers, even across many app instances. Never rolls back, so gaps are normal. |
| <b>System of record</b> | The authoritative source of a piece of data: PostgreSQL here.                                                                     |
| <b>TTL</b>              | Time To Live: seconds until Redis deletes a key automatically.                                                                    |

## B.2 References

- PostgreSQL 16 documentation: *Explicit Locking* (row-level locks, deadlocks), *Transaction Isolation*, *Constraints*, *Indexes: Multicolumn Indexes*, *pgbench*. <https://www.postgresql.org/docs/16/>
- Redis documentation: `SET` (NX, XX, EX, KEeptTL), *Key eviction*, `SCAN`, data types. <https://redis.io/docs/>
- Apache Kafka 4.x documentation: *Producer configs* (`acks`, `enable.idempotence`), *Consumer configs*, *Consumer groups*, `kafka-consumer-groups.sh`. <https://kafka.apache.org/documentation/>
- RFC 9457, *Problem Details for HTTP APIs*. <https://www.rfc-editor.org/rfc/rfc9457>
- IETF draft, *The Idempotency-Key HTTP Header Field* (httpapi working group).
- Spring Framework reference: *Transaction Management*, `@TransactionalEventListener`, `JdbcClient`. Spring for Apache Kafka reference. <https://docs.spring.io/>
- node-postgres (<https://node-postgres.com>), ioredis, KafkaJS documentation.
- psycopg 3 (<https://www.psycopg.org/psycopg3/docs/>), redis-py, confluent-kafka-python documentation.